

Future-Proofing Verifiable Credentials: Quantifying the Performance Tax of Cryptographic Agility in Post-Quantum Anonymous Credentials

A Thesis Submitted to the Department of Computer Science and Communications Engineering, the Graduate School of Fundamental Science and Engineering of Waseda University in Partial Fulfillment of the Requirements for the Degree of Master of Engineering

Takumi Otsuka

The Department of Computer Science and Communications Engineering, the
Graduate School of Fundamental Science and Engineering of Waseda University
5124FG15-6

Submission Date : , 2026

Research Guidance: : Research on Cryptographic Protocols

Advisor: : Prof. Kazue Sako

Abstract

Post-quantum anonymous credentials need a signature whose verification can be proved in zero knowledge. We ask whether one such scheme, the algebraic-hash PLUM signature, can be verified inside a general-purpose zero-knowledge virtual machine on a consumer machine, and we report a frontier result with a clear sign. Without a custom precompile, a single PLUM verification exhausts memory on 24 GB and cannot be proved at all. With a bespoke large-field Griffin- $\mathbb{F}_p^{192}$ precompile that removes the field-emulation tax, the same verification is proved in 32.5 minutes on SP1 at the 80-bit setting, the largest that completes within the memory bound; the paper’s 128-bit setting does not terminate, which is itself part of the frontier. Feasibility for the standalone signature is therefore reached, but the binding constraint then moves up the stack. The zero-knowledge wrap that anonymity requires exhausts memory at 24 GB, and the only wrap that fits is pairing-based and so not post-quantum. Deployable post-quantum anonymous credentials are not reached on this stack, and we locate exactly where the wall sits and what would move it. The proof we run is a succinct receipt and is not itself zero-knowledge. We also report update churn for both regimes; its wall-clock cost is sub-second against a transparent non-preprocessing argument, so the recompile burden does not separate the two substrates and survives only as a re-audit footprint argument conditional on the preprocessing model. We measured each substrate on consumer hardware (Apple M5 Pro, 24 GB), against a Loquat-instantiated Aurora static-circuit reference.

Contents

1	Introduction	8
1.1	Background	8
1.2	The Current State of Post-Quantum Anonymous Credentials	8
1.3	The Hidden Cost: Cryptographic Rigidity	9
1.4	Zero-Knowledge Virtual Machines as a Structural Alternative	9
1.5	Contributions of This Thesis	10
1.6	Scope and Non-Goals	12
1.7	Thesis Organisation	13
2	Related Work	14
2.1	Post-Quantum Signatures	14
2.2	Anonymous Credentials	15
2.3	Transparent Succinct Arguments and Low-Degree Testing	15
2.4	Zero-Knowledge Virtual Machines	16
2.5	Precompile-Based Cost Reduction	16
2.6	Evaluation Methodology and the Gap This Work Addresses	17
3	Preliminaries	19
3.1	Notation	19
3.2	Cryptographic Primitives	19
3.2.1	Power residue PRF	19
3.2.2	Algebraic hash function	20
3.2.3	Merkle commitment	20
3.3	Digital Signature Scheme	21
3.3.1	Loquat: a Legendre-PRF post-quantum signature with low R1CS verification cost	22
3.3.2	PLUM: a power-residue-PRF post-quantum signature with low R1CS verification cost	23
3.4	zkSNARK	25
3.4.1	Aurora: Transparent Succinct Arguments for R1CS	26
3.5	Anonymous Credential	27
3.5.1	Blockchain-based Digital Education Credentials (BDEC)	29
3.6	Zero-Knowledge Virtual Machines	33
3.6.1	RISC Zero	35
3.6.2	SP1	35
3.6.3	Static-circuit and zkVM proof systems as comparators	35
4	Theoretical Foundations of the Arithmetic Mismatch	37
4.1	Cost Dimensions	37
4.2	The Field-Mismatch Tax	37
4.3	Update Churn	38
4.4	Cost Decomposition of BDEC Relations	40

5	Construction: The Precompile Suite and PLUM-in-BDEC	41
5.1	Scope of the Implementation	41
5.2	The Field-Arithmetic Substrate: $\mathbb{F}_{p^{192}}$ Multiplication via the Host Precompile.....	42
5.3	The Bespoke Griffin- $\mathbb{F}_p^{192}$ Permutation AIR (SP1)	43
5.4	The Sponge and Compression Wrapper	44
5.5	Soundness of the Precompile Suite, and Two Open Questions	44
5.6	Integration into the BDEC Credential-Generation Relation	45
6	Security Analysis	47
6.1	Security Model and Threat Model	47
6.2	Security Preservation	48
6.3	The Zero-Knowledge Barrier as a Finding	51
7	Evaluation	53
7.1	Measurement Methodology	53
7.2	Experimental Setup	53
7.2.1	Settings Justification	53
7.2.2	Reproducibility	54
7.3	PLUM Verification With and Without the Precompile	54
7.4	The Circuit-SNARK Reference Point.....	56
7.5	Cost Attribution	57
7.6	PLUM-in-BDEC and the Consumer-Hardware Frontier.....	58
7.6.1	An internal-verifier rejection rather than a resource frontier.....	58
7.6.2	The genuine resource frontier	59
7.7	A Cost Model and the Substrate Crossover.....	59
7.7.1	An additive cost model for the BDEC relations	60
7.7.2	Validation against the measured points	60
7.7.3	The substrate crossover	61
7.8	Update Churn	62
7.9	What the Measurements Establish	63
7.10	Threats to Validity	64
7.11	Summary	65
8	Discussion	66
8.1	A Substrate-Selection Rule	66
8.2	The Security–Deployability Tension.....	67
8.3	What Generalises and What Does Not.....	67
8.4	Comparison to Prior Evaluations.....	67
9	Conclusion	69
9.1	Main Findings	69
9.2	Answer to the Research Question	69
9.3	Limitations	70
9.4	Future Work	70
9.5	Closing	70

Acknowledgement	72
References	73
A Formal Development of the Security Preservation	78
A.1 Building-block properties	78
A.2 Relation preservation	79
A.3 Proof of the preservation theorem.....	79
B Evidence for Premise (T2): Specification-Level Soundness of the Griffin-$\mathbb{F}_p^{192}$ AIR	81
B.1 Setup.....	81
B.2 Soundness	81
B.3 Per-family lemmas	82
B.4 Completeness	83
B.5 The argument layer	83
B.6 The open step	84

List of Figures

- 1 One relation, two outer provers. The same BDEC verification relation is proved *either* by a static Aurora circuit (the BDEC baseline) *or* by a general-purpose zkVM (this thesis), and the two are alternatives: the zkVM does not re-prove an Aurora proof. The figure locates the deployability wall: the static Aurora circuit reaches a post-quantum zero-knowledge proof, whereas the zkVM does not reach that proof on consumer hardware. The agility difference between the two is a footprint distinction that this thesis measures and finds does not translate into a per-proof wall-clock advantage. 10
- 2 Where the field-mismatch tax is incurred and what the precompile absorbs. Left: each $\mathbb{F}_p^{199}$ operation expands into multi-limb arithmetic that, un-precompiled, does not complete (OOM at 1m45s). Right: a syscall routes it to a native chip, the bespoke Griffin AIR on SP1 or `sys_bigint` on RISC Zero (*not* one artefact), bound to the trace by a cross-table lookup; the chip's cost is charged host-side, so a guest-cycle saving need not be a wall-clock one. 43
- 3 The dual obstruction. The proof feasible to generate (left, the bare STARK receipt) is not zero-knowledge, hence not anonymous; the proof that would be zero-knowledge (right, a Groth16/PLONK wrap) is blocked by two *independent* prongs: resource (OOM at ≈ 20 GB) and primitive (pairing-based, not post-quantum, which more memory cannot fix). Solid edges mark a produced step, dashed a blocked one. 51
- 4 Prove time on the configurations that complete ($\lambda = 80$, M5 Pro 24 GB, log scale). The bespoke Griffin precompile (32.5 min) is slower than the SHA-3 control (13.3 min) at matched security, and RISC Zero's `sys_bigint` route is slower again (6 h 19 m); these are different hash and substrate mechanisms, so the bars are not one artefact. The precompile-free baseline does not complete (DNF, OOM at ≈ 1 m45s) and so has no bar. 55

List of Tables

1	Evaluation platform. All prove-mode measurements are local single-machine runs with no network or dev-mode acceleration unless stated.	54
2	PLUM single-signature verification, prove mode, $\lambda = 80$, M5 Pro 24 GB. The precompile-free baseline does not complete; the precompile arm completes in 32.53 min. The SHA-3 arm is a <i>control</i> (what a standard-hash precompile alone buys), not a deployable PLUM variant. Single runs. [†] SHA-3 control at $\lambda = 80$. [‡] RISC Zero receipt-verify wall-clock not separately logged; the receipt verified.	55
3	Aurora over the BDEC R1CS, M5 Pro 24 GB, single run. This is the <i>Loquat</i> -instantiated circuit over $\mathbb{F}_{p^2}$ (127-bit base) [1], <i>not</i> the PLUM workload measured elsewhere; the two are not a same-scheme comparison. Prover time includes Aurora parameter setup but is not a recompile-time measurement (§7.8). Achieved security 107.5 bits, below the 128 target. Verification ACCEPT.	56
4	Cycle-level cost attribution, PLUM verification, SHA-3 hasher, $\lambda = 128$, <i>execute mode</i> . Routing field arithmetic through the host precompile (and removing redundant reductions) cuts 37.9% of SP1 cycles and 70.5% of RISC Zero cycles; RISC Zero’s larger reduction reflects its heavier per-multiplication baseline (6,625 vs. 1,826 cycles/mul).	57
5	BDEC credential generation, RISC Zero, $\lambda = 80$, M5 Pro 24 GB. Execute mode validates functional correctness (9.4×10^9 cycles, $\approx 4.69 \times 10^9$ per signature verification). The succinct-prove run yielded no receipt; the cause was an internal-verifier rejection, not a resource limit (§7.6.1), and is not used as a feasibility result.	58
6	Proof-mode feasibility matrix. The modes <i>produced</i> on the target hardware are not zero-knowledge; the only zero-knowledge mode the toolchain exposes (row 3, a pairing wrap) is neither post-quantum nor produced within the memory bound, which is the dual obstruction. [†] Aurora is zero-knowledge-capable but the measured run had ZK disabled and ran below $\lambda = 128$. The SHA-3 row is a control, not a deployable PLUM variant.	59
7	Circuit/parameter regeneration footprint ($N_{\text{circ}}, N_{\text{param}}$ of Definition 4.3). The static-circuit regime regenerates and re-audits its R1CS and constraint-dependent parameters on every localised change; the zkVM absorbs all changes as guest-program edits, leaving the universal relation unchanged ($N_{\text{circ}} = N_{\text{param}} = 0$). The components $N_{\text{src}}, N_{\text{audit}}$ are not integer-countable and are discussed in prose.	62
8	Per-change recompile cost R_{static} for a non-preprocessing (Aurora) versus a preprocessing (Fractal) transparent argument, same Loquat-BDEC R1CS (127-bit field), M5 Pro 24 GB. Aurora has no per-circuit indexer (R_{static} flat at $\sim \mu s$); Fractal’s indexer is superlinear and exceeds 24 GB at the BDEC size (OOM mid-indexer). The zkVM regenerates no circuit.	63

1 Introduction

1.1 Background

Digital credentials are increasingly used to verify educational achievements, professional qualifications, and trust relationships in online services. Several recent incidents point to a credibility crisis in the educational-credential space: a private university in India reportedly sold roughly 36,000 fake degrees, large-scale document-forgery operations have led to student deportations, and the LinkedIn breach of 2021 exposed approximately 700 million user profiles to the dark web [1]. Together with the rise of micro-credentials, which produce fragmented records of learning across institutions and platforms, these incidents motivate a credential system that is simultaneously *verifiable*, *privacy-preserving*, and *long-lived*.

A long-lived credential system must satisfy two further constraints. First, it must remain secure against adversaries equipped with large-scale quantum computers; this rules out classical anonymous-credential constructions whose security reduces to the discrete-logarithm or RSA assumption. Second, it must remain practically deployable as the surrounding cryptographic and policy environment evolves: the underlying signature scheme may be updated as standards mature, security parameters may need retuning, and disclosure policies may change as institutional rules evolve. A credential system that satisfies the first constraint but is brittle under the second is unsuitable for real-world deployment.

1.2 The Current State of Post-Quantum Anonymous Credentials

Two representative lines of work have attempted to construct post-quantum anonymous-credential systems (the broader lattice-native and PQ-token literature is surveyed in Section 2). The first follows the lattice-based approach exemplified by Jeudy et al. at Crypto 2023 [2], leveraging the hardness of the Short Integer Solution (SIS) and Learning With Errors (LWE) problems. While this approach achieves quantum resistance, it currently supports only a single message per credential and yields large proof and key sizes that make repeated showings cumbersome.

The second line of work, exemplified by the recent Blockchain-based Digital Education Credential (BDEC) framework of Li, Zhang et al. [1], takes a symmetric-primitive-based approach. BDEC instantiates a generic anonymous-credential construction with the Loquat post-quantum signature scheme [3], whose verification predicate is comparatively cheap to encode in rank-1 constraint systems (R1CS), and the Aurora transparent zkSNARK [4], which proves the R1CS relation in zero-knowledge. The resulting system achieves unforgeability, anonymity, unlinkability, conditional linkability, and revocability, and, per the authors’ analysis, outperforms the lattice-based approach [1].

A more recent post-quantum signature, PLUM [5], is a successor to Loquat that replaces the Legendre PRF with a t -th power residue PRF and the FRI low-degree test with the STIR protocol. At the 128-bit security level, PLUM signatures are approximately $1.5\times$ smaller than Loquat’s, and PLUM verification is expected to be up to $4\times$ faster, while requiring roughly $1.28\times$ fewer R1CS constraints [5]. Because PLUM shares Loquat’s overall structure, a symmetric-primitive PRF assumption verified through a transparent low-degree test, while reporting lower verification cost at matching security, it is the signature primitive we adopt for the zkVM instantiation studied in this thesis. PLUM is a research signature, not yet standardised, chosen for its SNARK-friendly verification structure. The NIST post-quantum signature standards (ML-DSA, SLH-DSA, FN-DSA) remain the deployment

baselines. Their lattice and hash verification predicates are precisely the ones that are expensive to arithmetise, and that is why a symmetric-primitive design is studied here.

1.3 The Hidden Cost: Cryptographic Rigidity

Both Loquat and PLUM are designed to be efficiently verifiable inside a zkSNARK. The dominant zkSNARK choice for BDEC-style constructions is a relation-specific transparent argument such as Aurora, in which the verification predicate is compiled into a fixed R1CS at deployment time. This compilation is what makes the resulting system efficient: the proof system can be specialised to the precise structure of the verification predicate.

Specialisation has a deployment-lifecycle cost that is rarely measured. Any change to the verification *relation*, whether substituting the signature scheme (Loquat $\rightarrow$ PLUM, or a future Sig'), retuning a security parameter, changing the presentation *structure* (the number of credentials shown, or the set of disclosed attributes), or evolving the on-chain ledger schema, requires recompilation of the R1CS matrices and re-derivation of any parameters that depend on the constraint count. (A presentation *policy* that only changes which condition the verifier checks on the disclosed attributes, at fixed structure, is handled verifier-side and forces no recompilation.) In a real-world deployment that must respond to evolving NIST standardisation choices, evolving institutional policies, and evolving threat models, this cost is incurred repeatedly. We refer to this hidden cost as the *cryptographic rigidity* of the static-circuit approach, and we refer to its quantitative manifestation as *update churn*.

Update churn is a structural property of the relation-specific design, the price paid for specialisation, and it is the part of the cost ledger that the current evaluation literature on PQ anonymous credentials does not capture. As long as it remains uncharacterised, the deployment-cost picture of static-circuit PQ anon-credentials is incomplete: the per-change recompile wall-clock turns out to be small (Section 7.8), but the regeneration-and-re-audit footprint it forces is not, and that footprint is what goes unmeasured.

1.4 Zero-Knowledge Virtual Machines as a Structural Alternative

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose underlying relation is fixed once, at design time of the prover, to the correctness of executing a program on a deterministic instruction-set architecture [6, 7, 8]. Anything that a developer can express in a high-level language becomes a guest program; the proof system is universal. Updating the verification predicate becomes a guest-program edit under a fixed universal relation. The audit obligation does not vanish, since the guest binary, its image ID, the host I/O binding, and any precompile interface still require review, but it changes in kind: from auditing a *new relation-specific circuit* to auditing *software and its image under the same, unchanged universal relation*.

In exchange for this generality, a zkVM pays a cost on algebraic primitives whose native arithmetic does not match the zkVM's prover field. PLUM verification involves hundreds of operations over a large prime field (PLUM's specification labels this field 192-bit, though the concrete modulus we instantiate is 199 bits; see Remark 3.5), while modern zkVM provers operate over small native fields such as BabyBear ($p = 2^{31} - 2^{27} + 1$); Loquat is analogous, over the 127-bit Mersenne prime field. Each large-field operation must therefore be emulated by multi-limb arithmetic over the small field. We refer to this cost as the *field-mismatch tax*. Without mitigation, the field-mismatch tax can render PQ-anon-

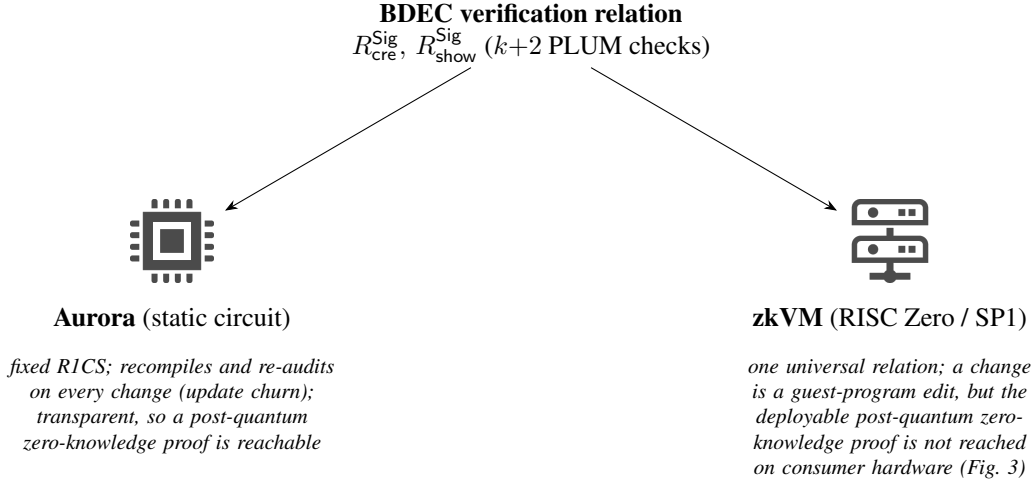


Figure 1: One relation, two outer provers. The same BDEC verification relation is proved *either* by a static Aurora circuit (the BDEC baseline) *or* by a general-purpose zkVM (this thesis), and the two are alternatives: the zkVM does not re-prove an Aurora proof. The figure locates the deployability wall: the static Aurora circuit reaches a post-quantum zero-knowledge proof, whereas the zkVM does not reach that proof on consumer hardware. The agility difference between the two is a footprint distinction that this thesis measures and finds does not translate into a per-proof wall-clock advantage.

credentials-in-zkVM impractical on consumer hardware: in our measurements the precompile-free standalone PLUM verification exhausts memory (OOM) on Apple M-series hardware with 24 GB, and the full credential relation does not terminate within a multi-hour budget (Section 7).

Both RISC Zero [7] and SP1 [9] permit *application-defined precompiles*: native sub-circuits for designated primitives that the prover recognises as syscalls. A precompile can absorb the field-mismatch tax for a primitive that would otherwise dominate the trace. The question, then, is whether a precompile suite covering the algebraic primitives of PLUM verification can make PLUM-instantiated BDEC tractable inside a zkVM on consumer hardware, and how the resulting system compares to the static-circuit baseline along the dimensions that actually matter for deployment.

1.5 Contributions of This Thesis

This thesis makes the following contributions.

1. **A PLUM-instantiated BDEC artifact, validated in execute mode.** We express the verification predicate of the BDEC credential-generation (CreGen) relation as a guest program on RISC Zero, with PLUM in place of Loquat as the signature primitive, and we additionally run PLUM signature verification in isolation on SP1. We validate functional correctness in execute mode: at $\lambda = 80$, BDEC CreGen executes both of its signature verifications to acceptance in 9.4×10^9 guest cycles, and we further realise the full ShowCre presentation relation as a $k + 2$ -verification guest and measure it in execute mode (1.41×10^{10} cycles at $k = 1$ and 1.88×10^{10} at $k = 2$, confirming the additive cost model). To our knowledge this is the first realisation of PLUM-based BDEC components, both credential generation and presentation, in execute mode and the first standalone PLUM-verify prove-mode measurement, inside a general-purpose zkVM.

2. **A deployable zero-knowledge proof not yet realised on consumer hardware.** Neither BDEC relation yet yields a zero-knowledge *proof* on the target hardware: the CreGen succinct prove terminated in an internal-verifier anomaly distinct from a resource limit (an anomaly under diagnosis, Section 7), and the ShowCre zero-knowledge wrap is bounded by the memory frontier of Section 7. This is a measured negative result about the consumer-hardware frontier, *not* an implementation gap in the artifact above.
3. **A custom precompile suite as the instrument that makes the field-mismatch tax measurable.** We design and implement a Griffin- $\mathbb{F}_p^{192}$ precompile for the algebraic-hash primitive that the cost analysis of Section 4 identifies as the dominant tax-bearing operation, together with supporting big-integer arithmetic precompiles [10]. We use it as the instrument that turns a non-terminating emulation into a finite standalone measurement, rather than treating it as a headline speedup: against the precompile-free path, which exhausts memory (OOM at $\approx 1\text{m}45\text{s}$; Table 2) on the target hardware, the precompile reduces PLUM signature verification to a 32.5-minute standalone prove on SP1 at $\lambda = 80$. Introducing a custom AIR adds a trusted component to the verification path; the security statement is therefore *conditional* on the constraint-level soundness of the Griffin AIR, which we carry as an open premise and discuss in Section 6 (the conditional security-preservation theorem and premise (T2)).
4. **The bespoke algebraic-hash precompile is slower than a software SHA-3 control in this regime.** At matched $\lambda = 80$, the Griffin- $\mathbb{F}_p^{192}$ precompile arm (Cell 2) does not outperform a PLUM arm that uses a software SHA-3 hasher and the existing big-integer precompile alone (Cell 3): it runs about $2.45\times$ slower in prove wall-clock (32.5 versus 13.3 minutes), the metric on which deployment cost depends. We report this inversion as a substantive finding rather than a measurement artefact: in the deployed RISC Zero/SP1 pipeline the cost of arithmetising a 199-bit-field algebraic hash through a custom chip exceeds the cost of a SHA-class hash that the substrate already supports natively.
5. **Update churn as a measured evaluation dimension.** The rigidity of relation-specific circuits is a known structural property; we *measure* its lifecycle cost. We define update churn quantitatively as the regeneration footprint incurred by a localised change to the verification relation: the set of compiled artefacts and proof-system parameters that must be regenerated and re-audited. We enumerate this footprint for both the static-circuit and zkVM regimes and use it to characterise a deployment-lifecycle trade-off that the current evaluation literature ignores.
6. **A measurement of the consumer-hardware proving frontier.** On an Apple M5 Pro with 24 GB of memory we report what the precompile-enabled zkVM regime can and cannot do at $\lambda = 80$: PLUM signature verification proves in 32.5 minutes on SP1, whereas BDEC CreGen runs correctly in execute mode but its default succinct proof terminated in an internal-verifier rejection: an anomaly under diagnosis (Section 7), categorically distinct from a resource limit, on which no claim of this thesis rests. We report cycle counts, execute-mode wall-clock times, peak resident-set sizes, and (where proving completes) prove and verify times, with all parameters and procedures documented for reproducibility. The two regimes are compared directly on the update-churn dimension, which is independent of the runtime measurement.
7. **A decision framework for practitioners.** On the basis of these measurements, we provide a

framework that maps deployment-scenario parameters (policy-update frequency, hardware budget, anonymity-strength requirement) onto a regime recommendation, distinguishing the dimensions on which the comparison is currently conclusive (update churn) from those bounded by the consumer-hardware proving frontier.

Taken together, these contributions answer a question the field has left open, namely whether a general-purpose zkVM is a viable substrate for post-quantum anonymous credentials on consumer hardware. The answer is *not yet*, and the load-bearing reason is a single finding: on this hardware the BDEC proof one can afford to generate fails to be zero-knowledge, while the zero-knowledge wrap one can afford fails to be post-quantum. The first half is a measured resource bound; the second is a fact about the deployed toolchain, since the only wraps it exposes are pairing-based, so it holds independently of how favourable any timing number is and more memory does not remove it. This dual obstruction is the central finding of the thesis. The remaining contributions are the measurements that establish the surrounding cost picture, and two of them are findings precisely because they *falsify* a natural prediction: a bespoke algebraic-hash precompile was expected to make verification fast, yet at matched λ it runs slower than a software SHA-3 control (contribution (4)); and a zkVM was expected to be more flexible than a static circuit, yet against the transparent baseline the recompile advantage is negligible (the update-churn dimension of contribution (5)), becoming large only against a preprocessing baseline. The precompile suite (contribution (3)) is what made these measurements finite; it serves as an instrument here rather than as a speedup in its own right, and no zero-knowledge BDEC proof was produced in this work. We report the implementation results as findings about where consumer-hardware proving holds and where it breaks, with contribution (7) translating them into a deployment decision.

1.6 Scope and Non-Goals

This thesis is concerned with the practical deployability of PQ anonymous credentials on consumer hardware. We therefore make the following scope choices.

We focus on the BDEC framework [1] as the anonymous-credential construction, because (i) it is the leading symmetric-primitive-based PQ anonymous-credential system in the recent literature, (ii) its structure of two signature-verification predicates under a hidden user public key inside a zkSNARK is representative of a broad class of PQ-anon-cred designs, and (iii) its reliance on Aurora makes the static-circuit baseline well-defined and reproducible.

We focus on RISC Zero and SP1 as the zkVM substrates, because both expose application-defined precompiles, both target consumer hardware, and both serve as representative examples of the current generation of RISC-V-based zkVMs.

We do *not* propose a new post-quantum signature scheme: we take PLUM as given and our contribution lies in how it is embedded into the broader credential system. We do *not* propose new cryptographic assumptions: the security of the resulting system reduces to those already established for Loquat, PLUM, BDEC, RISC Zero, and SP1, together with a small number of premises that we state explicitly and carry as *open* where they arise (the constraint-level soundness of the Griffin AIR, the existence of a PLUM signature-transcript simulator, and the quantum-random-oracle lifts), discharged in Section 6 only as far as those references allow. We do *not* claim that consumer-hardware PQ anonymous credentials are universally deployable today; rather, we quantify the cost frontier and identify the load-bearing pieces of the cost.

1.7 Thesis Organisation

The remainder of this thesis is structured as follows. Section 2 surveys related work on post-quantum signatures, anonymous credentials, zkSNARKs, zkVMs, and precompile design. Section 3 establishes the cryptographic primitives, signature schemes, zkSNARK and zkVM definitions, the BDEC construction, and the formal definition of update churn used throughout the thesis. Section 4 formalises the question we ask of each system regime and the comparative analysis we apply. Section 5 describes the design and implementation of PLUM-in-BDEC in both the static-circuit and the zkVM regimes, including the precompile suite. Section 7 reports the empirical measurements on consumer hardware. Section 8 interprets the measurements as a deployment-lifecycle decision framework. Section 9 concludes and identifies open problems.

2 Related Work

The Introduction set out the thesis question: whether a general-purpose zkVM can verify an algebraic-hash post-quantum signature feasibly on consumer hardware, and where the deployability wall for post-quantum anonymity then sits. Five strands of prior work bear on it. Post-quantum signatures and anonymous credentials establish what BDEC is assembled from; transparent succinct arguments and zero-knowledge virtual machines are the two substrates this thesis sets against each other; and precompile-based cost reduction is the instrument that makes the zkVM substrate measurable at all. We read each strand with one question in view: what the existing evaluation literature measures and what it leaves unmeasured. The closing subsection turns to the omission this thesis fills. A strand is included when it changes the comparison on one of four axes, namely post-quantum credential security, in-circuit post-quantum signature verification, zkVM proving cost, or update churn, which is why deployed verifiable-credential standards and BBS appear as background rather than as direct baselines.

2.1 Post-Quantum Signatures

The threat that large-scale quantum computers pose to RSA- and discrete-logarithm-based signatures has driven a multi-year standardisation effort. The NIST post-quantum cryptography process has produced two general-purpose signature standards, ML-DSA [11] (a lattice-based scheme derived from Dilithium) and SLH-DSA [12] (a stateless hash-based scheme derived from SPHINCS+), with a third, Falcon (FN-DSA, draft FIPS 206 [13]), also lattice-based. These signatures are designed for general-purpose authentication and are well suited to consumer hardware; they are, however, not well suited as the underlying primitive of an anonymous-credential system because their verification predicates do not embed cheaply inside a zkSNARK. Lattice-based verification involves expensive rejection sampling and norm checks, and hash-based verification involves on the order of 10^5 symmetric-primitive invocations per signature [12], both of which translate into large in-SNARK constraint counts. This concerns the *general* verification predicate. Narrower-functionality results, such as post-quantum Privacy Pass [14] and hardware or precompile acceleration of the hash-based families, reduce these costs in restricted settings, so the “not well suited” judgement applies to the general anonymous-credential predicate and admits these exceptions in restricted ones.

A complementary line of work develops post-quantum signatures whose verification predicate is deliberately structured for a low constraint count when encoded as an R1CS. Picnic, Banquet, and Limbo build signatures around the MPC-in-the-head paradigm [15, 16, 17]. Loquat [3] departs from this line: it bases unforgeability on the Legendre PRF, a symmetric-primitive assumption distinct from lattice and code-based hardness, and is designed so that its verification circuit fits into a comparatively small R1CS. PLUM [5], the focus of this thesis, replaces Loquat’s Legendre PRF with the t -th power residue PRF and replaces the FRI low-degree test with the STIR protocol. The result is, at 128-bit security, signatures approximately $1.5\times$ smaller, verification approximately $4\times$ faster, and approximately $1.28\times$ fewer R1CS constraints per verification, while preserving Loquat’s security guarantees [5]. This trio of improvements is what motivates evaluating PLUM as a replacement for Loquat in the BDEC framework; whether the replacement is strictly favourable inside a zkVM, as opposed to the static-circuit setting these figures describe, is one of the questions this thesis answers. A direct competitor in this SNARK-friendly line is CAPSS [18], a framework for SNARK-friendly post-quantum signatures; the schemes differ as much in *assumption* as in constraint count, since Loquat and PLUM rest on the

Legendre and power-residue PRFs, MPC-in-the-head designs on generic symmetric primitives, lattice schemes on SIS/LWE, and every algebraic-hash argument additionally on a random-oracle instantiation of its hash, so a constraint-count comparison alone does not settle the choice.

2.2 Anonymous Credentials

Anonymous credentials originate with Chaum’s pseudonymous-credentials proposal [19], formalised by Camenisch and Lysyanskaya (CL) [20], and have since developed into a substantial line including Pointcheval–Sanders signatures, Coconut, and the BBS family [21, 22, 23]. These remain the basis of deployed selective-disclosure systems (W3C Verifiable Credentials [24], BBS-based showing), which we treat as deployment baselines rather than post-quantum-anonymous-credential baselines, since none is quantum-resistant. These classical constructions rely on pairing-based or discrete-logarithm-based assumptions, and are therefore not quantum-resistant. The post-quantum case has, until recently, been pursued largely through lattice-based primitives: Jeudy et al. at Crypto 2023 [2] provide an efficient lattice-based anonymous-credential scheme, leveraging the SIS and LWE assumptions; subsequent work has refined and applied this template. The known limitations of the lattice approach include large proof and key sizes and, in the Jeudy et al. scheme as published, support for only a single message per credential.

The Blockchain-based Digital Education Credential (BDEC) framework of Li, Zhang et al. [1] takes a different path. BDEC is generic over a signature scheme and a zkSNARK, and the authors instantiate it with Loquat and Aurora to obtain a post-quantum anonymous-credential system whose security reduces to collision-resistance and the Legendre PRF. The system supports unforgeability, anonymity, unlinkability, conditional linkability, and revocability, and the authors report concrete-performance improvements over the lattice baseline of Jeudy et al. [2]. The BDEC construction is the anonymous-credential framework this thesis builds on; we examine it in detail in Section 3. We distinguish BDEC-as-proposed from the instantiation we measure: BDEC’s anonymity and unlinkability require the underlying argument to be zero-knowledge, which the consumer-hardware zkVM receipt we produce does not yet satisfy (Section 6), so our artifact does not inherit those properties unconditionally.

2.3 Transparent Succinct Arguments and Low-Degree Testing

A zkSNARK proof system consists of an interactive-oracle-proof (IOP) compiled into a non-interactive argument via a hash-based transformation, most commonly the Ben-Sasson–Chiesa–Spooner (BCS) transformation [25]. The BCS transformation requires a low-degree test that confirms a committed function is close to a low-degree polynomial. Two such tests dominate recent transparent-argument constructions: FRI (Fast Reed–Solomon Interactive Oracle Proof of Proximity) [26] and STIR (Reed–Solomon proximity testing with fewer queries) [27]. STIR achieves lower query complexity and a smaller code-rate after folding, at the cost of slightly more involved verifier logic; both rest on Reed–Solomon proximity-gap results [28].

Aurora [4] is the canonical transparent succinct argument for R1CS in the IOP-plus-BCS family. It produces zero-knowledge proofs of R1CS satisfiability with polylogarithmic proof size but *linear* verifier complexity, where the verifier reads the explicit R1CS matrices [4, Thm. 1.2], with no relation-specific trusted setup. (The holographic variant Fractal [29] attains a polylogarithmic verifier through

preprocessing; we use plain Aurora as the BDEC baseline.) Aurora is the zkSNARK choice of the BDEC framework as instantiated by Li, Zhang et al. [1], and is therefore the static-circuit baseline against which we compare in this thesis. Other transparent succinct arguments share the property of relation-specific R1CS encoding and therefore the same cryptographic rigidity discussed in Section 1; the holographic ones (Fractal, Marlin [30]) differ in one respect that matters for the recompile cost we measure, namely that they preprocess each relation into an index, so their per-circuit setup is large rather than negligible, unlike the non-preprocessing Aurora that the BDEC baseline uses.

2.4 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose underlying relation is the correctness of executing a guest program on a fixed instruction-set architecture [6, 8]. The term “zkVM” is ecosystem terminology: the *default* receipt these systems produce is succinct but not necessarily zero-knowledge, a gap that becomes this thesis’s central security finding (Section 6). Several zkVM designs are mature enough to be considered for credential-system deployment:

- **RISC Zero** [7] is a RISC-V zkVM whose prover is a recursive STARK over the BabyBear field. Application-defined precompiles are supported via the Zirgen big-integer dialect.
- **SP1** [9] is a RISC-V zkVM whose prover is built on the Plonky3 toolkit. SP1 supports application-defined precompiles via its custom-precompile framework, and offers an explicit chained API for wrapping a succinct STARK receipt in Groth16 or PLONK to obtain a zero-knowledge, succinctly-verifiable proof.
- **Jolt** [31] is a RISC-V zkVM whose prover uses a sum-check-and-lookup approach instead of segment-based STARK proving. Jolt’s design optimises for guest cycle counts of moderate size and is included in this discussion for completeness.

A separate strand of work, exemplified by zkLLVM [32], takes a compiler approach in which high-level programs are compiled into circuit-friendly intermediate representations rather than RISC-V; this approach inherits the cryptographic-rigidity property of static circuits and is therefore positioned closer to the Aurora regime than to the zkVM regime as we use the term in this thesis. The same applies to circuit-frontends such as Circom [33] and Noir, which target a single underlying SNARK system and lock the verification predicate at compile time.

2.5 Precompile-Based Cost Reduction

The general technique of replacing emulated computation in a zkVM with a native AIR-based sub-circuit, recognised by the prover via a syscall, has been applied to several primitives. The R0VM 2.0 release adds precompiles for BN254 and BLS12-381 elliptic-curve operations and reports reducing Ethereum block-proving from approximately 35 minutes to approximately 44 seconds [34]. Each such precompile introduces a primitive-specific arithmetisation that becomes part of the trusted computing base and must be audited, as our Griffin precompile is (premise (T2), Section 6). The speedup therefore comes with an audit obligation. A community RSA precompile for zkVMs has likewise been reported to cut RSA signature verification from tens of millions of guest cycles to the order of 10^5 , roughly two

orders of magnitude.¹ SP1’s standard library ships SHA-256 and ECDSA precompiles by default [9]; RISC Zero ships SHA-256 and BigInt-256 modular-multiplication precompiles by default [7].

These results share a structural pattern. A fixed primitive whose software emulation over the prover’s native field would dominate the trace is replaced by a native AIR, and the per-call cost from the guest’s perspective collapses to a single syscall. The Griffin- $\mathbb{F}_p^{192}$ precompile proposed in this thesis sits within this pattern, with the additional consideration that the primitive in question is parameterised over a non-trivial 192-bit prime field whose support requires a corresponding BigInt-192 precompile suite for soundness.

2.6 Evaluation Methodology and the Gap This Work Addresses

The recent literature on PQ-anonymous-credential systems reports evaluation along the conventional dimensions: prover wall-clock time, verifier wall-clock time, proof size, and, less commonly, peak memory [1, 5]. The same is true of the zkVM literature, which adds cycle counts and segment counts to the standard list.

The cost incurred when the verification predicate must be changed, in response to a change in the presentation structure, a primitive substitution, or a security-parameter retuning, is not reported alongside these dimensions. The concept that such changes carry a migration cost is itself not new: crypto-agility work motivates this cost as a design concern, and post-quantum verifiable-credential proposals have noted the need to re-derive parameters when the underlying primitive changes. For a static-circuit deployment that change requires recompilation of the R1CS matrices and re-derivation of any parameters that depend on the constraint count; for a zkVM deployment it requires only an edit to the guest program. Without accounting for this cost, comparisons between the two regimes omit the regeneration-and-re-audit footprint that each relation change imposes on the static side. We measure that footprint in Section 7, and we report there that its wall-clock component is sub-second against a transparent non-preprocessing argument, so the separation between the two regimes is one of re-audit footprint rather than recompile time, and is itself conditional on the static baseline’s preprocessing model. This is a deployment-lifecycle cost, paid once per relation change, and it falls outside the per-proof wall-clock figures that the prior work reports. The prior work that names the concern stops short of measuring it. We introduce *update churn* as a measurement dimension and report it alongside the conventional dimensions for both regimes.

The closest prior work confirms that this measurement dimension is unreported. Post-quantum anonymous credentials have been built lattice-natively (Bootle et al. [35]; Argo et al. [36], the latter with sub-80 KB credentials and sub-second native showing), classically from zkSNARKs composed with existing identity infrastructure (zk-creds [37], which is pairing-based and not post-quantum), and on a hand-rolled relation-specific STARK for a lattice signature (Policharla et al. [14]); none runs its verification predicate inside a general-purpose zkVM, and none reports the per-change deployment cost. SNARK-friendly post-quantum signature *design* has likewise been generalised into a framework (CAPSS [18]), but as a signature-design methodology, not a measured anonymous-credential-in-zkVM study. On the substrate side, individual post-quantum *signatures* have recently been verified inside general-purpose zkVMs as engineering demonstrations (Falcon and SPHINCS+ in Cairo [38]; ML-DSA in SP1, Groth16-wrapped [39]); we therefore do not claim the first post-

¹Reported in vendor/community engineering write-ups; we cite it only as an order-of-magnitude indication of the precompile pattern, not as a peer-reviewed measurement.

quantum signature in a zkVM. Our claim is narrower and measured: to our knowledge, the first reported execution-and-proving study of an algebraic-hash post-quantum *anonymous credential* (BDEC instantiated with PLUM) inside a general-purpose zkVM, with the update-churn dimension and the dual zero-knowledge/post-quantum obstruction that none of the above confronts. We position against the recent zkVM systematisation [40], which benchmarks general-purpose zkVMs on classical workloads but measures no post-quantum anonymous credential. None of these works reports the per-change update-churn cost; that is the gap this work fills. The next section develops the primitives the measurement rests on, beginning with PLUM, BDEC, and the two argument substrates.

3 Preliminaries

3.1 Notation

Let λ denote the security parameter. A function $f(\lambda)$ is negligible if for every polynomial p there exists N such that, for all $\lambda > N$, $f(\lambda) < 1/p(\lambda)$. We write $\text{negl}(\lambda)$ for an unspecified negligible function. All algorithms and adversaries considered below are classical probabilistic polynomial-time unless stated otherwise; security statements inherited from the underlying signature schemes hold in the classical random-oracle model, and any quantum (QROM) variant is flagged explicitly where it arises. For a randomized algorithm A , the notation $y \leftarrow A(x)$ means that A is run on input x using fresh randomness and outputs y . For a finite set S , $y \leftarrow S$ means that y is sampled uniformly from S . We write $[n] = \{1, \dots, n\}$. Attribute vectors are denoted by $A = (a_1, \dots, a_m)$, and predicates over attributes are denoted by ϕ . Public keys, secret keys, messages, and signatures are denoted by pk , sk , m , and σ , respectively.

Definition 3.1 (Efficiently decidable relation). A relation $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ is efficiently decidable if there exists a deterministic polynomial-time algorithm D_R such that, for every pair (x, w) ,

$$D_R(x, w) = 1 \iff R(x; w) = 1.$$

We call x the public instance and w the witness. The statement $R(x; w) = 1$ means that w is a valid witness for the public instance x .

3.2 Cryptographic Primitives

This subsection collects the primitives that appear inside both the signature schemes (Loquat, PLUM) and the BDEC construction below. We work with a single odd prime p throughout; the same field is reused by every primitive in any one instantiation, eliminating field-lifting overhead.

3.2.1 Power residue PRF.

Definition 3.2 (t -th power residue PRF). Let p be an odd prime and let $t \geq 2$ be an integer with $t \mid (p-1)$. Let $g \in \mathbb{F}_p^*$ be a generator and let

$$\zeta_t = g^{(p-1)/t} \in \mathbb{F}_p^*$$

be a primitive t -th root of unity, so that the cyclic subgroup

$$\mu_t = \langle \zeta_t \rangle = \{1, \zeta_t, \zeta_t^2, \dots, \zeta_t^{t-1}\}$$

is the set of t -th roots of unity in $\mathbb{F}_p^*$. For a key $K \in \mathbb{F}_p$ and an input $a \in \mathbb{F}_p$ with $K + a \neq 0$, define

$$\chi_t(K, a) = (K + a)^{(p-1)/t} \in \mu_t,$$

and let $L_t(K, a) \in \mathbb{Z}/t\mathbb{Z}$ be the unique exponent for which

$$\chi_t(K, a) = \zeta_t^{L_t(K, a)}.$$

The keyed function $L_t(K, \cdot) : \mathbb{F}_p \setminus \{-K\} \rightarrow \mathbb{Z}/t\mathbb{Z}$ is the t -th power residue PRF.

The special case $t = 2$ makes $\chi_2(K, a) \in \{+1, -1\}$ the Legendre symbol of $K + a$ modulo p ; the corresponding L_2 is the Legendre PRF used by Loquat. Taking $t > 2$ yields more entropy per evaluation. The PLUM construction described later in this section pins $t = 256$. Definition 3.2 only fixes the algebraic map; its use as a PRF rests on a separate hardness assumption, namely that $L_t(K, \cdot)$ is pseudorandom for a uniformly random key K . This assumption generalises the Legendre-PRF assumption underlying Loquat and inherits that assumption’s cryptanalytic status rather than being implied by the map being well-defined.

3.2.2 Algebraic hash function.

We require an algebraic hash function whose internal computation is dominated by low-degree field operations over the same field $\mathbb{F}_p$ used by the signature scheme and the PRF. We use the Griffin family for this purpose. Following the cited literature, a Griffin instance is parameterised by a state width t_{state} and a number of rounds; each round consists of constant-addition, linear mixing by a fixed matrix, and a low-degree S-box substep using small-degree power maps: a forward degree- d power map on one lane, its inverse $x \mapsto x^{1/d}$ (well-defined since $\gcd(d, p-1) = 1$) on a second, and a Horst-type quadratic map on the remaining lanes (§5.3). Each constraint then has degree at most d while the per-round map has high algebraic degree. We write

$$H_{\text{Griffin}} : \mathbb{F}_p^* \rightarrow \mathbb{F}_p^c$$

for a Griffin-based sponge or compression hash producing a c -element digest, following the construction of [10]. The instantiation used throughout has state width $t_{\text{state}} = 4$, capacity 2 (hence rate 2 and digest length $c = 2$), S-box degree $d = 3$ (the smallest integer ≥ 3 with $\gcd(d, p-1) = 1$ over PLUM’s prime; §5.3), and 14 rounds, the value the Griffin round-count argument of [10] yields for this width-4, 199-bit instance (computed and checked in the reference implementation, §5.3). The digest length $c = 2$ over the 199-bit field is intended to provide collision resistance at the targeted security level, conditional on the security analysis of [10] applying to this width-4, $d = 3$, 14-round instance.

When a zkVM exposes Griffin as a precompile we will write the corresponding syscall as $\text{Griffin}_{\mathbb{F}_p^{192}}$, indicating that the precompile operates natively over the same prime field used by PLUM (nominally labelled 192-bit; concretely the 199-bit prime of Remark 3.5). The cost of one Griffin permutation under this precompile is, by definition, one zkVM syscall.

3.2.3 Merkle commitment.

Definition 3.3 (Merkle commitment). Let $H : \mathbb{F}_p^c \times \mathbb{F}_p^c \rightarrow \mathbb{F}_p^c$ be a collision-resistant compression function and let $\ell = (\ell_1, \dots, \ell_n)$ with $\ell_i \in \mathbb{F}_p^c$ be a vector of leaves. The Merkle root

$$\text{rt} = \text{MerkleRoot}_H(\ell) \in \mathbb{F}_p^c$$

is defined by iterated pairwise application of H along a complete binary tree over ℓ , padding to the next power of two as needed. For index $i \in [n]$, the membership proof path_i is the sequence of sibling digests along the root path. The verification predicate

$$\text{MerkleVerify}(\text{rt}, \ell_i, \text{path}_i) \in \{0, 1\}$$

returns 1 iff recomputing the root from (ℓ_i, path_i) yields rt .

We instantiate the Merkle commitment used inside the BDEC relations below with $H = H_{\text{Griffin}}$ over $\mathbb{F}_p$, so that membership checks reduce to a small number of algebraic-hash invocations per path step. A *sparse* Merkle tree is the same structure indexed over the digest range $\{0, 1\}^\lambda$ rather than $[n]$; non-membership is shown by a membership proof of a domain-separated sentinel leaf at the queried index, and we write $\text{NonMemberVerify}(\text{rt}, x, \text{path})$ for the corresponding predicate.

3.3 Digital Signature Scheme

Definition 3.4 (Digital signature). A digital signature scheme is a tuple of probabilistic polynomial-time algorithms

$$\text{Sig} = (\text{Sig.Setup}, \text{Sig.KeyGen}, \text{Sig.Sign}, \text{Sig.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda).$$

The parameters pp_{Sig} determine a public-key space $\mathcal{PK}_{\text{ppSig}}$, a secret-key space $\mathcal{SK}_{\text{ppSig}}$, a message space $\mathcal{M}_{\text{ppSig}}$, and a signature space Σ_{ppSig} . The algorithms have the following syntax:

$$\begin{aligned} (\text{sk}, \text{pk}) &\leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}}), & (\text{sk}, \text{pk}) &\in \mathcal{SK}_{\text{ppSig}} \times \mathcal{PK}_{\text{ppSig}}, \\ \sigma &\leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m), & m &\in \mathcal{M}_{\text{ppSig}}, \sigma \in \Sigma_{\text{ppSig}}, \\ b &\leftarrow \text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m, \sigma), & b &\in \{0, 1\}. \end{aligned}$$

When a higher-level protocol uses Sig, every value to be signed must first be represented as an element of the message space $\mathcal{M}_{\text{ppSig}}$. We write

$$\text{Encode}_{\text{Sig}} : \{0, 1\}^* \rightarrow \mathcal{M}_{\text{ppSig}}$$

for this encoding map. Thus, if a protocol value v is signed, the actual message given to Sig.Sign is

$$m = \text{Encode}_{\text{Sig}}(v) \in \mathcal{M}_{\text{ppSig}}.$$

Correctness. Correctness requires that for all λ , all $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$, all $(\text{sk}, \text{pk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, and all messages $m \in \mathcal{M}_{\text{ppSig}}$, if $\sigma \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m)$, then

$$\Pr[\text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m, \sigma) = 1] \geq 1 - \text{negl}(\lambda),$$

where the probability is over the randomness of the algorithms.

Existential Unforgeability under Chosen Message Attack. For unforgeability, define $\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}}(\lambda)$ as follows. The challenger samples

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda), \quad (\text{sk}, \text{pk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}}),$$

and gives $(\text{pp}_{\text{Sig}}, \text{pk})$ to $\mathcal{A}$. The adversary has oracle access to

$$\mathcal{O}_{\text{Sig}}(m) : \sigma \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m),$$

for messages $m \in \mathcal{M}_{\text{ppSig}}$, and the challenger records every queried message in a set Q . Eventually $\mathcal{A}$ outputs (m^*, σ^*) . The experiment outputs 1 iff

$$m^* \in \mathcal{M}_{\text{ppSig}} \quad \wedge \quad m^* \notin Q \quad \wedge \quad \text{Sig.Verify}(\text{ppSig}, \text{pk}, m^*, \sigma^*) = 1.$$

The signature scheme is EUF-CMA secure if for every adversary $\mathcal{A}$,

$$\Pr[\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Plain EUF-CMA suffices for the credential reductions of Section 6: BDEC’s showing relation binds the signed message inside the proof, and the unforgeability reduction extracts a *witness* (a signature on a statement outside the issued set) rather than relying on signature uniqueness or non-malleability, so neither strong unforgeability nor resistance to signature re-randomisation is required.

Signature-Verification Predicate. Fix public parameters $\text{ppSig} \leftarrow \text{Sig.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{ppSig}}$, $m \in \mathcal{M}_{\text{ppSig}}$, and $\sigma \in \Sigma_{\text{ppSig}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}, m, \sigma) = 1 \iff \text{Sig.Verify}(\text{ppSig}, \text{pk}, m, \sigma) = 1.$$

The signature σ is computed by $\sigma \leftarrow \text{Sig.Sign}(\text{ppSig}, \text{sk}, m)$. The predicate does not compute σ ; it only checks whether a given σ is valid for (pk, m) . When this predicate is inserted into a zkSNARK relation, the public instance and private witness must be specified. For example,

$$R_{\text{Sig}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

3.3.1 Loquat: a Legendre-PRF post-quantum signature with low R1CS verification cost.

Loquat is a post-quantum digital signature scheme based on the Legendre PRF and collision-resistant hash functions, designed so that its verification algorithm can be implemented by a relatively small R1CS circuit. Its post-quantum claim rests on the conjectured hardness of the Legendre PRF, collision resistance of the hash, and the Fiat–Shamir/BCS transform; it is a research scheme and is not standardised in the manner of ML-DSA or SLH-DSA.

The Loquat literature [3] describes such verification as “SNARK-friendly”. We avoid that term as a load-bearing descriptor, since it conflates several distinct cost properties, and instead refer directly to the structural quantity that matters here: a low constraint count for the verification predicate when encoded as an R1CS.

Formally, a Loquat instance is a tuple of probabilistic polynomial-time algorithms

$$\text{Loquat} = (\text{Loquat.Setup}, \text{Loquat.KeyGen}, \text{Loquat.Sign}, \text{Loquat.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{ppLoquat} \leftarrow \text{Loquat.Setup}(1^\lambda).$$

These parameters fix, among other things, a prime field $\mathbb{F}_p$, a public list $I = (I_1, \dots, I_L) \in \mathbb{F}_p^L$ for some integer L , and auxiliary parameters for the underlying Legendre-PRF key-identification protocol and its compilation to a non-interactive proof system.

The key-generation algorithm samples a secret Legendre-PRF key and its corresponding public key as a vector of PRF evaluations:

$$\begin{aligned} \text{sk} &= K \leftarrow \text{Loquat.KeyGen}(\text{pp}_{\text{Loquat}}), \\ \text{pk} &= (L_K(I_1), \dots, L_K(I_L)) \in \{0, 1\}^L, \end{aligned}$$

where $L_K(\cdot)$ denotes the keyed Legendre pseudorandom function and each $L_K(I_\ell) \in \{0, 1\}$ is the Legendre-symbol output bit on input $K + I_\ell$.

The message space $\mathcal{M}_{\text{Loquat}}$ and signature space Σ_{Loquat} are those induced by the non-interactive key-identification protocol obtained from the underlying interactive-oracle proof via the Fiat–Shamir/BCS transformation. On input $(\text{pp}_{\text{Loquat}}, \text{sk}, m)$ with $m \in \mathcal{M}_{\text{Loquat}}$, the signing algorithm runs this protocol using the secret key K as the witness and outputs a signature

$$\sigma \leftarrow \text{Loquat.Sign}(\text{pp}_{\text{Loquat}}, \text{sk}, m) \in \Sigma_{\text{Loquat}}.$$

The verification algorithm is deterministic and checks the hash chain, Merkle-tree commitments, univariate-sumcheck proof, low-degree test, and quadratic-residuosity conditions implied by the Legendre-PRF relation:

$$b \leftarrow \text{Loquat.Verify}(\text{pp}_{\text{Loquat}}, \text{pk}, m, \sigma) \in \{0, 1\}.$$

Loquat-Verification Predicate. Fix $\text{pp}_{\text{Loquat}} \leftarrow \text{Loquat.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{Loquat}}$, $m \in \mathcal{M}_{\text{Loquat}}$, and $\sigma \in \Sigma_{\text{Loquat}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Loquat}, \text{pp}_{\text{Loquat}}}(\text{pk}, m, \sigma) = 1 \iff \text{Loquat.Verify}(\text{pp}_{\text{Loquat}}, \text{pk}, m, \sigma) = 1.$$

When this predicate is encoded as a zkSNARK relation, we take, for example,

$$R_{\text{Loquat}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Loquat}, \text{pp}_{\text{Loquat}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

3.3.2 PLUM: a power-residue-PRF post-quantum signature with low R1CS verification cost.

PLUM is a post-quantum digital signature scheme that follows the same overall structure as Loquat but replaces the Legendre PRF and FRI-based low-degree test with a t -th power residue PRF and the STIR protocol, and selects a prime field tailored to these primitives. These substitutions reduce the polynomial degrees and query complexity in the underlying proof system, leading to smaller signatures and fewer R1CS constraints in the verification circuit while, per the PLUM authors' analysis, retaining EUF-CMA security at the claimed level (subject to the prime-instantiation caveat of Remark 3.5).

Formally, a PLUM instance is a tuple of probabilistic polynomial-time algorithms

$$\text{Plum} = (\text{Plum.Setup}, \text{Plum.KeyGen}, \text{Plum.Sign}, \text{Plum.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Plum}} \leftarrow \text{Plum.Setup}(1^\lambda).$$

The setup chooses an odd prime p and an integer $t \geq 2$ such that $t \mid (p-1)$, fixes a generator $g \in \mathbb{F}_p^*$, and defines the t -th power residue PRF L_t as in Definition 3.2. It then samples a public list $I = (I_1, \dots, I_L) \in \mathbb{F}_p^L$ and sets the parameters for the univariate-sumcheck and STIR low-degree test (including the code domain U , folding parameter, degree bounds, and query-repetition parameters) as in Algorithm 1 of [5].

Remark 3.5 (Prime instantiation). The implementation evaluated in this thesis fixes a concrete 199-bit prime $p = 2^{64} p_0 + 1$ with p_0 a 135-bit prime,² chosen so that $t = 256 \mid (p-1)$ and the 2-adicity of $p-1$ is at least 64, as the STIR low-degree test requires. The value of p_0 used is *not* the decimal printed in the published parameter table [5, §3.3]: that printed value is composite (its smallest prime factor is 97, witnessed by a primality test in the artefact), almost certainly a typographical error in the manuscript, and is therefore unusable as a field modulus. We substitute a nearby 199-bit value satisfying all the structural requirements above. The substitution is benign for the measurements of this thesis, whose cost is a function of the field *bit-width* and smoothness rather than of the exact decimal value; it is recorded explicitly because any concrete security-bit claim that depends on the specific value of p is conditional on this instantiation, and a paper-faithful or interoperable deployment should confirm the intended modulus with the PLUM authors. The security analysis (Section 6) carries this conditionality as an explicit hypothesis.

The key-generation algorithm samples a secret PRF key and derives the public key as a vector of t -th power residue PRF evaluations:

$$\begin{aligned} \text{sk} &= K \leftarrow \text{Plum.KeyGen}(\text{pp}_{\text{Plum}}), \\ \text{pk} &= (L_t(K, I_1), \dots, L_t(K, I_L)) \in (\mathbb{Z}/t\mathbb{Z})^L, \end{aligned}$$

where each $L_t(K, I_\ell) \in \mathbb{Z}/t\mathbb{Z}$ is a symbol of the t -th power residue PRF on input $K + I_\ell$ (for $t = 2$ this degenerates to the Loquat binary Legendre symbol). The public key thus occupies $L \lceil \log_2 t \rceil$ bits. Compared to Loquat, taking $t > 2$ increases the entropy per symbol and allows PLUM to use fewer public-key symbols L and fewer challenged symbols B at a given security level.

The message space $\mathcal{M}_{\text{Plum}}$ and signature space Σ_{Plum} are induced by the non-interactive key-identification protocol obtained from the PLUM-specific interactive-oracle proof via the Fiat–Shamir/BCS transformation, with FRI replaced by STIR in the low-degree test component. On input $(\text{pp}_{\text{Plum}}, \text{sk}, m)$ with $m \in \mathcal{M}_{\text{Plum}}$, the signing algorithm runs this protocol using the secret key K as the witness and outputs a signature

$$\sigma \leftarrow \text{Plum.Sign}(\text{pp}_{\text{Plum}}, \text{sk}, m) \in \Sigma_{\text{Plum}}.$$

The verification algorithm is deterministic and checks the hash chain, Merkle-tree commitments, univariate-sumcheck proof, STIR low-degree test, and t -th-power-residue conditions implied by the PLUM PRF relation:

$$b \leftarrow \text{Plum.Verify}(\text{pp}_{\text{Plum}}, \text{pk}, m, \sigma) \in \{0, 1\}.$$

²The implementation prime is 199-bit (with an approximately 135-bit p_0). The “192-bit” / “ $\mathbb{F}_p^{192}$ ” naming used elsewhere in this thesis for the PLUM field and the Griffin precompile is a loose label rather than the exact bit-width; PLUM’s own source is internally inconsistent on this point, the prose and the printed p_0 disagreeing.

For the 128-bit parameter set, PLUM uses $t = 256$, a public-key length of $L = 2^{12}$ PRF symbols, and $B = 28$ challenged symbols, and verifying a single signature with Griffin as the hash instantiation requires about 1.16×10^5 R1CS constraints, versus $\approx 1.48 \times 10^5$ for Loquat under comparable assumptions [5, §4.2]. The corresponding signature size is about 37 KB (versus 57 KB for Loquat); the signing and verification runtimes are *estimated*, from the Loquat implementation rather than a from-scratch PLUM measurement [5, Table 2], to be up to a factor 4 faster than Loquat at the same security level.

PLUM-Verification Predicate. Fix $\text{pp}_{\text{Plum}} \leftarrow \text{Plum.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{Plum}}$, $m \in \mathcal{M}_{\text{Plum}}$, and $\sigma \in \Sigma_{\text{Plum}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Plum}, \text{pp}_{\text{Plum}}}(\text{pk}, m, \sigma) = 1 \iff \text{Plum.Verify}(\text{pp}_{\text{Plum}}, \text{pk}, m, \sigma) = 1.$$

When this predicate is encoded as a zkSNARK relation, we can take, for example,

$$R_{\text{Plum}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Plum}, \text{pp}_{\text{Plum}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

Quantitative Comparison to Loquat. At the same 128-bit security level and with both schemes instantiated using Griffin as the algebraic hash, PLUM reduces signature size from 57 KB to 37 KB, reduces the verification circuit from about 148,825 (Loquat [3]) to 116,285 R1CS constraints, and is expected to decrease signing and verification time by up to a factor of four, according to the analysis and parameter choices of [5]. These improvements come from (i) using a t -th power residue PRF to halve the polynomial degrees in the sumcheck relation, (ii) replacing FRI with the lower-query-complexity STIR protocol, and (iii) choosing a prime p that simultaneously supports the PRF and the smooth multiplicative subgroups required by STIR, avoiding the field-lifting overhead present in Loquat. These are *static R1CS* advantages; Section 7 shows they do not translate cleanly into zkVM proving cost, where the field-mismatch tax on Griffin over a non-native prover field dominates.

3.4 zkSNARK

Let R be an efficiently decidable relation and let

$$L_R = \{x : \exists w \text{ such that } R(x; w) = 1\}$$

be the corresponding language. A zkSNARK for R is a tuple

$$\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify}).$$

Two orthogonal axes matter here: *transparency* (whether $\Pi.\text{Setup}$ needs a relation-specific trusted setup) and *universality* (whether the proving key/preprocessing is independent of the concrete relation R). Update churn is driven by the latter: a transparent but relation-specific argument such as Aurora still re-derives relation-dependent parameters on every change. We first state the universal/relation-independent case, in which the setup is independent of the concrete relation:

$$\text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda).$$

If a relation-specific proof system is used instead, setup should be written as $\text{crs}_R \leftarrow \Pi.\text{Setup}(1^\lambda, R)$. The proving and verification algorithms are

$$\pi \leftarrow \Pi.\text{Prove}(\text{crs}, x, w), \quad b \leftarrow \Pi.\text{Verify}(\text{crs}, x, \pi) \in \{0, 1\}.$$

Completeness. Completeness requires that for every (x, w) such that $R(x; w) = 1$,

$$\Pr[\Pi.\text{Verify}(\text{crs}, x, \Pi.\text{Prove}(\text{crs}, x, w)) = 1] \geq 1 - \text{negl}(\lambda).$$

Soundness. Soundness requires that for every adversary $\mathcal{A}$,

$$\Pr[x \notin L_R \wedge \Pi.\text{Verify}(\text{crs}, x, \pi) = 1 : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}(\text{crs})] \leq \text{negl}(\lambda).$$

Knowledge soundness (argument of knowledge). A property strictly stronger than soundness, and the one a higher-level reduction needs when it must *extract* a witness rather than merely conclude that one exists. The argument is *knowledge-sound* if there is a probabilistic polynomial-time extractor $\mathcal{E}$ such that, for every prover $\mathcal{A}$ producing an accepting proof for a statement x with non-negligible probability, $\mathcal{E}$, given $\mathcal{A}$'s code and randomness (or rewinding/oracle access), outputs a witness w with $R(x; w) = 1$, except with negligible *knowledge error*. BDEC's unforgeability reduction relies on this property of Π to pull a credential and its signatures out of an accepting showing; plain soundness, which only certifies that a witness exists, does not suffice.

Zero-knowledge. Zero-knowledge requires that there exists a simulator Sim such that, for every x and w satisfying $R(x; w) = 1$, the distributions

$$\{(\text{crs}, x, \pi) : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), \pi \leftarrow \Pi.\text{Prove}(\text{crs}, x, w)\}$$

and

$$\{(\text{crs}, x, \tilde{\pi}) : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), \tilde{\pi} \leftarrow \text{Sim}(\text{crs}, x)\}$$

are computationally indistinguishable. Unless stated otherwise this is computational zero-knowledge with a simulator in the (classical) random-oracle model; whether a given instantiation's simulator extends to the QROM is flagged where the distinction affects the receipt-mode claims of Section 5.

3.4.1 Aurora: Transparent Succinct Arguments for R1CS

Aurora [4] is a transparent zkSNARK for the rank-1 constraint system (R1CS) relation. Given an R1CS instance (A, B, C) over a finite field $\mathbb{F}$ and a candidate assignment $z \in \mathbb{F}^n$ satisfying the Hadamard identity

$$(Az) \odot (Bz) = Cz,$$

Aurora produces a succinct, publicly verifiable, zero-knowledge proof of the existence of a satisfying witness, with polylogarithmic proof size and linear verifier complexity in n . Aurora is transparent in the sense that it requires no relation-specific trusted setup beyond a collision-resistant hash function. The Aurora reference measured in this thesis is run with zero-knowledge disabled (`zk=false`, 107.5-bit security achieved). With zero-knowledge off, the measurement bounds proving cost only; it says nothing about anonymity.

When instantiated for a fixed relation R , the R1CS encoding fixes the matrices (A, B, C) at compile time; any change to R that affects which gates appear in the verification predicate requires recompilation of these matrices and re-derivation of any parameters that depend on the constraint count. This is the property that motivates the zkVM approach: the underlying relation is fixed to the ISA semantics once, and program changes are expressed inside the witness rather than in the matrices.

The BDEC construction of [1] instantiates its zkSNARK Π with Aurora over the R1CS encoding of the credential-issuance and credential-showing relations defined later in this section. The dominant component of proof generation and verification in this regime is Aurora’s outer low-degree-test (LDT) sub-protocol; we measure its share of total cost in Section 7.

3.5 Anonymous Credential

Definition 3.6 (Anonymous credential). Let λ be a security parameter, let Attr be an attribute space, and let Φ be a class of predicates over attribute vectors. An anonymous credential scheme is a tuple of algorithms or interactive protocols

$$\text{AC} = (\text{AC.Setup}, \text{AC.UserKeyGen}, \text{AC.IssuerKeyGen}, \text{AC.Issue}, \text{AC.Show}, \text{AC.Verify}).$$

For an attribute vector $A = (a_1, \dots, a_m) \in \text{Attr}^m$, issuance gives a user a credential cred . For a predicate $\phi \in \Phi$, the showing algorithm outputs a public statement and proof

$$(\text{stmt}, \pi) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}, \text{usk}, \text{ipk}, A, \phi).$$

The statement contains intentionally disclosed information. The proof proves membership in a relation

$$R_{\text{AC}}(\text{stmt}; w) = 1,$$

where the witness contains at least $w = (\text{cred}, \text{usk}, A)$ and the relation checks that cred is valid and that $\phi(A) = 1$.

Correctness. Correctness requires that honestly generated credentials and honestly generated showing proofs are accepted by the verifier. More formally, for every λ , every

$$\text{pp} \leftarrow \text{AC.Setup}(1^\lambda),$$

every honestly generated user key pair and issuer key pair

$$(\text{usk}, \text{upk}) \leftarrow \text{AC.UserKeyGen}(\text{pp}), \quad (\text{isk}, \text{ipk}) \leftarrow \text{AC.IssuerKeyGen}(\text{pp}),$$

every honestly issued credential

$$\text{cred} \leftarrow \langle \text{User}(\text{pp}, \text{usk}, \text{upk}, A), \text{Issuer}(\text{pp}, \text{isk}, \text{ipk}, \text{upk}, A) \rangle.$$

and every predicate $\phi \in \Phi$, if $\phi(A) = 1$, then

$$\Pr[\text{AC.Verify}(\text{pp}, \text{ipk}, \text{stmt}, \pi) = 1] \geq 1 - \text{negl}(\lambda),$$

where

$$(\text{stmt}, \pi) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}, \text{usk}, \text{ipk}, A, \phi).$$

The probability is taken over the randomness of the setup, key-generation, issuance, showing, and verification algorithms.

Unforgeability. Unforgeability requires that no adversary can produce an accepting showing proof unless it knows a credential honestly issued by an issuer and a corresponding user secret key satisfying the statement. Let $\mathcal{L}$ be the issuance log containing all tuples

$$(\text{ipk}, \text{upk}, A, \text{cred})$$

honestly issued during the experiment. The adversary may interact with issuing oracles and then outputs $(\text{ipk}^*, \text{stmt}^*, \pi^*)$. The adversary wins if

$$\text{AC.Verify}(\text{pp}, \text{ipk}^*, \text{stmt}^*, \pi^*) = 1$$

and there is no tuple

$$(\text{ipk}^*, \text{upk}^*, A^*, \text{cred}^*) \in \mathcal{L}$$

and no user secret key usk^* such that $(\text{usk}^*, \text{upk}^*)$ is an honestly generated user key pair such that, for

$$w^* = (\text{cred}^*, \text{usk}^*, A^*),$$

we have

$$R_{\text{AC}}(\text{stmt}^*; w^*) = 1.$$

We write $\text{Adv}_{\text{AC}}^{\text{unf}}(\mathcal{A})$ for the probability that $\mathcal{A}$ wins this experiment; the scheme is unforgeable if $\text{Adv}_{\text{AC}}^{\text{unf}}(\mathcal{A}) \leq \text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$.

Anonymity. Anonymity requires that a showing proof does not reveal which honest user generated it, beyond the information intentionally disclosed in the public statement. This is a cryptographic indistinguishability notion: non-cryptographic side channels, such as wall-clock timing, network and ledger-access patterns, revocation-update timing, and issuer or wallet metadata observed out of band, are outside the model and are not covered by the bound below. In the anonymity experiment, the adversary chooses two honestly issued credentials

$$(\text{cred}_0, \text{usk}_0, A_0), \quad (\text{cred}_1, \text{usk}_1, A_1),$$

and a predicate ϕ , such that both credentials satisfy the predicate,

$$\phi(A_0) = \phi(A_1) = 1,$$

and the corresponding public statements reveal the same allowed leakage. More formally, if

$$(\text{stmt}_0, \pi_0) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_0, \text{usk}_0, \text{ipk}, A_0, \phi)$$

and

$$(\text{stmt}_1, \pi_1) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_1, \text{usk}_1, \text{ipk}, A_1, \phi),$$

then the challenge is well formed only if

$$\text{Leak}(\text{stmt}_0) = \text{Leak}(\text{stmt}_1),$$

where $\text{Leak}(\text{stmt})$ is the projection of the public statement onto its intentionally-disclosed components, namely the revealed attribute set $A^\downarrow$, the teaching-authority pseudonym multiset $\{\text{ppk}_{U,TA}^{(j)}\}$, and any

linking tag carried in the predicate ϕ ; the verifier-facing pseudonym $\text{ppk}_{U,V}$ is freshly sampled at showing time, and the remaining statement components are witness-derived and made independent of the challenge bit by the simulators of the anonymity argument (Appendix A). The challenger samples $b \leftarrow \{0, 1\}$ and returns

$$(\text{stmt}, \pi_b) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_b, \text{usk}_b, \text{ipk}, A_b, \phi).$$

The adversary outputs b' . We write $\text{Adv}_{\text{AC}}^{\text{anon}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$; the scheme is anonymous if $\text{Adv}_{\text{AC}}^{\text{anon}}(\mathcal{A}) \leq \text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$.

Unlinkability. Unlinkability requires that a shown credential cannot be linked to the pseudonym under which it was issued, unless the public statements intentionally contain a linking tag. In the unlinkability experiment, with the oracle interface of the unforgeability experiment, the challenger creates two pseudonym key pairs $(\text{psk}^0, \text{ppk}^0)$ and $(\text{psk}^1, \text{ppk}^1)$ and an attribute set $A^\downarrow$, samples $b \leftarrow \{0, 1\}$, and issues a single credential $c^{(b)}$ with respect to $\text{ppk}^{(b)}$ on $A^\downarrow$, forwarding it to the adversary. The adversary outputs b' . We write $\text{Adv}_{\text{AC}}^{\text{unlink}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$, and the scheme is unlinkable if it is $\text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$. When the statements intentionally contain a public linking tag, linkability is allowed only as determined by that tag.

3.5.1 Blockchain-based Digital Education Credentials (BDEC)

BDEC is a blockchain-based anonymous credential system for educational credentials. It is parameterized by a digital signature scheme Sig and a zkSNARK Π . The main parties are a user U , teaching authorities TA , and a verifier V .

The blockchain stores credential-record digests, authorized teaching-authority keys, and revocation state. We assume that H and H_{rev} are collision-resistant and domain-separated hash functions. In the privacy-preserving case, the blockchain stores record digests ρ rather than raw credential attributes.

Construction 1. The BDEC construction uses Sig through Sig.Setup , Sig.KeyGen , Sig.Sign , and Sig.Verify . It uses Π to prove relations containing signature-verification predicates, ledger-membership checks, authorized-TA checks, and revocation checks. The BDEC algorithms instantiate the minimal anonymous-credential syntax introduced above as follows: $\text{BDEC.Setup} \leftrightarrow \text{AC.Setup}$; $\text{BDEC.PriGen} \leftrightarrow \text{AC.UserKeyGen}$; $\text{BDEC.TAKeyGen} \leftrightarrow \text{AC.IssuerKeyGen}$; issuance BDEC.IssueCre with credential-proof generation BDEC.CreGen together realising AC.Issue ; $\text{BDEC.ShowCre} \leftrightarrow \text{AC.Show}$; and $\text{BDEC.ShowVer} \leftrightarrow \text{AC.Verify}$. The pseudonym-key, credential-verification, and revocation steps (BDEC.NymKey , BDEC.CreVer , BDEC.RevCre) are BDEC-specific refinements with no counterpart in the minimal syntax.

BDEC.Setup. On input 1^λ , compute

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda), \quad \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda).$$

Output the static parameters $\text{par} = (\text{pp}_{\text{Sig}}, \text{crs})$. At epoch e (the blockchain epoch index, distinct from the power-residue PRF modulus $t = 256$), the public blockchain state is

$$\text{st}_e = (\text{rt}_{\text{cred},e}, \text{rt}_{TA,e}, \text{rt}_{LR,e}),$$

where $\text{rt}_{\text{cred},e}$ is the accepted credential-record root, $\text{rt}_{TA,e}$ is the authorized teaching-authority registry root, and $\text{rt}_{LR,e}$ is the revocation-state root.

BDEC.PriGen. On input par , compute the user's key pair

$$(\text{sk}_U, \text{pk}_U) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$$

and output $(\text{sk}_U, \text{pk}_U)$.

BDEC.TAKeyGen. On input par , a teaching authority computes

$$(\text{sk}_{TA}, \text{pk}_{TA}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$$

and publishes pk_{TA} . The public key is accepted only if it is authenticated externally or included in the authorized-TA registry represented by $\text{rt}_{TA,e}$.

BDEC.NymKey. On input $(\text{par}, \text{sk}_U)$, sample a pseudonym $\text{ppk}_{U,TA} \leftarrow \{0, 1\}^\lambda$ and compute

$$m_{\text{nym},U,TA} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,TA}) \in \mathcal{M}_{\text{pp}_{\text{Sig}}}, \quad \text{psk}_{U,TA} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, m_{\text{nym},U,TA}).$$

Output $(\text{ppk}_{U,TA}, \text{psk}_{U,TA})$. This user-side signature binds the pseudonym to the user's key; it does not certify educational attributes.

BDEC.IssueCre. On input $(\text{par}, \text{sk}_{TA}, \text{pk}_U, \text{ppk}_{U,TA}, A, \text{aux})$, the teaching authority first checks externally that the user is entitled to the attributes A . If the check accepts, define

$$m_{\text{cred},U,TA} = \text{Encode}_{\text{Sig}}(\text{pk}_U, \text{ppk}_{U,TA}, A, \text{aux}) \in \mathcal{M}_{\text{pp}_{\text{Sig}}}$$

and compute

$$\sigma_{\text{cred},U,TA} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_{TA}, m_{\text{cred},U,TA}).$$

The corresponding blockchain record digest is

$$\rho_{U,TA} = H(\text{pk}_{TA}, \text{ppk}_{U,TA}, A, \text{aux}, \sigma_{\text{cred},U,TA}).$$

The blockchain stores $\rho_{U,TA}$ in the accepted credential-record ledger. It need not store A , aux , or $\sigma_{\text{cred},U,TA}$ in the clear.

BDEC.CreGen. On input $(\text{par}, \text{sk}_U, \text{pk}_U, \text{ppk}_{U,TA}, \text{psk}_{U,TA}, A, \text{aux})$, the algorithm first computes the attribute digest $h_{U,TA} = H(A)$ and the credential $c_{U,TA} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, h_{U,TA})$. It then generates a proof with respect to the zkSNARK statement and witness; in a zkVM instantiation the public-instance components x_{cre} below must be committed to the receipt journal for the proof to bind to them; this is a statement-binding requirement on which soundness depends, and its status in the prototype is treated in Section 5,

$$\begin{aligned} x_{\text{cre}} &= (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA}), \\ w_{\text{cre}} &= (\text{pk}_U, \text{psk}_{U,TA}), \end{aligned}$$

where the proved relation is the conjunction of two signature verifications, both under the user's long-term public key pk_U :

$$\begin{aligned} R_{\text{cre}}^{\text{Sig}}(x_{\text{cre}}; w_{\text{cre}}) = 1 &\iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}_U, h_{U,TA}, c_{U,TA}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}_U, \text{ppk}_{U,TA}, \text{psk}_{U,TA}) = 1. \end{aligned}$$

Equivalently, the circuit proved by the zkSNARK is the AND concatenation of two signature-verification algorithms [1]: the first attests that $c_{U,TA}$ is a valid credential on the attribute digest, the second that the user controls the pseudonym $\text{ppk}_{U,TA}$. Compute

$$\varepsilon_{c_{U,TA}} \leftarrow \Pi.\text{Prove}(\text{crs}, x_{\text{cre}}, w_{\text{cre}})$$

and output $(c_{U,TA}, \varepsilon_{c_{U,TA}})$.

Remark 3.7 (Optional Merkle accumulator; not part of the base relation). The base $R_{\text{cre}}^{\text{Sig}}$ above is exactly the two-verification relation of [1], for which that work’s unforgeability and anonymity theorems are proved. To additionally *hide* attributes at presentation time, the same work suggests an optional Merkle-accumulator variant: the user signs the Merkle *root* of the attributes in BDEC.CreGen rather than $H(A)$, and the membership (authentication-path) checks are performed by the verifier in BDEC.ShowVer, not inside $R_{\text{cre}}^{\text{Sig}}$. This thesis implements and measures the base two-verification relation; the accumulator variant, and any consequent re-derivation of the security reductions for an enlarged relation, are out of scope and would constitute a strengthening beyond the cited construction rather than a re-instantiation of it.

BDEC.CreVer. On input $(\text{par}, \text{st}_e, x_{\text{cre}}, \varepsilon_{c_{U,TA}})$, output

$$b \leftarrow \Pi.\text{Verify}(\text{crs}, x_{\text{cre}}, \varepsilon_{c_{U,TA}}) \in \{0, 1\}.$$

BDEC.ShowCre. On input $(\text{par}, \text{st}_e, \text{pk}_U, (\text{ppk}_{U,TA}^{(j)}, \text{psk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}, \text{psk}_{U,V}, c_{U,V}, A^\downarrow, \phi)$, where $A^\downarrow$ are the selectively disclosed attributes, ϕ the claimed presentation predicate, and k the number of accepted credential records used, the algorithm forms the public instance

$$x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}).$$

The private witness is

$$w_{\text{show}} = (\text{pk}_U, (\text{psk}_{U,TA}^{(j)})_{j=1}^k, \text{psk}_{U,V}, c_{U,V}).$$

Following the base relation of [1], the witness is the user’s long-term verification key pk_U , used here as a *private* witness despite being a public key, together with the pseudonym secret keys and the shown credential. Hiding pk_U is what delivers anonymity, since the showing is then not tied to the user’s identity. The pseudonym public keys and the disclosed attributes $A^\downarrow$ are public, so the messages verified inside the proof are formed from public values and the verifier evaluates ϕ on $A^\downarrow$. Membership, non-revocation, and the disclosure check are performed by the verifier in BDEC.ShowVer, which additionally takes the disclosed predicate ϕ , the published roots $\text{rt}_{\text{cred},e}$, $\text{rt}_{TA,e}$, $\text{rt}_{LR,e}$, the credential commitments $(\rho^{(j)})_{j=1}^k$, the membership paths, and the revocation tag r_U . These are verifier-side inputs and are not part of the proof statement or witness. Concealing the attributes outside $A^\downarrow$ is the optional Merkle-accumulator strengthening of Remark 3.7, which this thesis does not implement.

Concrete presentation predicates. The predicate class Φ is the set of Boolean combinations of arithmetic comparisons and equalities over attribute values. Two predicates serve as running examples throughout the thesis and as the workload for the update-churn measurement of Section 7. The

first, used at credential count $k = 1$, is a single-attribute threshold; writing $A.\text{gpa}$ for the grade-point attribute,

$$\phi_1(A) = [A.\text{gpa} > 3.5].$$

The second, used at $k = 2$, is a conjunction over two credentials drawn from possibly distinct issuers; writing $A.\text{degree}$, $A.\text{gradYear}$, and $A.\text{major}$ for the relevant attributes,

$$\phi_2(A^{(1)}, A^{(2)}) = [A^{(1)}.\text{degree} = \text{Bachelor}] \wedge [A^{(1)}.\text{gradYear} > 2020] \wedge [A^{(2)}.\text{major} = \text{CS}].$$

Both predicates are evaluated by the verifier on the disclosed attribute set $A^\downarrow$, *outside* the proof relation $R_{\text{show}}^{\text{Sig}}$: following base BDEC (Remark 3.7), the attributes named in ϕ are disclosed in the clear and the verifier checks $\phi(A^\downarrow) = 1$ on the revealed values. For ϕ_1 the GPA is disclosed and the verifier checks the threshold; for ϕ_2 the degree, graduation year, and major are disclosed and the verifier checks the conjunction. The privacy the showing provides is *anonymity*, in that the proof is not linked to the user's identity pk_U , together with unlinkability across showings; it is *not* concealment of the disclosed attribute *values*. Proving a predicate over an attribute that stays hidden (e.g. “gpa > 3.5” without revealing the GPA) is beyond the base relation and would require the optional Merkle-accumulator strengthening that Remark 3.7 places out of scope. The two predicates differ in arity ($k = 1$ versus $k = 2$), in issuer multiplicity (one issuer versus two), and in Boolean structure (atomic versus a three-clause conjunction). Because ϕ_1 and ϕ_2 require a *different number of credentials*, moving between them changes the presentation *structure*, namely the credential count k , and hence the relation $R_{\text{show}}^{\text{Sig}}$ itself, which is the source of the static-circuit recompilation cost measured in Section 7; a policy change at fixed structure (e.g. retuning the GPA threshold) is absorbed entirely by the verifier-side check and forces no recompilation. For the threat model of Section 6, the timing of ϕ matters: it is chosen by the verifier at presentation time and is not known when the credential is issued.

For each $j \in [k]$, define $m_{\text{nym}}^{(j)} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,TA}^{(j)})$. Also define $m_{\text{nym},U,V} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,V})$ and $m_{\text{show}} = \text{Encode}_{\text{Sig}}(A^\downarrow)$, the encoding of the disclosed attributes carried by the shown credential $c_{U,V}$. The showing relation is

$$\begin{aligned} R_{\text{show}}^{\text{Sig}}(x_{\text{show}}; w_{\text{show}}) = 1 &\iff \bigwedge_{j=1}^k \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{PPSig}}(\text{pk}_U, m_{\text{nym}}^{(j)}, \text{psk}_{U,TA}^{(j)}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{PPSig}}(\text{pk}_U, m_{\text{nym},U,V}, \text{psk}_{U,V}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{PPSig}}(\text{pk}_U, m_{\text{show}}, c_{U,V}) = 1. \end{aligned}$$

Following BDEC [1], the proof circuit contains $k + 2$ signature-verification predicates: k pseudonym-ownership checks (each binding a teaching-authority pseudonym to pk_U), one verifier-facing pseudonym check, and one check on the shown credential $c_{U,V}$ over the disclosed attributes. The relying party additionally checks, *outside* the proof: accepted-ledger membership and authorised-TA membership against the published roots, non-revocation against the revocation tree, and that the disclosed subset $A^\downarrow$ satisfies the presentation predicate ϕ . Selective disclosure is realised by the chosen subset $A^\downarrow$; no in-circuit predicate enforces it. Compute

$$\varepsilon_{c_{U,V}} \leftarrow \Pi.\text{Prove}(\text{crs}, x_{\text{show}}, w_{\text{show}})$$

and output $\varepsilon_{c_{U,V}}$.

BDEC.ShowVer. On input $(\text{par}, \text{st}_e, x_{\text{show}}, \varepsilon_{c_{U,V}})$, output

$$b \leftarrow \Pi.\text{Verify}(\text{crs}, x_{\text{show}}, \varepsilon_{c_{U,V}}) \in \{0, 1\}.$$

The verifier first checks the zkSNARK proof against the public statement, which attests the $k + 2$ signature-verification predicates of $R_{\text{show}}^{\text{Sig}}$. The relying party then performs the ledger-membership, authorised-TA, non-revocation, and selective-disclosure checks *outside* the proof, against the published roots, the revocation tree, and the disclosed subset $A^\downarrow$, consistent with the relation above.

BDEC.RevCre. On input $(\text{par}, \text{pk}_U)$, compute

$$r_U = H_{\text{rev}}(\text{pk}_U)$$

and update the revocation state so that r_U is included in the revocation list represented by $\text{rt}_{\text{LR},e}$. Future showings are accepted only if the prover supplies a valid non-membership witness for r_U with respect to the current revocation root.

Conditional linkability. Conditional linkability allows controlled linking only when the user intentionally discloses linking information. Let τ be an optional public linking tag contained in the statement. There is a deterministic algorithm Link such that

$$\text{Link}(\text{stmt}_0, \text{stmt}_1) = 1 \iff \tau_0 = \tau_1 \neq \perp.$$

If no tag is disclosed, then linkability reduces to the unlinkability requirement above. The tag τ is chosen by the *user* at showing time, not by the verifier; we assume an honest user does not reuse a tag across contexts they wish to keep unlinkable, and we treat verifier-induced linkability (a verifier that refuses service unless the user supplies a fixed τ) as an out-of-scope deployment risk rather than a property of the scheme.

Revocability. Let LR be a revocation state represented by $\text{rt}_{\text{LR},e}$. A showing is revocation-valid only if

$$r_U = H_{\text{rev}}(\text{pk}_U) \quad \wedge \quad \text{NonMemberVerify}(\text{rt}_{\text{LR},e}, r_U, \text{path}_{\text{LR}}) = 1.$$

If r_U is added to LR, then any future showing transcript linked to r_U is rejected except with negligible probability. For honest, unrevoked users, correctness continues to hold.

3.6 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose relation is fixed once, to the correctness of executing a guest program on a deterministic instruction-set architecture. In contrast to a relation-specific zkSNARK such as Aurora, where the rank-1 constraint system encoding a relation R is compiled together with the proof system, a zkVM commits to a single universal relation R_{vm} describing its instruction-set semantics, and treats the guest program as an input to that relation. Background surveys and step-by-step expositions of this paradigm are given in [6, 8].

Definition 3.8 (Zero-knowledge virtual machine). Let ISA be a deterministic instruction-set architecture with a fixed cycle-level semantics. A zkVM for ISA is a zkSNARK

$$\Pi_{\text{vm}} = (\Pi_{\text{vm}}.\text{Setup}, \Pi_{\text{vm}}.\text{Prove}, \Pi_{\text{vm}}.\text{Verify})$$

for the universal relation

$$R_{\text{vm}}((\text{img}, \text{in}_{\text{pub}}, \text{out}); (P, \text{in}_{\text{priv}})) = 1$$

iff $\text{img} = \text{digest}(P)$ and executing the guest program P on ISA with inputs $(\text{in}_{\text{pub}}, \text{in}_{\text{priv}})$ terminates and emits public output out . The public instance is $x_{\text{vm}} = (\text{img}, \text{in}_{\text{pub}}, \text{out})$ and the witness is $w_{\text{vm}} = (P, \text{in}_{\text{priv}})$.

Host-guest model. A zkVM partitions execution into a *guest* program that runs on the virtual processor under the proof system, and a *host* program that runs natively on the prover machine. The host orchestrates proof generation, supplies inputs to the guest via a read channel, and receives the guest’s public output via a commit channel. Values placed on the commit channel become part of the public statement out ; values placed on the read channel that the guest does not commit remain in the witness in_{priv} . In the BDEC instantiation considered in this thesis, the user’s long-term secret key sk_U and the credential signatures appear only via the read channel and are never committed, so they remain in w_{vm} across the entire zkSNARK relation.

Receipt structure and the zero-knowledge property. The output of $\Pi_{\text{vm}}.\text{Prove}$ is a *receipt* consisting of a journal of committed outputs together with a succinct proof. Concrete instantiations distinguish two regimes:

- A *succinct STARK receipt* is transparent and large, and verifies in time logarithmic in the number of executed cycles. Whether such a receipt satisfies the formal zero-knowledge property of Section 3 depends on the specific instantiation; in the systems we evaluate the default succinct STARK receipt is not formally zero-knowledge.
- A *SNARK-wrapped receipt* is obtained by composing the succinct STARK with an outer zero-knowledge SNARK (e.g. Groth16 or PLONK). The wrapped receipt is small and zero-knowledge, but the wrap step itself has additional prover memory and time cost.

Any privacy claim about an anonymous-credential scheme instantiated on a zkVM that relies on the underlying zkSNARK being zero-knowledge therefore holds only for the wrapped receipt. We make this dependency explicit when stating anonymity and unlinkability of the BDEC instantiation in Section 5.

Precompiles. A *precompile* is a fixed sub-circuit recognised by the zkVM’s instruction decoder as a syscall and proved natively by the prover rather than via emulated instruction-level execution. From the guest’s perspective a precompile call costs a single syscall instruction; from the prover’s perspective the cost is the proof of one execution of the precompile’s underlying AIR (algebraic intermediate representation). Precompiles trade proof-system generality for performance on algebraic primitives whose software emulation over the prover’s native field would otherwise dominate the trace. A precompile is sound only if (i) its AIR enforces exactly the intended function, accepting no more inputs than the

reference specification, and (ii) its memory inputs and outputs are bound to the surrounding VM execution trace via the cross-table lookup argument; otherwise the syscall could be satisfied by values unrelated to the guest’s computation. This functional-equivalence-plus-trace-binding condition is the premise addressed for the Griffin precompile in Section 6 (premise T2), where its status is examined.

We refer to the cost of emulating n -bit field arithmetic in software when the underlying prover field is small as the *field-mismatch tax*. A precompile that implements the same arithmetic natively absorbs this tax. The Griffin- $\mathbb{F}_p^{192}$ precompile introduced and measured in this thesis is the load-bearing instance of this pattern.

Update churn. *Update churn* denotes the engineering and verification cost incurred when a deployed proof system must be changed in response to evolving policy, security parameters, or primitive substitutions. In a relation-specific zkSNARK every such change requires regenerating and re-auditing the relation-specific circuit and its parameters; this regeneration-and-re-audit footprint, rather than the recompile wall-clock (which can be small for a transparent argument), is the dominant cost. In a zkVM the same change is a guest-program edit; the universal relation R_{vm} and the trusted setup (if any) are unchanged. Update churn is the evaluation dimension we add to runtime cost in Section 7.

3.6.1 RISC Zero

RISC Zero [7] is a zkVM instantiation whose ISA is RV32IM. The prover is a recursive STARK over the BabyBear field $\mathbb{F}_p$ with $p = 2^{31} - 2^{27} + 1$. Per [7], the prover decomposes guest execution into segments of bounded cycle count, produces a STARK proof per segment, and recursively folds segment proofs into a single succinct receipt. Application-defined precompiles are supported via the Zircen big-integer dialect.

A RISC Zero receipt is verified against the public statement (img, out), where img is the digest of the guest ELF and out is the contents of the commit journal. The image digest pins the receipt to a specific guest binary: a receipt for one guest does not verify under the image digest of any other guest.

3.6.2 SP1

SP1 [9] is a second zkVM instantiation whose ISA is also RV32IM and whose prover is built on the Plonky3 toolkit. Per the SP1 documentation, the default prove path produces a succinct STARK receipt that is not formally zero-knowledge; a zero-knowledge receipt is obtained by chaining the succinct STARK with a SNARK wrap, e.g. a Groth16 or PLONK wrap (`compressed().groth16()` or `compressed().plonk()` in the SP1 SDK). SP1 supports application-defined precompiles via its custom-precompile framework.

3.6.3 Static-circuit and zkVM proof systems as comparators

We will instantiate BDEC in two regimes that this thesis compares directly:

1. A *static-circuit* regime, in which the BDEC CreGen and ShowCre relations are encoded into a fixed R1CS and proved via Aurora (Section 3 above). Each change to the relation requires recompilation of the R1CS.

2. A *zkVM* regime, in which the same relations are expressed as guest programs on RISC Zero (and SP1, where indicated). Changes to the relation become guest-program edits; the universal relation R_{vm} is unchanged.

The runtime cost, memory footprint, and update-churn properties of these two regimes are the empirical subject of Section 7.

4 Theoretical Foundations of the Arithmetic Mismatch

This section sets up the formal cost framework against which the rest of the thesis is measured. We name the dimensions along which the static-circuit and zkVM regimes will be compared (Section 4.1), formalise the *field-mismatch tax* as the quantitative manifestation of the arithmetic mismatch (Section 4.2), formalise *update churn* as a deployment-lifecycle dimension (Section 4.3), and decompose the cost of the BDEC CreGen and ShowCre relations into the components that dominate either regime (Section 4.4).

4.1 Cost Dimensions

We measure each system regime along four dimensions.

Time. Prover and verifier wall-clock time, measured in seconds on the target hardware. For the prover this is the time from invocation of $\Pi.\text{Prove}$ (or $\Pi_{\text{vm}}.\text{Prove}$) until the receipt is produced; for the verifier it is the time from invocation of the verification predicate until the binary outcome is returned.

Memory. Peak resident-set size (peak RSS) of the prover process on the target hardware, measured in gigabytes. Verifier memory is reported when it materially differs from prover memory; otherwise it is dominated by the receipt size.

Proof size. The size of the produced receipt, measured in kilobytes for the relevant receipt class. We report this separately for the succinct STARK receipt and the SNARK-wrapped receipt, the two receipt regimes of Section 3, where each is produced.

Update churn. The footprint of artefacts that must be regenerated and re-audited when the verification predicate changes by one localised modification. We formalise this dimension in Section 4.3 below.

The first three dimensions are standard in the zkSNARK and zkVM literature [3, 5, 7, 9, 31]. The fourth dimension is introduced in this thesis as a deployment-lifecycle accounting frame. Unlike the first three it is a footprint characterisation rather than a performance metric, and Section 7 shows that it does not translate into a wall-clock advantage against the non-preprocessing static baseline.

4.2 The Field-Mismatch Tax

The verification predicate of a post-quantum signature such as Loquat or PLUM is dominated by algebraic-hash (Griffin) permutations together with arithmetic over a prime field $\mathbb{F}_p$ of large characteristic (in PLUM, a nominal 192-bit / concrete 199-bit prime; see Section 3). Concretely, PLUM [5, §4.2] reports that hash evaluations account for 105,952 of the 116,285 R1CS constraints ($\approx 91\%$) of a PLUM-128 verification, the field-arithmetic constraints making up the remaining $\approx 9\%$; the tax below is carried by this hash-dominated, large-field workload as a whole, and not by field arithmetic on its own. This 91% is a constraint count in a relation-specific zkSNARK and is not a measured zkVM-cycle share: it does not, on its own, fix an Amdahl ceiling on the zkVM cost, and the measured zkVM-cycle attribution is reported separately in Section 7. When such a predicate is proved by a relation-specific

zkSNARK over $\mathbb{F}_p$, every field operation in the predicate is a single constraint over the same field. When it is proved by a zkVM whose prover field $\mathbb{F}_{p_{vm}}$ is small (e.g. BabyBear, $p_{vm} = 2^{31} - 2^{27} + 1$), each $\mathbb{F}_p$ operation must be emulated by software running on the zkVM's RISC-V instruction set, which in turn is encoded in $\mathbb{F}_{p_{vm}}$. The number of native RISC-V instructions required to emulate one $\mathbb{F}_p$ operation by multi-limb arithmetic is a fixed function of the bit-widths of p and the limb size of the underlying integer encoding.

The emulation tax this definition names is characterised qualitatively in the recent zkVM systematisation [40]. The definition below is a cost-accounting frame that names, in one symbol, the per-operation overhead the precompile suite is built to remove.

Definition 4.1 (Field-mismatch tax). Let op be a field operation over $\mathbb{F}_p$ used inside a verification predicate $Pred$, and let $cyc_{em}(op)$ denote the number of zkVM guest cycles required to emulate op by software on a zkVM whose native field is $\mathbb{F}_{p_{vm}}$. Let $cyc_{nat}(op)$ denote the number of zkVM guest cycles that the same operation would consume if the prover field were $\mathbb{F}_p$ itself, so that no multi-limb emulation is needed; under this idealisation $cyc_{nat}(op)$ is the cycle count of a single native field instruction. Because the target zkVMs expose no native $\mathbb{F}_p$ instruction, cyc_{nat} is a conceptual normaliser, not a directly measurable quantity; the empirically measured quantity is the emulated cost Cyc_{em} reported in Section 7. Both quantities are in the same unit, so their ratio is dimensionless. The *field-mismatch tax* of op is

$$FMT(op) = \frac{cyc_{em}(op)}{cyc_{nat}(op)}.$$

Writing $ops(Pred)$ for the multiset of field operations executed by an honest evaluation of $Pred$, the cumulative *emulated cost* of $Pred$ is the distinct quantity

$$Cyc_{em}(Pred) = \sum_{op \in ops(Pred)} cyc_{em}(op).$$

A precompile, as defined in Section 3, replaces $cyc_{em}(op)$ for a designated operation by a single zkVM syscall, whose cost from the guest's perspective is one cycle plus the constant overhead of the syscall ABI. Writing $Pre \subseteq ops(Pred)$ for the set of operations covered by precompiles and $cyc_{syscall}$ for the per-syscall overhead, the cumulative emulated cost of $Pred$ under a precompile-enabled zkVM is therefore

$$Cyc_{em}^{pre}(Pred) = \sum_{op \in ops(Pred) \setminus Pre} cyc_{em}(op) + |\{op \in ops(Pred) \cap Pre\}| \cdot cyc_{syscall}.$$

The performance benefit of adding a primitive to the precompile set is, by this accounting, the difference $cyc_{em}(op) - cyc_{syscall}$ summed over its occurrences. This is a *guest-trace* reduction: it counts cycles removed from the emulated execution, not the prover's wall-clock cost. The precompile shifts those cycles into a relation-specific AIR whose proving cost is charged on the host side, so a positive guest-cycle benefit need not translate into a faster prover, and Section 7 reports a primitive (Griffin) for which it does not.

4.3 Update Churn

Definition 4.2 (Localised verification-predicate change). A *localised verification-predicate change* is a modification to the verification predicate $Pred$ that can be described as one of the following: substitution of one signature scheme for another at matching security level; retuning of a single numerical

security parameter (e.g. λ, t, η); a change to the presentation *structure* of a showing relation (the number of credentials shown, or the set of disclosed attributes); or a one-line change to the encoding of a public input. A change that only alters which condition the verifier checks on the *disclosed* attributes, a presentation *policy* at fixed structure, is *not* a verification-predicate change: following base BDEC it is evaluated outside the proof relation (Section 3) and touches neither Pred nor its compiled artefacts.

The deployment-lifecycle cost of a localised change is dominated by the artefacts that must be regenerated and re-audited as a consequence, rather than by the change itself. We formalise this cost as follows.

Definition 4.3 (Update churn). Fix a deployment $\mathcal{D}$ that proves Pred in either the static-circuit or the zkVM regime, and let Δ denote a localised verification-predicate change. The *update-churn footprint* of Δ in $\mathcal{D}$ is the tuple

$$\text{Churn}(\Delta, \mathcal{D}) = (N_{\text{src}}(\Delta, \mathcal{D}), N_{\text{circ}}(\Delta, \mathcal{D}), N_{\text{param}}(\Delta, \mathcal{D}), N_{\text{audit}}(\Delta, \mathcal{D})),$$

where

N_{src} is the number of source-code files that must be modified in response to Δ .

N_{circ} is the number of arithmetic circuits or AIR programs that must be regenerated, counted as the distinct circuit or AIR artefacts whose serialised form changes between the pre- and post- Δ builds.

N_{param} is the number of proof-system parameters (e.g. commitment parameters, R1CS-shape-dependent constants) that must be re-derived.

N_{audit} is the number of distinct components requiring re-audit before redeployment, counted at the granularity of the deployment’s existing security-review boundary. This is an *audit-footprint* count, not a weighted effort model: it reports how many components re-enter the review boundary, not the person-time or risk class of reviewing them.

For the static-circuit regime, a localised change to Pred typically yields a non-trivial N_{circ} (the R1CS must be recompiled), a non-trivial N_{param} (constants that depend on the constraint count must be re-derived), and a non-trivial N_{audit} (the new circuit shape must be audited). For the zkVM regime, the same change always yields a non-trivial N_{src} (the guest program is edited) and zero regenerations of the *universal* relation R_{vm} , which is unchanged. The remaining components depend on whether the change touches a precompiled primitive. The bespoke Griffin _{$\mathbb{F}_p^{192}$} AIR is itself a relation-specific circuit; a change that alters the hash family or the field width therefore regenerates and re-audits that AIR, so $N_{\text{circ}} \geq 1$, $N_{\text{param}} \geq 1$, and $N_{\text{audit}} \geq 1$. The zero-circuit, zero-parameter case applies only to changes that leave every precompiled primitive intact (e.g. raising the presentation count k from 1 to 2), for which $N_{\text{circ}} = 0$, $N_{\text{param}} = 0$, and N_{audit} is confined to the changed guest source. The configuration that would need zero new circuits for *any* change is the pure-emulation zkVM without precompiles, which Section 7 measures as infeasible (does not finish within the resource bound on the target hardware).

The empirical evaluation of Section 7 reports each of these four components for both regimes under a fixed set of localised changes that we exhibit as benchmarks: signature substitution (Loquat $\rightarrow$ PLUM), security-parameter retuning ($\lambda = 80 \rightarrow 128$), and a presentation-structure change (raising the credential count k from 1 to 2).

4.4 Cost Decomposition of BDEC Relations

We now apply the cost framework above to the two BDEC relations of interest. We treat each relation as a multiset of cost-bearing components and compute the cumulative cost under each regime.

CreGen. The BDEC CreGen relation $R_{\text{cre}}^{\text{Sig}}$ defined in Section 3 is the base two-verification relation: two signature-verification predicates $\text{Pred}_{\text{Verify}}^{\text{Sig,PPSig}}$ under a hidden user public key. The credential digest $\rho_{U,TA} = H(\cdot)$ is computed at issuance, and ledger membership is checked outside the proof, as in ShowCre; the full acceptance condition is therefore the conjunction of the in-proof signature-verification predicates with the verifier-side membership check performed by BDEC.CreVer/BDEC.ShowVer (Section 3), so a proof for valid signatures whose digest is absent from the accepted ledger is rejected outside the proof. Of these, the two $\text{Pred}_{\text{Verify}}^{\text{Sig,PPSig}}$ predicates dominate the cost. Inside PLUM verification (Section 3), the load-bearing components are (i) the t -th power-residue PRF symbol checks, (ii) the BCS Merkle-commitment recomputations, and (iii) the low-degree-test (LDT) sub-protocol invocations. The cost of (ii) and (iii) is dominated by the algebraic hash function H_{Griffin} over $\mathbb{F}_p$, which is precisely the primitive absorbed by the $\text{Griffin}_{\mathbb{F}_p^{192}}$ precompile.

ShowCre. The ShowCre proof circuit contains $k + 2$ signature-verification predicates (where k is the number of credentials shown): k pseudonym-ownership checks, one verifier-facing pseudonym check, and one shown-credential check (Section 3). Ledger membership, non-revocation, and the disclosure predicate are checked by the relying party outside the proof. For modest k (the design target is $k \leq 4$), the cost is dominated by these signature-verification predicates and the algebraic-hash operations inside their Merkle and low-degree-test components.

In both relations, the field-mismatch tax of Definition 4.1 is concentrated in the signature-verification predicates: every Griffin permutation involved in a Merkle path or in PLUM’s internal hashing is a tax-bearing $\mathbb{F}_p^{192}$ operation under a zkVM with a small native field.

The decomposition locates the dominant term of both relations in the Griffin permutation count and the field-mismatch tax it bears. It also separates two costs that the framework keeps apart: the per-proof cost, which the precompile suite attacks, and the regeneration cost incurred when a relation changes, which only the static-circuit regime pays. Section 7 measures the per-proof term directly and characterises the regeneration term as a footprint: where the zkVM cost falls and how much of it the precompile removes, the resource frontier at which the unmitigated and the zero-knowledge-wrapped configurations cease to complete on the target hardware, and the recompile/setup time together with the enumerated regeneration footprint that separates the two regimes under a presentation, signature, or security-parameter change. The re-audit burden is reported as a component-count footprint and is not quantified as effort; the static baseline is Loquat/Aurora over $\mathbb{F}_{p_{127}}$ rather than same-scheme PLUM. The next chapter constructs the precompile suite that absorbs the dominant term this decomposition has located.

5 Construction: The Precompile Suite and PLUM-in-BDEC

The theoretical analysis of Section 4 identified the algebraic-hash and large-prime-field operations of PLUM verification as the operations that carry the field-mismatch tax inside a small-field zkVM. This section presents the instrument we built to absorb that tax so that the remaining proving cost can be measured: a precompile suite, developed in two stages, and its integration into the BDEC credential-generation relation. The suite serves the measurement reported in Section 7. Its purpose is to move the dominant cost out of emulated guest code so that what remains can be measured, attributed, and bounded. Throughout, we grade the evidence supporting each component, distinguishing what is proved from what is empirically tested and what is trusted from upstream, since the suite is not uniformly sound. Two soundness questions are left explicitly open (§5.5); a measurement whose correctness rested on unstated assumptions would be worth little, so stating the open questions is part of the work.

5.1 Scope of the Implementation

We state plainly what is and is not built, since the honesty of every measurement in Section 7 depends on this scope being explicit.

- **Fp192 modular multiplication via the host precompile** (§5.2), routed through SP1’s `UINT256_MUL` and RISC Zero’s `sys_bigint`, implemented for both targets. *Built; reduction argued; empirically tested (14/14 adversarial cases).*
- **A bespoke Griffin- $\mathbb{F}_p^{192}$ permutation AIR for SP1** (§5.3), the load-bearing precompile, since PLUM’s constraint budget is dominated by Griffin. All ten constraint families implemented ($\approx 3,900$ lines of Rust over 22 chip-only commits); a single-permutation smoke proof completes (execute+prove+verify) in 13.0 s on the target hardware. *Built; constraint-level chip-equivalence not yet verified (§5.5).*
- **PLUM signature verification on SP1**, suite active. *Built; proves end-to-end (32.5 min at $\lambda = 80$; Section 7).*
- **PLUM-in-BDEC credential generation (CreGen) on RISC Zero**, with field arithmetic via `sys_bigint`. *Built; validated in execute mode (9.4×10^9 guest cycles, both signature verifications accepting); its succinct proof did not yield a receipt, ending in an internal-verifier rejection under diagnosis rather than a resource limit (Section 7).*
- **PLUM-in-BDEC presentation (ShowCre) on RISC Zero**, the $k+2$ -verification generalisation of CreGen. *Built; validated in execute mode at $k = 1$ (1.41×10^{10} guest cycles) and $k = 2$ (1.88×10^{10} cycles), all $k+2$ verifications accepting; its succinct proof was not attempted, sharing CreGen’s prover path (Section 7).*

One component is deliberately *not* built, and we treat it as future work: a port of the bespoke Griffin AIR to RISC Zero’s syscall ABI. For reading Section 7 this matters, because “the precompile” does not denote one artefact uniform across both zkVMs. On SP1 the suite includes the bespoke Griffin AIR. On RISC Zero the same arithmetic is carried by `sys_bigint`. We state which mechanism is active wherever a measurement is reported.

5.2 The Field-Arithmetic Substrate: $\mathbb{F}_{p192}$ Multiplication via the Host Precompile

Field and naming. PLUM operates over the prime field $\mathbb{F}_p$ with $p = 2^{64} \cdot p_0 + 1$, p_0 a 135-bit prime, so p is a 199-bit prime. We retain the label “ $\mathbb{F}_p^{192}$ ” for parity with the PLUM paper, whose §3.3 titles the parameter “192-bit smooth prime field”; all soundness arguments use the true 199-bit modulus, which is fixed in the implementation and audited against the paper.³

Construction. We do not write a dedicated 199-bit multiplication circuit. Both target zkVMs already ship a 256-bit modular-multiplication precompile modulo a runtime-supplied modulus: SP1’s `UINT256_MUL` and RISC Zero’s `sys_bigint(OP_MULTIPLY)` [9, 7]. Each $\mathbb{F}_{p192}$ multiplication $a \cdot b \bmod p$ is computed by zero-padding a , b , and p into the precompile’s 256-bit slots and invoking the syscall with modulus $m = p$. The wiring is selected at compile time per target.

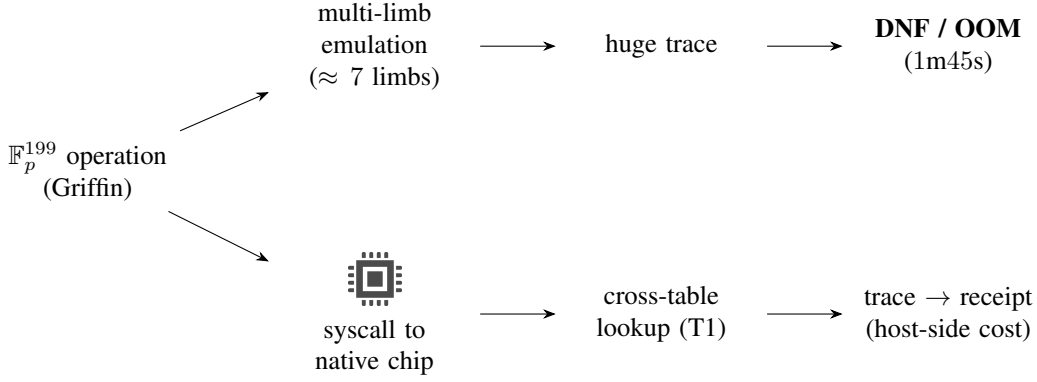
Soundness (reduction).

Upstream assumption. For every row of the host modular-multiplication table in a verifying proof, the host AIR enforces $x' \equiv x \cdot y \pmod{m}$ and $x' \in [0, m)$ when $m \neq 0$. For SP1 we point to the two specific AIR constraints that enforce these (the modular product and the output range check); for RISC Zero the same guarantee is enforced by auto-generated constraint code that we *cite* as production-audited rather than re-derive. This RISC Zero citation is the weakest link and is graded accordingly. The precompiles are those of SP1 v6.2.1 (forked) and RISC Zero 3.0.3 (the versions reported in Section 7), so the assumption is pinned to these specific versions and not to an unversioned “upstream release process.”

Reduction. $\mathbb{F}_{p192}$ operands are maintained in $[0, p)$ (structurally for the reducing constructor, by input domain for the small-integer constructors). Since $p < 2^{200} < 2^{256}$, the zero-pad is the identity on the bit pattern. The assumption with $m = p$ yields $x' = a \cdot b \bmod p \in [0, p)$, exactly the value the software fallback computes; the implementation reconstructs the field element directly from x' , omitting a redundant reduction, with a debug-build assertion guarding against regression of the upstream range check.

Edge cases. The argument addresses a malicious modulus $m = 0$ (prevented by loading the modulus from a compile-time constant the zkVM’s memory checking binds), non-canonical operand truncation (guarded by a debug assertion on operand width), and regression of the upstream range check (under which the omitted reduction would become load-bearing, detected by the same assertion).

³Two distinct points must not be conflated. (i) The 192-vs-199 *label* is a naming convention: PLUM’s §3.3 titles the field “192-bit” while the concrete modulus is 199 bits; we keep the paper’s label but compute with the true value. (ii) Separately, the specific decimal prime is a *substitution*: as recorded in Remark 3.5 and the open prime-instantiation hypothesis of Section 6, the value used is a nearby prime satisfying PLUM’s structural requirements, and any bit-exact security claim is conditional on it matching the intended parameters. This conditional dependence is an open item rather than a naming convention.



all-zero-padding-row identity failure.

What works. A single-permutation smoke proof passes execute, prove, and verify in 13.0 s on the target hardware, establishing the per-syscall proving cost in isolation.

5.4 The Sponge and Compression Wrapper

The permutation is wrapped in the sponge/compression construction of the Loquat instantiation to provide the 2-to-1 compression used by the BDEC Merkle trees. The state width is $t_{\text{state}} = 4$, the capacity $c_{\text{cap}} = 2$, and the rate $r = t_{\text{state}} - c_{\text{cap}} = 2$, so a digest is the first two lanes of the output state. The 2-to-1 compression of two child digests $x = (x_0, x_1)$ and $y = (y_0, y_1) \in \mathbb{F}_p^2$ is $H_2(x, y) = (\pi(x_0, x_1, y_0, y_1))_{0,1}$: one application of the width-4 permutation π , truncated to the rate lanes. The SHAKE256 domain-separation seed and the MDS matrix are matched to the reference hasher, and cross-codebase tests pin the modulus, round count, MDS matrix, and seed against the reference implementation.

5.5 Soundness of the Precompile Suite, and Two Open Questions

This subsection states the soundness claim and the two questions it leaves open. The per-constraint audit is deferred to Appendix B.

Theorem 5.1 (Precompile soundness, conditional). *This is a functional (input–output) equivalence statement, and, conditional on premise (T3), a transfer of PLUM’s EU-CMA reduction; it is not a standalone cryptographic-security claim. Assume (T1) the host zkVM’s cross-table lookup argument is sound; (T2) the Griffin AIR’s constraints are satisfied only by a witnessed transition equal to the reference Griffin- π permutation; (T3) PLUM’s EU-CMA reduction treats Griffin and field multiplication as black-box input–output maps, inspecting no intermediate representation (such as a limb of a field product); and the upstream modular-multiplication guarantee of §5.2. Then substituting the precompile suite for the emulated operations preserves the input–output value of every PLUM and BDEC verification predicate. Consequently, under (T3), PLUM’s EU-CMA reduction transfers unchanged: that reduction is in the random-oracle model and depends only on the input–output behaviour of the hash function instantiating the oracle, which, granting (T2), the precompile reproduces exactly: under that premise it computes the same function the emulated code did, so no random-oracle programming step in the reduction is affected. (The reduction’s reliance on (Q)ROM-replaceability of this Griffin instantiation is a separate, upstream assumption, recorded as an open upstream assumption in Section 6 and not discharged here.)*

What is established. The field-substrate half rests on the reduction of §5.2. The suite is platform-specific: on SP1 it includes the bespoke Griffin AIR, to which premise (T2) applies; on RISC Zero the Griffin arithmetic is carried by `sys_bigint` emulation, so (T2) is not invoked there and is replaced by the analogous input–output equality of the emulated permutation. For the Griffin AIR, the per-family analysis (Appendix B) exhibits, family by family, the constraint *intended* to catch any deviation from reference Griffin, the forgery that deviation would enable, and the implementation status of each; memory binding, output canonicity, parameter immutability, and row scheduling are implemented and constrained. What that analysis does *not* establish, and says so explicitly, is that the

implemented constraint system is satisfied *only* by reference-equal witnesses: this is premise (T2), the *chip–constraint equality*. It is empirically supported by the 256-vector cross-codebase agreement below (per-vector false-pass probability $\approx 2^{-199}$), but that agreement pins the *executor* against the reference, a weaker *chip–executor* equality, not the *constraints*; (T2) is not verified at the constraint level.

Open question 1: assumption (T2) is empirically supported but unproved. The 256-vector agreement establishes that the executor’s native compute equals the reference permutation; it does *not* by itself establish that the *AIR constraints* admit only that computation (that the chip witness equals the executor). Closing this requires deriving the AIR witness from the same reference code and checking constraint satisfaction directly; we report (T2) as empirically supported and constraint-level-unverified, rather than as proved.

Open question 2: the QROM status of this Griffin instantiation is upstream of the AIR. PLUM’s EU-CMA argument under Fiat–Shamir requires Griffin to be replaceable by a (quantum) random oracle. The Griffin analyses cover the original instantiations; whether they cover this $d = 3$, width-4, 199-bit-field, 14-round instantiation is not something the AIR can settle. If this instantiation were QROM-distinguishable, no amount of AIR engineering would recover security. We flag this as a thesis-level assumption that still has to be checked against the Griffin literature; we do not close it here.

Further limitations not hidden. The RISC Zero `sys_bigint` soundness is cited from the upstream release process rather than re-derived; the EU-CMA reduction is argued to transfer on the grounds that the reference reduction treats Griffin and field multiplication as black-box input–output maps, but we have not audited PLUM’s reduction line by line for a step that inspects an *intermediate* representation (such as a limb of a field product), which input–output equality alone would not cover; and the zero-knowledge property required by BDEC’s anonymity guarantee is a property of the proof wrap, analysed as the security–deployability tension of Section 6 and measured in Section 7, not closed here.

5.6 Integration into the BDEC Credential-Generation Relation

The credential-generation relation $R_{\text{cre}}^{\text{PLUM}}$ of Section 3 is the AND concatenation of two PLUM signature verifications, both under the user’s public key pk_U , with public statement $x_{\text{cre}} = (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$ and witness $w_{\text{cre}} = (\text{pk}_U, \text{psk}_{U,TA})$. We implement this relation *in full* as a RISC Zero guest, with PLUM in place of Loquat and field multiplications routed through `sys_bigint`; the guest computes exactly the two $\text{Pred}_{\text{Verify}}$ conjuncts that define base CreGen, with no omitted predicate. (The optional Merkle-accumulator variant of Remark 3.7 is not part of this relation and performs its membership checks in `BDEC.ShowVer`; it is out of scope here.) One *statement-binding* requirement remains open at the receipt level, and it bears on security, so we do not treat it as a cosmetic completeness gap: the prototype commits the verification outcome to the journal but not yet the public statement x_{cre} itself, so a receipt currently attests “two signatures verify under some witness” without binding the proof to the specific public $(c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$. Until x_{cre} is committed to the journal, the artifact serves as

a *functional benchmark* and does not yet act as a statement-bound credential verifier: an accepting receipt does not certify *which* statement was proved. Committing x_{cre} is a small, cost-negligible change (a few field elements) and is noted as immediate next work (Section 9); it does not alter the proved relation. In execute mode at $\lambda = 80$ the guest runs both verifications to acceptance in 9.4×10^9 guest cycles, the two costing essentially the same ($\approx 4.69 \times 10^9$ each), confirming that both pass through the same precompile-backed path. The transition from this execute-mode validation to a completed proof is the subject of Section 7. The full presentation relation (ShowCre), the $k + 2$ -verification generalisation of this relation, is likewise implemented as a RISC Zero guest and validated in execute mode at $k = 1, 2$ (Section 7); its zero-knowledge proof, like CreGen’s, has not been produced on the target hardware (Section 7).

6 Security Analysis

Section 5 described an instrument: a precompile suite that moves PLUM’s dominant field operations out of emulated guest code, together with a RISC Zero and SP1 realisation of the BDEC credential-generation relation. A performance instrument is meaningful for a credential system only if it does not silently weaken the security the system provides. This section makes that requirement precise. We fix the security model and threat model in Section 6.1, prove in Section 6.2 a preservation theorem that bounds how far the move to a zkVM substrate and the precompile suite can shift the adversary’s advantage in each of BDEC’s security experiments, and then isolate in Section 6.3 the one property the analysis does not deliver on consumer hardware, namely zero-knowledge, and hence anonymity, of the proof actually produced. This last point follows from how the toolchain produces proofs and not from any weakness in the construction itself, so we treat it as a finding about the substrate.

6.1 Security Model and Threat Model

BDEC is proved secure in three experiments, formalised in Section 3. Unforgeability requires that an adversary with issuance and showing oracles cannot produce an accepting showing for a statement outside its honestly issued set, beyond negligible advantage. Anonymity and unlinkability are challenge-bit experiments in which the adversary cannot distinguish, beyond advantage $|\Pr[b' = b] - \frac{1}{2}|$, which of two honestly issued credentials produced a challenge showing (anonymity) or which of two pseudonym keys issued a single challenge credential (unlinkability). We write $\text{Adv}_{\text{AC}}^{\text{unf}}$, $\text{Adv}_{\text{AC}}^{\text{anon}}$, and $\text{Adv}_{\text{AC}}^{\text{unlink}}$ for the respective advantages, all defined in Section 3.

Definition 6.1 (zkVM-BDEC advantage). For $\bullet \in \{\text{unf}, \text{anon}, \text{unlink}\}$, write $\text{Adv}_{\text{zkVM-BDEC}}^{\bullet}(\mathcal{A})$ for the advantage $\text{Adv}_{\text{AC}}^{\bullet}(\mathcal{A})$ of Section 3 evaluated on the BDEC scheme instantiated with PLUM as the signature scheme Sig and the zkVM as the zkSNARK Π .

BDEC’s Theorems 1 through 3 [1] already prove these properties, so this section does not re-prove them. It bounds how far the move to a zkVM substrate shifts each advantage.

The parties are the teaching authorities and the issuer, who hold signing keys, together with the holder and the relying party. The ledger roots $\text{rt}_{\text{cred},e}$, $\text{rt}_{\text{TA},e}$, and $\text{rt}_{\text{LR},e}$ are public and authenticated. The prover is the holder generating a showing, and the verifier is the relying party, which chooses the presentation predicate ϕ at presentation time. The adversary is the standard anonymous-credential adversary of Section 3, lifted to the zkVM execution setting: it may request issuance of credentials on attribute vectors of its choice, observe showings produced by honest holders, act as a malicious verifier choosing ϕ adaptively, and act as a malicious prover attempting to produce an accepting showing it should not be able to prove. Following PLUM, the unforgeability we inherit is established against classical adversaries in the classical random-oracle model. Its post-quantum reading rests on the underlying power-residue PRF and collision-resistance assumptions being quantum-hard, whereas the lift of the EU-CMA reduction itself to the quantum random-oracle model is a separate question that PLUM does not settle and that we do not re-establish here. That such a lift is *attainable* for this PRF family is shown by Pegasus [42], which proves (Q)ROM security for a power-residue-PRF signature; Pegasus is a different construction (a commit-and-open Σ -protocol, not PLUM’s STIR-based design), so it does not close PLUM’s own lift but it removes any presumption that the PRF family itself is the obstacle.

Side-channel and timing attacks on the prover host, denial of service, and the physical security of signing keys are out of scope, since they are orthogonal to the substrate question this thesis studies.

The trusted computing base of the zkVM regime is the RISC-V execution circuit, the precompile chips, and the lookup argument that binds them to the main execution trace.

The preservation theorem of Section 6.2 and the precompile-soundness theorem of Section 5 rest on four named premises, each also stated inline where it is used. Premise (T1), that the cross-table lookup argument binds the precompile chips to the execution trace, is assumed from the SP1 and RISC Zero implementations (their logarithmic-derivative lookup arguments [43]), not independently verified here. Premise (T2), that the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation, equivalently that ϵ_{AIR} is negligible, is open: it is argued and empirically supported in Appendix B but not established at the constraint level. Premise (T3), that PLUM’s EU-CMA reduction treats Griffin and field multiplication as black-box input–output maps, is argued from the structure of that reduction. Premise (T4), that PLUM admits a signature-transcript simulator, equivalently that ϵ_{sZK} is negligible, is open and is required only by the anonymity and unlinkability bounds, PLUM’s analysis establishing EU-CMA security but not simulatability. The zkVM is additionally assumed knowledge-sound and the modular-multiplication precompile sound, the latter established for SP1 (§5.2) and cited for RISC Zero; the quantum-random-oracle lift of PLUM’s EU-CMA reduction and of the nested Fiat–Shamir transform is open, the analysis being in the classical random-oracle model.

6.2 Security Preservation

The analysis rests on one structural observation. The zkVM regime changes how the BDEC relation $R_{\text{show}}^{\text{Sig}}$ (and the structurally analogous credential-generation relation $R_{\text{cre}}^{\text{Sig}}$) is proved; the relation itself, what it states, is unchanged. The relation is fixed in Section 3, and both the static-circuit and the zkVM regimes are proof systems for that same relation. Preservation therefore reduces to two claims: that the substrate computes the same relation, and that its proof system supplies the soundness and zero-knowledge that BDEC’s reductions consume. We treat the two in turn.

Lemma 6.2 (Relation preservation). *Suppose the host modular-multiplication precompile is sound, the cross-table lookup argument binds the precompile tables to the main execution trace, the zkVM is a knowledge-sound argument of knowledge for RISC-V execution, and the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation. Then the zkVM guest program with the precompile suite active accepts a pair (x, w) if and only if $R^{\text{Sig}}(x; w) = 1$, where R^{Sig} denotes either of the BDEC relations $R_{\text{cre}}^{\text{Sig}}$, $R_{\text{show}}^{\text{Sig}}$ of Section 3. The lemma is stated at the level of the intended guest specification; the code-level claim additionally requires that the zkVM journal be bound to the public statement x , which the current RISC Zero prototype does not yet implement (Section 5). Absent that binding, an accepting receipt does not certify which statement was evaluated, so the code-level reading of the lemma is conditional on the journal fix.*

Proof sketch. A guest evaluation of the BDEC relations invokes only large-field multiplications and Griffin permutations, which the precompiles return identically under the lookup binding, together with native control flow covered by the zkVM’s knowledge-soundness; composing these per-operation equalities along the finite, input-fixed execution yields equality of the accept-or-reject decision at the program level. The Griffin step’s constraint-level soundness rests on the AIR argument of Appendix B, so the equality holds modulo premise (T2); the 9.4×10^9 -cycle execute-mode run confirms the guest program’s I/O but does not exercise the AIR constraints. The step-by-step argument, including the $k + 2$ presentation predicates, is given in Appendix A. $\square$

Theorem 6.3 (Security preservation, conditional; classical ROM). *Suppose PLUM is EU-CMA secure in the classical random-oracle model and, for the anonymity and unlinkability bounds only, additionally admits a signature-transcript simulator with error ϵ_{sZK} (premise (T4), an open premise parallel to (T2): PLUM’s analysis establishes only EU-CMA, whereas BDEC’s Theorems 2–3 simulate the credential and pseudonym transcripts with the signature’s own simulator). Suppose the zkVM is a knowledge-sound argument of knowledge for RISC-V execution, the host modular-multiplication precompile is sound, and the cross-table lookup argument binds the precompile tables to the trace. Suppose further, as premise (T3), that PLUM’s EU-CMA reduction treats Griffin and field multiplication as black-box input–output maps, so that the precompile substitution leaves that reduction unchanged. Suppose further, as an unverified premise (T2), that the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation; this premise is argued in Appendix B but not established at the constraint level (see the discussion after the proof). Then for every classical adversary $\mathcal{A}$ against the PLUM-instantiated, zkVM-executed BDEC system, in the random-oracle model, the advantage is bounded by five leaf errors: ϵ_{EUF} (PLUM’s EU-CMA error), ϵ_{KS} (the zkVM argument’s knowledge-soundness error), ϵ_{AIR} (the Griffin-AIR constraint-level soundness error), ϵ_{ZK} (the produced proof’s zero-knowledge error), and ϵ_{sZK} (PLUM’s signature-transcript-simulation error):*

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{unf}}(\mathcal{A}) \leq \epsilon_{\text{EUF}} + \epsilon_{\text{KS}} + \epsilon_{\text{AIR}}, \quad (1)$$

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{anon}}(\mathcal{A}) \leq \epsilon_{\text{ZK}} + \epsilon_{\text{sZK}}, \quad (2)$$

and the same bound $\epsilon_{\text{ZK}} + \epsilon_{\text{sZK}}$ holds for unlinkability, whose game (BDEC Theorem 3) forwards a single challenge credential, so the one-transcript simulation applies once. These three bounds re-instantiate BDEC’s Theorems 1–3 [1] with PLUM as the signature scheme and the zkVM as the zkSNARK, expressed directly in the leaf errors below. The term ϵ_{AIR} is the AIR’s constraint-level soundness error: the probability, over the prover-chosen trace and the argument’s verifier randomness, that the argument accepts a trace whose Griffin transition differs from the reference permutation. Premise (T2) is the open, empirically-supported claim (Appendix B) that ϵ_{AIR} is negligible. ϵ_{KS} is the knowledge-soundness error of the zkVM argument, ϵ_{EUF} is the EU-CMA error of PLUM ([5, Theorem 1], carried over from Loquat’s reduction [3]), ϵ_{ZK} is the zero-knowledge error of the produced proof, and ϵ_{sZK} is the signature-transcript simulation error of PLUM (premise (T4)), required because BDEC’s anonymity and unlinkability reductions simulate the credential and pseudonym transcripts with the signature’s own simulator (Theorems 2–3 [1]), not only the proof simulator.

Proof sketch. By Lemma 6.2 the zkVM proves the same relation as BDEC’s static circuit, so we re-instantiate BDEC’s three reductions with PLUM as the signature scheme and the zkVM as the zkSNARK; all bounds are for classical $\mathcal{A}$ in the random-oracle model. For *unforgeability*, the zkVM knowledge extractor recovers a witness from an accepting showing (cost ϵ_{KS}); a malformed Griffin trace that passes the precompile costs ϵ_{AIR} (premise (T2)); the recovered signature is then a PLUM EU-CMA forgery (cost ϵ_{EUF}), giving (1). For *anonymity* and *unlinkability*, a simulator replaces the credential and pseudonym transcripts by PLUM’s signature-transcript simulator (cost ϵ_{sZK} , premise (T4)) and the proof by the zkVM’s zero-knowledge (cost ϵ_{ZK}); no witness is extracted, so ϵ_{KS} and ϵ_{AIR} do not enter, giving (2) and, for the single-credential unlinkability game, the same bound. Because BDEC proves its three theorems for a generic (signature, zkSNARK) pair [1], substituting PLUM and the zkVM changes nothing beyond these leaf errors. These guarantees are inherited in principle; Section 6.3 shows that the produced receipt mode has no established zero-knowledge simulator, and that

the ZK-wrapped mode is not produced within the resource and memory bound of the target hardware, so anonymity and unlinkability are not delivered there. The step-by-step reductions, with the explicit adversary constructions and hybrids, are given in Appendix A. $\square$

The hypotheses of Theorem 6.3 are not all equally settled, and the honest reading of the theorem turns on which are established and which remain open. PLUM’s EU-CMA security is taken from its specification and the Loquat analysis it builds on [3], which establish it in the classical random-oracle model against classical adversaries; the lift of that reduction to the quantum random-oracle model is open, so the unforgeability bound holds against classical adversaries and its quantum case inherits that open lift. The soundness of the modular-multiplication precompile is established for SP1 by the modular-multiplication reduction of §5.2 and is cited for RISC Zero from its production release, while the knowledge-soundness hypothesis is obtained from the STARK and BCS analyses underlying the two systems [25]; the cross-table lookup-argument binding (premise (T1)) is instead assumed from the SP1 and RISC Zero implementations of their logarithmic-derivative lookup arguments, whose published soundness analyses we do not re-establish here. These last are themselves obtained through the Fiat–Shamir compilation [44], whose soundness is a random-oracle property invoked twice and nested, PLUM inside the zkVM, and whose quantum case is again open.

The one hypothesis below the level of an established or cited result is premise T2, the Griffin AIR’s constraint-level soundness. The chip’s compute matches the reference permutation on the 256-vector equivalence suite, with per-vector false-pass at most $1/p \approx 2^{-199}$ and aggregate at most 2^{-191} . This figure is the test *coverage* of a positive-direction equivalence suite, run on valid inputs, and is not a soundness bound: it does not show that the constraint system *rejects* a malformed Griffin trace, which is the direction T2 actually asserts. The equality of the constraint system with the reference is argued rather than machine-checked in Appendix B. Two further points require disclosure. The implementation fixes a concrete 199-bit prime $p = 2^{64}p_0 + 1$, so any security-bit claim that depends on the exact value of p is conditional on this instantiation matching PLUM’s intended parameters. And whether Griffin at the instantiation used here, a degree-3 S-box over a width-4 state of the 199-bit field with 14 rounds, may be replaced by a random oracle in PLUM’s EU-CMA argument is a question the AIR cannot settle and that we carry as open. The reduction of Theorem 6.3 is therefore sound *modulo its hypotheses*, but four of its inputs are not yet closed: the constraint-level AIR equality (T2), the black-box-transfer premise (T3, argued from the reduction’s structure but not audited line by line, §5), PLUM’s signature-transcript simulatability (T4), and the quantum lifts of PLUM and of Fiat–Shamir. We state the theorem conditionally on them rather than assert guarantees the cited proofs do not establish. The conservative reading is explicit: absent a PLUM transcript simulator (T4), only the EU-CMA-derived *unforgeability* of the credential transfers, and anonymity and unlinkability are conditional on (T4); absent the quantum lift, every guarantee holds against classical adversaries only. For each open premise we therefore state which guarantee survives it, and we make no unconditional claim.

Premise T2 is not firewalled from the BDEC credential-generation prove anomaly reported in Section 7: that run was rejected by the zkVM’s own internal verifier on a trace of the same relation. The constraint-level behaviour of the chip on this workload is exactly what T2 leaves unsettled. The genuinely dangerous case for T2, silent *acceptance* of an invalid Griffin trace by the AIR, has not been tested at all; the equivalence suite exercises only valid inputs. Resolving T2 properly, by metamorphic and fault-injection testing of the AIR, or by constraint-level verification of the chip against the reference permutation (tractable because SP1’s prover is built on Plonky3), is left as future work; the

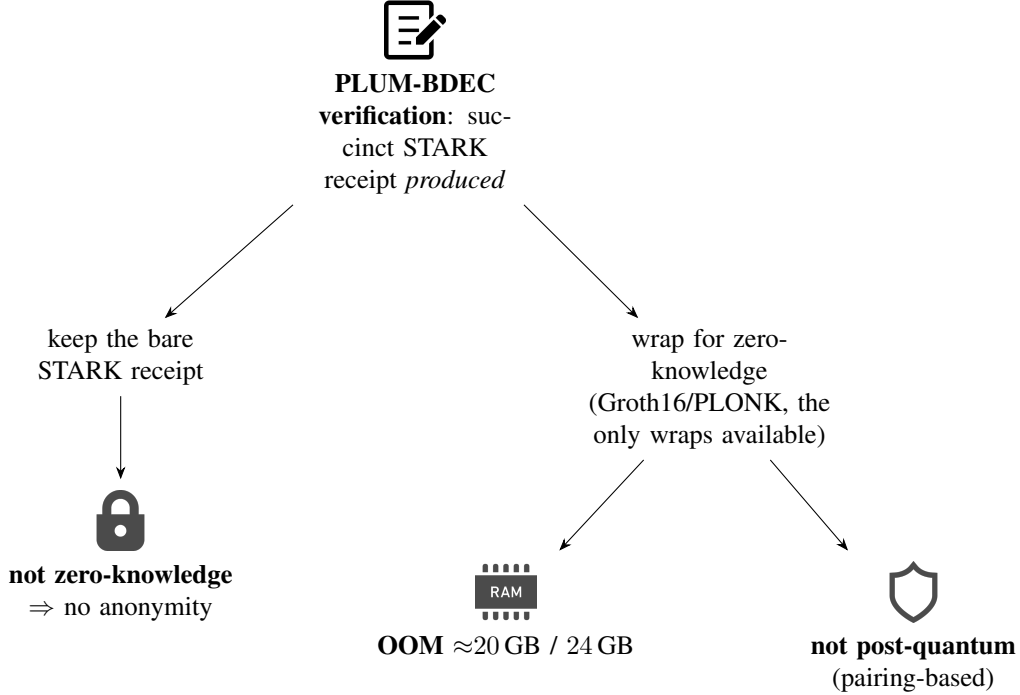


Figure 3: The dual obstruction. The proof feasible to generate (left, the bare STARK receipt) is not zero-knowledge, hence not anonymous; the proof that would be zero-knowledge (right, a Groth16/PLONK wrap) is blocked by two *independent* prongs: resource (OOM at ≈ 20 GB) and primitive (pairing-based, not post-quantum, which more memory cannot fix). Solid edges mark a produced step, dashed a blocked one.

metamorphic and fault-injection approach is exactly that of Arguzz [45], which targets zkVM soundness and completeness bugs by that technique.

6.3 The Zero-Knowledge Barrier as a Finding

BDEC’s anonymity and unlinkability require the underlying argument to be zero-knowledge. The succinct STARK receipt produced by either zkVM is, in the configurations we run, an argument of knowledge but not a zero-knowledge one, and this holds on both substrates for separately documented reasons. SP1’s specification states plainly that its STARK proofs are not zero-knowledge [46]. RISC Zero documents perfect zero-knowlgedgeness for its STARK, but independent analysis by Habock and Al Kindi, together with RISC Zero’s own security advisory, finds that several STARK implementations, RISC Zero among them, do not provably meet the zero-knowledge requirement [47, 48]. We therefore treat the bare receipt as not dependably zero-knowledge on either substrate, a concrete instance of the caution that a system may be called a zkVM without delivering zero-knowledge. Zero-knowledge is obtained only by wrapping the receipt in a succinct proof, and the wraps SP1 exposes, Groth16 and PLONK, are pairing-based and therefore only classically secure. The barrier splits into two independent obstructions, set out below. Both block anonymity for separate reasons, and naming each one precisely is the security-level result of this analysis.

The first obstruction is one of resources. On the target hardware at $\lambda = 80$ the PLONK wrap ex-

hausts memory at ≈ 20 GB on the 24 GB machine (killed under memory pressure; Section 7), so the anonymity-providing proof is infeasible to generate even though the bare, non-anonymous receipt is. The second obstruction is one of primitives. Even with unlimited memory, wrapping a post-quantum STARK receipt in a pairing-based Groth16 or PLONK proof re-introduces a quantum-vulnerable assumption into the anonymity guarantee, the very class of assumption the post-quantum motivation rules out (Section 2). Such a wrap remains acceptable for a classically-secure deployment, and Groth16 and PLONK are not unsuitable in general. They are ruled out here only by the thesis’s post-quantum-anonymity goal. Recovering post-quantum anonymity requires a post-quantum-sound zero-knowledge wrap, for instance a STARK-based or lattice-based recursive argument, which the deployed toolchains do not yet provide.

We therefore use the classically secure Groth16 or PLONK wrap to locate the resource cost. It is not a deployable post-quantum-anonymous configuration. The two obstructions together leave no good option on commodity hardware: the proof that is feasible to generate is not anonymous, and the proof that would be anonymous is neither feasible to generate nor, with the available wraps, post-quantum. Within the scope of the literature we surveyed, we did not find this conjunction stated explicitly for post-quantum anonymous credentials, and surfacing it is a contribution of this thesis: we quantify the resource prong, the 24 GB wrap frontier of Section 7, and identify the primitive prong as a toolchain gap.

The barrier is scoped to the substrate and hardware studied: an Apple M5 Pro with 24 GB of memory, the CPU-bound provers of RISC Zero and SP1, and the parameter set $\lambda = 80$. We do not claim the barrier is fundamental to the primitive or that it persists on server-class hardware. Characterising how it moves with the memory budget and the security level is left to Section 9.

7 Evaluation

This section reports what the precompile-enabled zkVM regime can and cannot do for PLUM-based BDEC on consumer hardware, and uses the measurements to revisit the cost framework of Section 4. Consistent with the rest of the thesis, the contribution is the *quantification*: within the scope of the literature we surveyed, these are among the first reported figures for executing and proving a PLUM-instantiated post-quantum anonymous-credential workload inside a general-purpose zkVM, and we report them honestly, including the configurations that do not complete.

7.1 Measurement Methodology

The evaluation addresses four questions: whether PLUM verification, and the BDEC credential relation built on it, run inside the zkVM on the target hardware, with and without the precompile suite; where the zkVM cost falls and how much of it the precompile suite removes; for the configurations that do not complete, what the binding resource is and where the boundary lies; and how the two regimes compare on the deployment-lifecycle dimension of Definition 4.3.

Two measurement modes, and when each is reportable. A zkVM runs in two modes relevant here. *Execute mode* runs the guest and counts RISC-V *cycles* without producing a proof; its cost is machine-independent and terminates within the guest’s memory space. *Prove mode* produces the cryptographic receipt and is the deployment-relevant cost, but is orders of magnitude heavier and, as reported below, does not always complete on the target hardware. Our rule is therefore: *report prove wall-clock where proving completes, and fall back to execute-mode cycle counts (machine-independent and reproducible) where it does not*. Cycle counts and wall-clock times are never compared across this boundary as if commensurable; each table states its mode.

What we deliberately do not claim. We do not report a single “ $N \times$ speedup” headline. As §7.3 makes explicit, the with/without-precompile contrast at the prove level is an *infeasible-to-feasible* transition rather than a ratio of two finite times, and the finite reductions we do report are cycle-level reductions at a stated security level and mode. Conflating these into one multiplier would produce the kind of setting-dependent figure we avoid.

7.2 Experimental Setup

7.2.1 Settings Justification

A measurement is only persuasive if its settings are ones a deployer would plausibly choose, rather than ones selected to flatter a number. We state and justify each fixed parameter, and disclose where a setting is tuned.

Consumer hardware (M5 Pro, 24 GB). The research question concerns deployability on a personal machine, so this hardware is the thesis’s explicit scope. Every feasibility claim is scoped to this envelope and is not asserted in general.

Component	Specification
Hardware	Apple M5 Pro, 18-core CPU, 20-core GPU, 24 GB unified memory, 1 TB SSD
OS	macOS 26.5 (Darwin 25.5.0), Apple Silicon (arm64)
Prover	CPU-bound; no CUDA/network prover (Apple Silicon has no NVIDIA GPU path)
zkVMs	SP1 v6.2.1 (forked), RISC Zero v3.0.x
Signature	PLUM over $\mathbb{F}_p$, $p = 2^{64}p_0 + 1$ (199-bit; see §5.2)
Security levels	$\lambda = 80$ (prove-mode runs), $\lambda = 128$ (execute-mode cycle attribution)

Table 1: Evaluation platform. All prove-mode measurements are local single-machine runs with no network or dev-mode acceleration unless stated.

Security level: $\lambda = 80$ **for prove-mode**, $\lambda = 128$ **for cycle attribution**. We disclose this seam rather than hide it. Prove-mode runs use $\lambda = 80$ because, as §7.6 reports, $\lambda = 128$ proving exceeds the memory budget; $\lambda = 80$ is the largest level at which prove-mode wall-clock is observable on this hardware, which is why we use it and not because it is favourable. The cost-attribution analysis (§7.5) is reported at the audience-relevant $\lambda = 128$ because it is machine-independent (execute mode) and so remains measurable where proving is not. No number mixes the two levels silently; each table states λ .

Prover memory tuning. The base prove-mode runs ran at SP1’s *default* settings and completed within the 24 GB envelope: Cell 2 in 32.53 min and the SHA-3 control in 13.28 min (`shard_size` and `rayon_threads` at default; §7.2.2). Memory tuning enters only for the *zero-knowledge wrap* attempts, where even default settings OOM-kill within minutes; there we applied a documented reduced configuration (halved then quartered shard size, fewer rayon threads) in an attempt to fit, and the wrap still exhausted memory (§7.6.2). The tuning therefore bears on the wrap’s infeasibility. It has no bearing on the headline timing: no reduced-shard setting was used to obtain the 32.53-minute figure.

Micro-benchmark isolation. Cost-attribution runs isolate single primitives (one Griffin permutation; one verification) so that per-operation cost can be attributed rather than inferred from an aggregate.

7.2.2 Reproducibility

Prove-mode and attribution runs use fixed RNG seeds and fixed messages; the SP1/RISC Zero submodule versions are pinned in the lockfile; the memory-tuning environment variables, host binaries, and the adversarial-probe harness are recorded in the project’s measurement notes, together with the per-run commit hashes and, where applicable, guest-image identifiers.

7.3 PLUM Verification With and Without the Precompile

Table 2 reports prove-mode results for a single PLUM signature verification at $\lambda = 80$. The central comparison is between the precompile-free baseline and the precompile-enabled arm on the same hardware.

zkVM / mechanism	λ	Prove time	Verify time	Peak mem	Outcome
SP1, rv32im Griffin (no precompile)	80	n/a	n/a	87.94%	DNF (OOM $\approx 1\text{m}45\text{s}$)
SP1, Griffin- $\mathbb{F}_p^{192}$ precompile	80	32.53 min	3.06 s	81.31%	receipt verified
SP1, SHA-3 hasher (control)	$80^\dagger$	13.28 min	2.80 s	n/a	receipt verified
RISC Zero, <code>sys_bigint</code>	80	6 h 19 m	n/a ‡	n/a	receipt verified

Table 2: PLUM single-signature verification, prove mode, $\lambda = 80$, M5 Pro 24 GB. The precompile-free baseline does not complete; the precompile arm completes in 32.53 min. The SHA-3 arm is a *control* (what a standard-hash precompile alone buys), not a deployable PLUM variant. Single runs. † SHA-3 control at $\lambda = 80$. ‡ RISC Zero receipt-verify wall-clock not separately logged; the receipt verified.

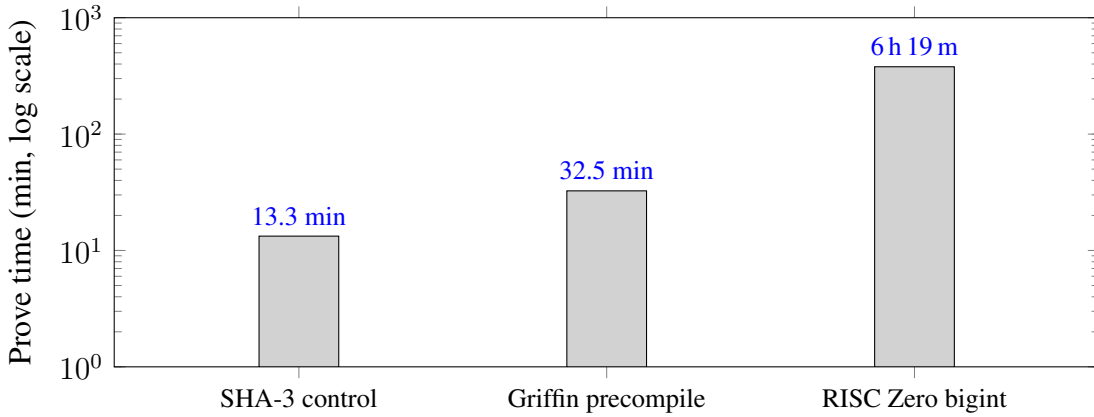


Figure 4: Prove time on the configurations that complete ($\lambda = 80$, M5 Pro 24 GB, log scale). The bespoke Griffin precompile (32.5 min) is slower than the SHA-3 control (13.3 min) at matched security, and RISC Zero’s `sys_bigint` route is slower again (6 h 19 m); these are different hash and substrate mechanisms, so the bars are not one artefact. The precompile-free baseline does not complete (DNF, OOM at $\approx 1\text{m}45\text{s}$) and so has no bar.

The honest reading. The with/without-precompile contrast is not a ratio of two finite times: without the precompile the workload *does not complete* on this hardware, and with it the workload completes in 32.53 minutes. The defensible claim is therefore qualitative at the prove level (*the precompile moves PLUM verification from infeasible to feasible on consumer hardware*) and quantitative only at the cycle level (§7.5). The two zkVMs are not directly comparable here: SP1 runs the bespoke Griffin AIR, whereas RISC Zero carries the field arithmetic through `sys_bigint` without a native Griffin chip (§5.3); the large SP1-vs-RISC Zero gap reflects that mechanism difference and should not be read as a like-for-like substrate ranking. The SHA-3 control isolates the hash family: the verification harness is identical except that every Griffin- $\mathbb{F}_p^{192}$ sponge call is replaced by a SHA3-256 call computed in software, so that the SHA-3 arm fires *no* hash precompile and only the shared `UINT256_MUL` field-multiplication precompile remains active. It shows that a workload using a *standard* hash in software is markedly cheaper than one using the algebraic hash PLUM requires through a bespoke chip, which is the quantitative form of the arithmetic-mismatch argument of Section 4. The inversion is direct: the bespoke Griffin- $\mathbb{F}_p^{192}$ precompile (32.53 min) is about $2.45\times$ *slower* at prove than the software SHA-3

control (13.28 min) at matched $\lambda = 80$. We read this as a genuine property of the pipeline rather than a measurement artefact: in the deployed SP1 pipeline, arithmetising a 199-bit-field algebraic hash through a custom AIR costs more than a SHA-class hash the substrate already accelerates natively. What the precompile bought us was a finite measurement; it did not buy a speedup.

Proof size, and why we do not re-measure it. Receipt size in SP1 is a property of the proof *type*, not of the workload. The runs above use SP1’s default *core* proof, whose size *scales with execution size* [49]; the compact, on-chain-deployable proof is instead the constant-size SNARK *wrap* of that receipt, about 260 bytes for Groth16 over BN254, or 868 bytes for PLONK [49]. Producing that wrap is exactly the step that exhausts memory on the target hardware (the PLONK wrap was killed at ≈ 20 GB, §7.6.2). On a 24 GB machine, then, the proof a deployer can actually generate is the large, shard-scaling core receipt; the 260-byte SNARK is a documented size the consumer-hardware frontier prevents us from realising here. We therefore report the deployable figure as documented and do not serialise the workload-dependent core receipt, whose byte count is not the deployment-relevant number.

7.4 The Circuit-SNARK Reference Point

To situate the zkVM costs against the static-circuit regime they are meant to escape, we measured the Aurora prover end-to-end over the BDEC R1CS on the same hardware. This is the baseline the BDEC construction itself uses [1]. We measure it directly rather than cite it, so that the comparison is on our hardware rather than across machines.

Quantity	Value
Constraints (padded)	524,288
Prover time (setup + prove)	227.1 s (3.78 min)
Verifier time	41.2 s
Proof size	688 KB
Achieved security	107.5 bits (target 128)
Zero-knowledge	disabled

Table 3: Aurora over the BDEC R1CS, M5 Pro 24 GB, single run. This is the *Loquat*-instantiated circuit over $\mathbb{F}_{p^2}$ (127-bit base) [1], *not* the PLUM workload measured elsewhere; the two are not a same-scheme comparison. Prover time includes Aurora parameter setup but is not a recompile-time measurement (§7.8). Achieved security 107.5 bits, below the 128 target. Verification ACCEPT.

What this reference does and does not license. Two readings of this point are legitimate and one is not. *Legitimate*: on this hardware the static-circuit regime proves a BDEC credential statement in minutes, against the zkVM’s tens-of-minutes-to-infeasible (Tables 2, 5), so for a *fixed* predicate the static circuit is the faster prover, as one would expect from a specialised encoding. *Also legitimate*: this Aurora instance, like the zkVM succinct receipt, is non-zero-knowledge here, so it does not by itself discharge BDEC’s anonymity requirement either. *Not licensed*: reading Table 3 as a like-for-like “Aurora beats the zkVM” verdict on the same scheme, since it is Loquat over a 127-bit field, whereas

the zkVM numbers are PLUM over a 199-bit field, and it is timed without the predicate-change cost that is the whole point of the update-churn comparison (§7.8).

7.5 Cost Attribution

To attribute the cost we use execute-mode cycle counts at $\lambda = 128$, the audience-relevant security level, where prove-mode is infeasible but cycle counts remain machine-independent. Table 4 reports the cumulative effect of routing PLUM’s field arithmetic through the host precompile, as an ablation over the SHA-3-hashers verification path.

Configuration (cumulative)	SP1 cycles	RISC Zero cycles	Δ vs. baseline
Baseline (BigUint emulation)	289,778,709	812,646,400	n/a
+ field mul via host precompile	276,643,994	n/a	SP1 −4.5%
+ power as square-and-multiply	240,857,140	n/a	SP1 −16.9%
+ skip redundant reduction	179,952,920	239,599,616	SP1 −37.9%, R0 −70.5%

Table 4: Cycle-level cost attribution, PLUM verification, SHA-3 hasher, $\lambda = 128$, *execute mode*. Routing field arithmetic through the host precompile (and removing redundant reductions) cuts 37.9% of SP1 cycles and 70.5% of RISC Zero cycles; RISC Zero’s larger reduction reflects its heavier per-multiplication baseline (6,625 vs. 1,826 cycles/mul).

Where the cost lives, and the parameter-set caveat. At $\lambda = 128$ a single PLUM verification issues about 112,902 $\mathbb{F}_p^{192}$ multiplication syscalls through the host precompile, as counted in our own SP1 run log; this figure counts *all* field multiplications, the large majority of which occur *inside* the Griffin permutations rather than in the explicit field-arithmetic of the verifier (PLUM §4.2 reports only $\approx 10^3$ explicit verification multiplications). The count is therefore consistent with the arithmetic-mismatch model: the cost is dominated by the algebraic hash, whose internal field operations are routed through the precompile. We caution explicitly against transferring the PLUM paper’s circuit-SNARK figure (that 91% of *RISC constraints* are hash work) directly to the zkVM cost model: that figure is a constraint count in a different proving system, whereas Table 4 is a RISC-V cycle count. The qualitative conclusion (hash/field arithmetic dominates) agrees; the exact share differs by cost model and is reported here in the zkVM’s own units.

Execute-mode delta for the Griffin precompile (PLUM-80). For the bespoke Griffin AIR specifically, the precompile-free versus precompile-enabled execute-mode cycle counts differ by a factor of $56.4\times$ at $\lambda = 80$, the ratio of precompile-free to Griffin-AIR-enabled cycles on the Griffin arm; this is distinct from the 37.9%/70.5% of Table 4, a separate field-multiplication-reduction ablation reported at $\lambda = 128$. We report this as an *execute-mode cycle ratio* rather than a wall-clock speedup: it is the machine-independent counterpart of the infeasible-to-feasible prove transition of Table 2, and it is the figure that should be cited when a single number is wanted, in place of any prove-time “speedup,” which is undefined when the baseline does not complete.

7.6 PLUM-in-BDEC and the Consumer-Hardware Frontier

Moving from a single signature to the BDEC credential-generation relation $R_{\text{cre}}^{\text{PLUM}}$ (two signature verifications under a hidden user key; §5.6), Table 5 reports the application-level result on RISC Zero at $\lambda = 80$.

Workload	Mode	Cost	Outcome
BDEC CreGen (PLUM, $\lambda = 80$)	execute	9.40×10^9 cyc, ≈ 65 s	both verifications accept
BDEC CreGen (PLUM, $\lambda = 80$)	prove (succinct)	≥ 15.6 h, peak RSS 11.24 GB	internal-verifier rejection (anomaly; §7.6.1)

Table 5: BDEC credential generation, RISC Zero, $\lambda = 80$, M5 Pro 24 GB. Execute mode validates functional correctness (9.4×10^9 cycles, $\approx 4.69 \times 10^9$ per signature verification). The succinct-prove run yielded no receipt; the cause was an internal-verifier rejection, not a resource limit (§7.6.1), and is not used as a feasibility result.

The execute-mode result establishes that the BDEC credential relation is functionally correct end-to-end inside the zkVM: both signature verifications accept, at 9.4×10^9 cycles. The prove-mode outcome requires care, because it is *not* the resource frontier it might first appear to be, and we separate the two readings explicitly.

7.6.1 An internal-verifier rejection rather than a resource frontier

The succinct-prove run (RISC Zero 3.0.3, default `ProverOpts::succinct()`) terminated after 15.6 hours with RISC Zero’s own recursive aggregation step rejecting one of its intermediate segment proofs (`verify segment / verification indicates proof is invalid`). This is categorically different from running out of memory: peak resident memory was only 11.24 GB on the 24 GB machine, with kernel free memory above 70% throughout, so the machine was *not* the limiting resource. An internal-verifier rejection is a correctness anomaly; it tells us nothing about deployability, and we therefore do *not* report it as a frontier or fold it into any feasibility claim. Its candidate causes are (i) an edge case in RISC Zero’s recursion/segment-composition at a segment count (9.4×10^9 guest cycles, $\approx 65 \times$ the single-verification workload) beyond what public benchmarks exercise; (ii) a non-ECC memory error over the long run; or (iii) a precompile trace malformation that manifests only across a segment boundary, which would bear on the chip–constraint equality (premise (T2)) of Section 6. The segment count, segment size, and failing-segment index were not captured in this run; a re-run with segment-level diagnostics (`RUST_LOG=risc0_zkvm=debug`), intended to record those and to distinguish these causes, is the appropriate next step; pending it, the anomaly is recorded as exactly that (an anomaly under diagnosis), and the thesis’s claims do not rest on it. The single PLUM verification *does* prove successfully on RISC Zero (Table 2 reports the SP1 analogue; the RISC Zero single-verify prove completes and verifies). That is consistent with cause (i): the anomaly appears at the larger segment count, and the verification logic itself proves clean.

7.6.2 The genuine resource frontier

The deployability frontier of this thesis rests on *resource* failures, which are unambiguous and independent of the anomaly above. Two are measured. First, the PLONK zero-knowledge wrap of the standalone PLUM single-signature-verification receipt (Cell 2 of Table 2, the 32.53-min prove, not CreGen or ShowCre) was killed under memory pressure at $\approx 82\%$ of the 24 GB budget (≈ 20 GB of system memory pressure at the kill; peak RSS ≈ 17.8 GB); this is the cost specific to the anonymity-providing wrap, and it is the measured form of the anonymity–deployability tension of Section 6. Second, and separately, a *non-zero-knowledge* baseline proof of a standardised-lattice signature verification (an ML-DSA proxy: the 17 NTT-domain polynomial multiplications of 256 coefficients modulo $q = 8,380,417$ that dominate ML-DSA-65 verification, proved as a baseline succinct STARK on SP1, not a zero-knowledge wrap) reached a peak footprint of 82.7 GB; this is a substrate base-cost datum that *strengthens but does not instantiate* the tension, since it bears on the cost of proving any PQ verification on this substrate, not on the zero-knowledge wrap specifically. These are the data points the frontier claim uses; the positive feasibility anchor remains the PLUM single-verification prove of Table 2, which completes in 32.53 min. Table 6 summarises, for every proof mode attempted, whether a receipt is produced and whether it is zero-knowledge and post-quantum.

Proof mode (substrate / configuration)	Produced?	Zero-knowledge?	Post-quantum?	Resource outcome
SP1, PLUM verify, Griffin precompile (core)	yes	no	yes	completes, 32.5 min
SP1, PLUM verify, SHA-3 control (core)	yes	no	yes	completes, 13.3 min
SP1, PLUM verify, Groth16/PLONK wrap	no	yes	no	OOM ≈ 20 GB
RISC Zero, PLUM verify, sys_bigint (succinct)	yes	no	yes	completes, 6 h 19 m
RISC Zero, BDEC CreGen (succinct)	no	n/a	n/a	≥ 15.6 h, internal-verifier rejection
Aurora, BDEC static (Lo-quint)	yes	no [†]	yes	completes, 3.78 min

Table 6: Proof-mode feasibility matrix. The modes *produced* on the target hardware are not zero-knowledge; the only zero-knowledge mode the toolchain exposes (row 3, a pairing wrap) is neither post-quantum nor produced within the memory bound, which is the dual obstruction. [†]Aurora is zero-knowledge-capable but the measured run had ZK disabled and ran below $\lambda = 128$. The SHA-3 row is a control, not a deployable PLUM variant.

7.7 A Cost Model and the Substrate Crossover

The measurements above are point observations; to answer “what would this cost at a configuration we did not run, and when is the flexible substrate the rational choice,” we fit a cost model to them

and validate it against the points we have. This is the analysis that converts the measurements from a benchmark table into a basis for a deployment decision. The model is fitted and validated in *execute-mode* cycle counts; carrying it to prove-mode wall-clock assumes the additive form survives recursion, memory pressure, segment aggregation, and the zero-knowledge wrap, any of which can introduce nonlinearity, so the prove-mode reading is an extrapolation, not a measured fit.

7.7.1 An additive cost model for the BDEC relations

Both BDEC relations are, at the cycle level, dominated by repeated signature verifications (Section 4.4): CreGen contains two, and ShowCre contains $k+2$ for k shown credentials. Writing $C_{\text{ver}}(\lambda, h)$ for the execute-mode cycle cost of one PLUM verification at security level λ under hasher h , and C_0 for the fixed per-run overhead (input parsing and witnessing; the ledger-membership, revocation, and disclosure checks are performed by the relying party outside the proof), the model is

$$C_{\text{CreGen}}(\lambda, h) \approx 2 C_{\text{ver}}(\lambda, h) + C_0, \quad (3)$$

$$C_{\text{ShowCre}}(k, \lambda, h) \approx (k + 2) C_{\text{ver}}(\lambda, h) + C_0, \quad (4)$$

The model is deliberately linear in the verification count, because the cost decomposition of Section 4.4 shows that everything else in the proof is dominated by the $\Theta(k)$ algebraic-hash work inside the verifications; the ledger-membership, revocation, and disclosure checks do not enter the proving cost, as the relying party performs them outside the proof.

7.7.2 Validation against the measured points

The model is falsifiable against the two BDEC points we measured. The strongest internal check is CreGen: Equation (3) predicts the two verifications should each cost $\approx C_{\text{ver}}$ and together dominate the trace. The measurement bears this out directly: the two verifications cost 4.69×10^9 cycles *each* (the two within 0.01% of *each other*), their sum 9.38×10^9 accounting for $\approx 99.8\%$ of the 9.40×10^9 total (Table 5); so for the Griffin hasher at $\lambda = 80$, $C_{\text{ver}}(80, \text{Griffin}) \approx 4.69 \times 10^9$ cycles and C_0 is small. We caution that this single relation validates the *linearity* and the *verification-dominance*, not the absolute constant at other (λ, h) ; the $k = 14$ projection below is labelled as such.

The hasher gap measures the tax. A single PLUM verification is markedly cheaper with the SHA-3 hasher control than with the Griffin algebraic hash, an order-of-magnitude separation that is the field-mismatch tax of Section 4 quantified at the verification level. We state the gap only as indicative: the two figures we have to hand sit at different security levels (the Griffin 4.69×10^9 at $\lambda = 80$, the SHA-3 control in the $\lambda = 128$ ablation of Table 4), and our own non-comparability rule forbids dividing across that seam to produce a single ratio. The qualitative conclusion (that the algebraic hash is the tax-bearing component while the field arithmetic is not, so C_{ver} must be reported per hasher) holds regardless of the exact multiplier. The precompile narrows this gap (Table 4); closing it entirely is what a native Griffin AIR on the proving path would buy.

ShowCre execute-mode cost (measured at $k = 1, 2$). The ShowCre relation is realised by the guest `bdec_showcre_plum_griffin`, the $k + 2$ generalisation of the CreGen guest, and measured in execute mode at the two running examples. At $k = 1$ (the ϕ_1 workload) ShowCre executes in

1.41×10^{10} cycles, and at $k = 2$ (the ϕ_2 workload) in 1.88×10^{10} cycles, each with all $k+2$ verifications accepting. Both agree with Equation (4) at $C_{\text{ver}}(\mathbf{80}, \text{Griffin}) \approx 4.69 \times 10^9$: the per-verification cost is constant at $\approx 4.69 \times 10^9$ cycles across CreGen and both ShowCre instances, so the additive $(k+2) C_{\text{ver}}$ form is measured rather than assumed. Applying the same measured constant to a stress-test disclosure-set size, well beyond the running examples ($k = 1, 2$) and the $k \leq 4$ design target, projects $\approx 7.5 \times 10^{10}$ cycles at $k = 14$. A direct $k = 14$ execute run does not reach this projection: the 16 accumulated verifications exhaust the guest’s default non-reclaiming bump allocator before the cycle count completes, an in-guest memory limit distinct from the prover-side memory frontier and clearable only by the reclaiming allocator at additional cycle cost. The $k = 1$ and $k = 2$ executions are correct and their cost is measured; the zero-knowledge *proof*, however, rests on the same anonymity-providing wrap whose memory exhaustion is the consumer-hardware frontier of §7.6.2 (the ≈ 20 GB PLONK wrap; 82.7 GB for the standardised-lattice baseline). A zero-knowledge proof of ShowCre at any $k \geq 1$ is therefore bounded by that wrap frontier, not by the CreGen prove anomaly (an internal-verifier rejection, not a resource limit; §7.6.1). More memory would clear the resource bound, but a post-quantum-anonymous proof additionally requires a post-quantum-sound wrap (Section 6).

7.7.3 The substrate crossover

The decision a deployer faces (Section 8) is not about which substrate is faster per proof, since the static circuit wins that in any case. It is about which substrate is cheaper over a deployment *lifetime* during which the predicate changes. Let the lifetime contain P proof generations and U localised predicate changes. Amortised total cost is, schematically,

$$\text{static: } U \cdot R_{\text{static}} + P \cdot t_{\text{static}}^P, \quad (5)$$

$$\text{zkVM: } U \cdot 0 + P \cdot t_{\text{zkVM}}^P, \quad (6)$$

where R_{static} is the per-change recompilation cost and t^P is per-proof time. We isolate R_{static} directly (§7.8): for the full BDEC R1CS it is R1CS construction (≈ 0.2 s) plus Aurora parameter setup (measured at $\approx 3 \mu\text{s}$, because Aurora is non-preprocessing), so $R_{\text{static}} \approx 0.3$ s. The multi-minute figure of Table 3 is the per-*proof* prover, not the per-change recompile, and does *not* enter R_{static} . The zkVM’s update term is zero (Table 7). The static regime is cheaper only while

$$U \cdot R_{\text{static}} < P (t_{\text{zkVM}}^P - t_{\text{static}}^P),$$

i.e. while the relation-change rate U/P stays below a crossover

$$\left(\frac{U}{P}\right)^* = \frac{t_{\text{zkVM}}^P - t_{\text{static}}^P}{R_{\text{static}}}.$$

The measured $R_{\text{static}} \approx 0.3$ s is decisive, and it cuts *against* a naive flexibility argument. Because R_{static} is sub-second while the zkVM’s per-proof penalty exceeds the static prover’s ($t_{\text{zkVM}}^P > t_{\text{static}}^P$ even granting the cross-scheme seam of §7.4: static Loquat- $\mathbb{F}_{p^{127}}$, zkVM PLUM- $\mathbb{F}_p$), the crossover $(U/P)^*$ of Eq. 7.7.3 is on the order of thousands of relation changes per proof, a regime no realistic deployment reaches. **Against a transparent, non-preprocessing argument such as Aurora, the zkVM thus has no wall-clock flexibility advantage:** both regimes rebuild a sub-second artefact per change, and the zkVM is then dearer per proof. Recompile wall-clock does not capture the genuine

per-change cost of the static regime; that cost is the *re-audit and engineering* footprint of a new relation-specific circuit (Table 7, $N_{\text{circ}}/N_{\text{param}}$). A *wall-clock* flexibility advantage would appear only against a *preprocessing* argument (Groth16 [50], Marlin [30], or circuit-specific PLONK [51]), whose per-circuit key generation makes R_{static} large, and Aurora, the BDEC baseline, is not such a system. We therefore report the flexibility result as *qualitative* (the regeneration footprint of Table 7) together with this measured wall-clock caveat, rather than as a numeric crossover that favours the zkVM.

7.8 Update Churn

Table 7 enumerates the regeneration footprint of Definition 4.3 for the three localised changes of §4.3, using $\phi_1 \rightarrow \phi_2$ (par. 1) as the presentation-change instance. Here $\phi_1 \rightarrow \phi_2$ is a presentation-*structure* change: it moves the credential count from $k=1$ to $k=2$, altering the relation $R_{\text{show}}^{\text{Sig}}$ and hence the static R1CS. A presentation *policy* change at fixed structure (e.g. retuning a threshold the verifier checks on the disclosed attributes) is handled verifier-side and triggers no regeneration on either substrate, so the rows below measure structural changes.

Localised change	Static circuit (Aurora)		zkVM	
	N_{circ}	N_{param}	N_{circ}	N_{param}
Presentation change ($\phi_1 \rightarrow \phi_2, k:1 \rightarrow 2$)	≥ 1	≥ 1	0	0
Signature swap (Loquat $\rightarrow$ PLUM)	≥ 1	≥ 1	0	0
Security retune ($\lambda=80 \rightarrow 128$)	≥ 1	≥ 1	0	0

Table 7: Circuit/parameter regeneration footprint ($N_{\text{circ}}, N_{\text{param}}$ of Definition 4.3). The static-circuit regime regenerates and re-audits its R1CS and constraint-dependent parameters on every localised change; the zkVM absorbs all changes as guest-program edits, leaving the universal relation unchanged ($N_{\text{circ}} = N_{\text{param}} = 0$). The components $N_{\text{src}}, N_{\text{audit}}$ are not integer-countable and are discussed in prose.

Structural separation, and the measured recompile cost. The structural separation in Table 7 is established by construction: a zkVM’s relation is fixed to the ISA, so a presentation-structure or signature change is absorbed as a source edit ($N_{\text{circ}} = N_{\text{param}} = 0$) with no recompilation. *Both* components of the static side are now measured, and the result overturns the earlier conjecture that recompilation is expensive. R1CS *construction* from the BDEC relation is sub-second (0.18 s at Loquat- $\lambda=80$, 0.24 s at $\lambda=128$, three runs each). The Aurora *parameter setup* that a relation change forces to recompute is, measured directly, about $3 \mu\text{s}$ (the runner reports $t_{\text{setup}} = 0.003 \text{ ms}$ at the full 2^{19} -padded BDEC R1CS), because Aurora is *non-preprocessing*: it has no per-circuit indexer. The prover’s low-degree-encoding FFTs (its dominant step) run *per proof*, not per relation change, and so do not enter R_{static} . Hence $R_{\text{static}} \approx 0.3 \text{ s}$ for the whole BDEC relation. A $\phi_1 \rightarrow \phi_2$ change alters only the credential count and does not enlarge the microsecond parameter setup, so this bound also holds for the per-presentation delta; a k -parameterised emitter would refine the R1CS-construction term but cannot move the dominant conclusion, namely that for a transparent non-preprocessing argument the per-change recompile is sub-second. Consequently the zkVM’s wall-clock flexibility advantage over Aurora is negligible, and the static regime’s real per-change cost is the *re-audit* of a new circuit (Table 7) rather than recompile time.

The advantage is conditional on the baseline’s preprocessing model. The negligible recompile cost above is a property of Aurora’s *non-preprocessing* structure; it does not generalise to static circuits at large. A *preprocessing* (holographic) transparent argument such as Fractal [29] must instead build a per-circuit index, and that index is exactly what a relation change forces to regenerate. Timing Fractal’s indexer (its `fractal_snark_indexer` stage) in isolation on the same Loquat-BDEC R1CS over the 127-bit field (the same harness as §7.4; ≈ 107 -bit achieved, zero-knowledge disabled; a Loquat proxy for PLUM) gives the contrast of Table 8: Aurora’s parameter setup is microseconds and flat in circuit size, whereas Fractal’s holographic indexer grows superlinearly and, at the 2^{19} -padded BDEC size, is killed under memory pressure mid-indexer (during a 2^{27} -element FFT) before returning.

R1CS size (constraints)	Aurora R_{static} (param. setup)	Fractal R_{static} (indexer)
2^{10} (1,024)	0.0021 ms	0.86 s
2^{12} (4,096)	0.0023 ms	3.74 s
2^{14} (16,384)	0.0025 ms	18.5 s
2^{19} (BDEC, 524,288)	0.0029 ms	OOM-killed at ≈ 33 min

Table 8: Per-change recompile cost R_{static} for a non-preprocessing (Aurora) versus a preprocessing (Fractal) transparent argument, same Loquat-BDEC R1CS (127-bit field), M5 Pro 24 GB. Aurora has no per-circuit indexer (R_{static} flat at $\sim \mu\text{s}$); Fractal’s indexer is superlinear and exceeds 24 GB at the BDEC size (OOM mid-indexer). The zkVM regenerates no circuit.

The flexibility reading is therefore *conditional on the static baseline’s preprocessing model*: the zkVM’s wall-clock recompile advantage is negligible against a non-preprocessing argument (Aurora) and large against a preprocessing one (Fractal), where at the BDEC circuit size the baseline’s per-circuit indexing is itself infeasible on the target hardware. The same-scheme and field seam of §7.4 applies throughout (a Loquat proxy, not PLUM), and a smaller folding parameter than the Aurora-matched $\text{RS} = 5$ might let the Fractal indexer fit and yield a finite but still large R_{static} .

7.9 What the Measurements Establish

We close by reading the measurements back against the cost framework of Section 4.

Feasibility and the field-mismatch tax. Without the precompile a single PLUM verification does not complete on the target hardware (out-of-memory at $\approx 1\text{m}45\text{s}$, Table 2); with it the verification completes in 32.53 min. At the cycle level the bespoke Griffin precompile removes a $56.4\times$ factor at $\lambda = 80$ (§7.5) and 37.9%/70.5% of SP1/RISC Zero cycles at $\lambda = 128$ (Table 4). This is the field-mismatch tax of Definition 4.1 made concrete: the precompile turns an infeasible workload into a feasible one, and the residual cost is what the rest of the evaluation measures. The with/without contrast at the prove level is an infeasible-to-feasible transition rather than a finite ratio (§7.3); the factors above are cycle-level and single-verification, and we do not promote them to a CreGen-relation prove speedup we did not run.

Where the cost lives. The cost attribution of §7.5 bears out the decomposition of §4.4: the dominant term is the algebraic hash, whose internal field operations account for the large majority of the $\mathbb{F}_p^{192}$

multiplications, not the explicit verifier field arithmetic. We report this in the zkVM’s own cycle units and caution against transferring PLUM’s circuit-SNARK “91% of constraints are hash” figure to the cycle model.

Per-proof cost against the static circuit. For a *fixed* predicate the specialised static circuit is the faster prover, as expected from a specialised encoding: Aurora proves the BDEC circuit in 3.78 min (Table 3) against 32.53 min for precompiled PLUM verification (Table 2). The prove times do *not* converge; but the two points are not the same scheme (Loquat over $\mathbb{F}_{p^2}$ versus PLUM over its 199-bit prime field, §7.4), so we state the gap as indicative, and the comparison the thesis turns on is the deployment-lifecycle one below rather than the per-proof one. Closing the per-proof gap requires a same-scheme measurement: either running Loquat verification inside the zkVM to match the static Loquat/Aurora point, or building an $\mathbb{F}_p$ (199-bit) Aurora harness to run PLUM statically (the current Aurora harness is $\mathbb{F}_{p_{127}}$ -only); we identify this as the required follow-up experiment.

Update churn. The structural separation of Definition 4.3 holds by construction (Table 7): every localised change regenerates at least one circuit and one parameter set on the static side and none on the zkVM side, provided the change does not alter a precompiled primitive or its field; a change to the hash family or field width regenerates the Griffin AIR and so incurs $N_{\text{circ}} \geq 1$ on the zkVM side too (Definition 4.3). The recompile wall-clock of the static side is now measured ($R_{\text{static}} \approx 0.3$ s; §7.8) and is sub-second because Aurora is non-preprocessing; the gap that remains is a *same-scheme* per-proof static prover time (§7.4) rather than the recompile, our static point being Loquat and our zkVM point PLUM.

The zero-knowledge frontier. The base proof completes; what binds on consumer hardware is the zero-knowledge wrap. A PLONK wrap of a PLUM proof exhausts memory (the cost specific to the anonymity-providing wrap), which is the measured form of the anonymity–deployability tension of Section 6; a *non-zero-knowledge* standardised-lattice baseline separately reaches 82.7 GB peak (§7.6.2), a substrate base-cost that strengthens but does not instantiate that tension. The frontier claim rests on this resource-failure evidence, *not* on the CreGen prove anomaly of §7.6.1, which is an internal-verifier rejection.

7.10 Threats to Validity

- **Single-run timings.** The prove-mode wall-clock figures (Tables 2, 3) are single runs; we report no variance, and thermal or scheduling noise on a laptop-class machine over multi-minute runs is not characterised.
- **Thermal throttling.** Runs are on a laptop-class M5 Pro; sustained multi-minute proving may throttle, which would inflate wall-clock relative to a thermally-unconstrained host.
- **Hardware/OS-string provenance.** The reported platform string is from `sysctl`; we have not cross-checked every sub-component against an independent inventory.
- **Incomplete statement binding.** The RISC Zero prototype does not yet bind the receipt journal to the public statement (c, h, ppk) (Section 5); until it does, an accepting receipt is a functional

benchmark and not a statement-bound credential proof, which is a security gap and not a timing one.

- **Unresolved CreGen prove anomaly.** The 15.6-hour BDEC credential-generation prove run ended in an internal-verifier rejection (§7.6.1), not a resource limit; it is a single run with undetermined cause, no claim of this thesis rests on it, and a segment-diagnostic re-run is required before any statement about it. It is explicitly *not* part of the resource frontier (§7.6.2).
- **Parameter-set seam.** Prove-mode wall-clock is at $\lambda = 80$; cycle attribution is at $\lambda = 128$. The two are never combined into one figure, but a reader must track which is which; the tables are labelled accordingly.
- **Non-uniform precompile.** SP1 uses the bespoke Griffin AIR; RISC Zero uses `sys_bigint`. Cross-zkVM numbers reflect this mechanism difference and are not a like-for-like substrate comparison.
- **Constraint-level soundness of the Griffin AIR is empirically supported, not proved** (the chip–constraint equality, premise (T2), of Section 6); the cost figures assume the chip computes the reference permutation, tested on 256 random vectors but not verified at the constraint level.
- **Control vs. deployable.** The SHA-3 arm is a diagnostic control; it is not a secure PLUM variant and its timing is not a deployment claim.

7.11 Summary

On consumer hardware, the precompile suite moves PLUM verification from infeasible to feasible (Table 2), with the cost dominated by large-field/algebraic-hash arithmetic and reduced by 37.9–70.5% at the cycle level (Table 4); the full BDEC credential relation is functionally correct in execute mode, while its succinct prove ended in an internal-verifier rejection categorically distinct from the resource frontier (Table 5, §7.6.1), the binding consumer-hardware frontier being the zero-knowledge wrap (§7.6.2); and the zkVM regime is structurally free of the update churn that the static-circuit regime incurs on every localised change (Table 7). The one comparison the data does not yet settle is a *same-scheme* per-proof prover time across the two substrates; the static-circuit recompile wall-clock is now measured ($R_{\text{static}} \approx 0.3$ s) and is not the bottleneck. The next chapter takes up what these results mean for the deployment decision and which questions they leave open.

8 Discussion

The measurements of Section 7 and the security analysis of Section 6 together answer the question this thesis set out to ask. That question concerns whether a general-purpose zkVM is a viable substrate for post-quantum anonymous credentials, and at what cost, rather than how fast the zkVM runs in isolation. This section interprets the answer as guidance a deployer can act on.

8.1 A Substrate-Selection Rule

The two regimes do not dominate each other; they trade different costs, and the right choice depends on a single property of the deployment: how often the verification predicate changes relative to how often proofs are generated.

- **When the predicate is fixed at deployment time** and proofs are generated frequently, the static-circuit regime is preferable: its specialised R1CS amortises across many showings, and it pays its update cost only once. The zkVM’s per-proof overhead (Section 7) is not justified when there is nothing to update.
- **When the predicate is chosen at presentation time, or the signature or security parameters evolve over the deployment’s lifetime**, the static-circuit regime pays its update-churn cost (Table 7) *repeatedly*, and the zkVM regime, which absorbs each such change as a guest-program edit, becomes the structurally appropriate substrate, at the proving cost quantified in Section 7. For a non-preprocessing static argument such as Aurora this update-churn cost is a regeneration-and-re-audit (governance and assurance) burden rather than a large wall-clock penalty, which the next paragraph makes precise.

This rule is made precise by the crossover of §7.7.3: the predicate-change rate $(U/P)^*$ above which the zkVM is the cheaper substrate over a deployment lifetime is the ratio of the zkVM’s per-proof penalty to the static circuit’s per-change recompilation cost. The recompilation cost is now measured (Section 7): for a transparent non-preprocessing argument such as Aurora it is $R_{\text{static}} \approx 0.3$ s (R1CS construction plus a microsecond parameter setup), and it sits outside the multi-minute prover, which is paid per proof. At that magnitude, and with the zkVM dearer per proof, the wall-clock crossover favours the static circuit in any realistic churn regime: against Aurora the zkVM has no wall-clock flexibility advantage. The flexibility claim we can defend is the *qualitative* regeneration-and-re-audit footprint of Table 7 (zero on the zkVM side, ≥ 1 on the static side per change), which is an engineering-and-assurance cost paid by humans, not prover time. A wall-clock advantage would require a *preprocessing* baseline such as Groth16, Marlin, or circuit-specific PLONK; the transparent Aurora that BDEC uses does not offer one. This zero-regeneration property is partitioned rather than absolute. A change confined to the verification predicate is a guest-program edit that reuses the existing universal relation, but a change to the hash family or to the field width still regenerates and re-audits the bespoke Griffin-Fp192 precompile, so the decision rule applies to predicate churn and not to substrate-level changes of the signature itself. What the cost model licenses is limited: it fixes the *form* of the rule and the *sign* of every term, but not the numeric location of the crossover, which requires a same-scheme static prover time we do not yet have (§7.7.3). So the rule gives directional guidance, telling a deployer which side a high- or low-churn deployment lies on without yet handing them a numeric threshold to evaluate.

The contribution of this thesis to that rule is the *quantification of both sides*: the zkVM’s per-proof cost is now measured (Tables 2–5), and the structural form of the static circuit’s update cost is enumerated (Table 7). The static circuit’s recompile *time* is now measured ($R_{\text{static}} \approx 0.3$ s, §7.8) and is sub-second because Aurora is non-preprocessing; the term the rule still lacks is therefore a *same-scheme* per-proof prover time, our static point being Loquat and our zkVM point PLUM. Until that is available, the crossover is bounded in form but not located in wall-clock. We conjecture, but have not measured, that a same-scheme circuit-SNARK prover for PLUM over its 199-bit field would remain below the zkVM’s per-proof cost. Both the larger field and the substitution of PLUM for Loquat are uncontrolled across our measured point pair (static Loquat- $\mathbb{F}_{p^2}$ versus zkVM PLUM- $\mathbb{F}_{p^{199}}$), so the sign of a same-scheme comparison is unverified, not only its numeric location. Confirming it requires the same-scheme measurement we lack.

8.2 The Security–Deployability Tension

The sharpest finding is the tension of Section 6, which outweighs any single timing figure. On the target hardware at $\lambda = 80$, the proof that is feasible to generate is not zero-knowledge, and the proof that is zero-knowledge, the one BDEC’s anonymity guarantee requires, is not feasible to generate within the memory budget. A deployer who needs anonymity on commodity hardware today cannot, on the evidence here and with the current toolchain versions, have it for this workload. More memory would clear the resource obstruction, but the only wraps the deployed pipeline exposes are pairing-based and therefore not post-quantum, so they would not restore the post-quantum anonymity that motivates the construction. Recovering it requires a post-quantum-sound zero-knowledge wrap, a STARK- or lattice-based recursive argument, which the toolchains do not yet provide. We locate this obstruction along two axes at once, resource and primitive, and it holds independently of how favourable our timing numbers are.

8.3 What Generalises and What Does Not

The arithmetic-mismatch mechanism (Section 4) and the update-churn dimension (Definition 4.3) are substrate-independent: they apply to any large-field algebraic-hash post-quantum signature of this form verified in any small-field zkVM, not only to PLUM in RISC Zero/SP1. The specific magnitudes (Section 7) do not generalise: they are tied to PLUM’s 199-bit field, the M5 Pro’s 24 GB envelope, and the CPU-bound provers of the two zkVMs studied. We drew the boundary deliberately, so that the shape of the cost carries to other pairings of a large-field signature with a small-field zkVM while the numbers stay pinned to the configuration measured here.

8.4 Comparison to Prior Evaluations

As anticipated in Section 2, the prior-evaluation record omits two axes: the cost of a *change* to the verification predicate, and the consumer-hardware proving frontier for an algebraic-hash post-quantum signature. We do not restate that gap here; instead we report what our measurements add to the record. For predicate-change cost, Table 7 fixes its structural form (zero regenerations on the zkVM side against ≥ 1 per change on the static side) and §7.8 bounds its wall-clock magnitude; for the frontier, §7.6.2 locates the binding constraint as the zero-knowledge wrap rather than the base proof. Where

our per-proof timings overlap the existing record, they agree in order of magnitude with the hitherto-qualitative observation that algebraic-hash workloads are expensive inside small-field zkVMs, now quantified at the verification level (Tables 2–4). The next section draws these threads together into the thesis’s overall conclusion.

9 Conclusion

This thesis asked whether a general-purpose zero-knowledge virtual machine (zkVM) is a viable substrate for post-quantum anonymous credentials on consumer hardware. The goal was to make BDEC credential verification, instantiated with the PLUM signature, generate proofs in a practically acceptable time inside a zkVM on a personal computer (Apple M5 Pro, 24 GB); the means was a custom precompile suite that absorbs the field-mismatch tax of PLUM’s large-prime-field algebraic hashing. We realised the construction on RISC Zero and SP1, measured it against an Aurora static-circuit baseline, and analysed what the substitution preserves and what it costs.

9.1 Main Findings

The central result is a frontier, and neither substrate is a clear winner. On the target hardware the BDEC proof one can afford to generate is not zero-knowledge, and the zero-knowledge wrap the toolchain exposes is not post-quantum (Section 6.3). The first prong is a measured resource bound: the PLONK wrap of the standalone PLUM verification receipt is killed under memory pressure at ≈ 20 GB of the 24 GB budget. The second is a fact about the deployed toolchain: the only succinct wraps it offers are pairing-based, so additional memory would not restore post-quantum anonymity. This obstruction on both sides holds independently of any timing number, and the rest of the thesis rests on it.

Around that finding, the measurements establish the surrounding cost picture. The custom Griffin- $\mathbb{F}_p^{192}$ precompile turns a non-terminating emulation into a finite measurement: without it, standalone PLUM verification exhausts memory at $\approx 1\text{m}45\text{s}$ on the target hardware; with it, the verification proves in 32.5 minutes on SP1 at $\lambda = 80$. Two of the measurements falsify natural predictions. First, the bespoke algebraic-hash precompile is *slower* than a software SHA-3 control at matched security (32.5 versus 13.3 minutes in prove wall-clock, about $2.45\times$): arithmetising a 199-bit-field algebraic hash through a custom chip costs more than a hash the substrate already supports. Second, against the transparent, non-preprocessing Aurora baseline the zkVM’s update-churn advantage never shows up in wall-clock. The static-side recompile is sub-second ($R_{\text{static}} \approx 0.3$ s, Section 7.8), so what separates the two regimes is the regeneration-and-re-audit *footprint* (Table 7), a cost in engineering and assurance rather than in proving time. A wall-clock flexibility advantage would appear only against a *preprocessing* baseline, which is not what BDEC uses.

The BDEC relations themselves run correctly inside the zkVM: credential generation executes both of its signature verifications to acceptance in 9.4×10^9 guest cycles, and the presentation relation in 1.41×10^{10} (one credential shown) and 1.88×10^{10} (two) cycles, consistent with the additive cost model. No zero-knowledge *proof* of either relation was produced on consumer hardware: the credential-generation succinct prove terminated in an internal-verifier rejection (RISC Zero 3.0.3; an anomaly under diagnosis, categorically distinct from a resource limit and on which no claim of this thesis rests, Section 7.6.1), and the presentation wrap is bounded by the same memory frontier.

9.2 Answer to the Research Question

The answer is *not yet*, for a reason more specific than “too slow.” Standalone PLUM verification becomes feasible on consumer hardware once the precompile absorbs the field-mismatch tax. Deployable post-quantum *anonymity* is not reached, because it requires a zero-knowledge wrap that is at once feasible to generate within 24 GB and post-quantum-sound, and the current toolchains provide neither.

The substrate choice is therefore governed by the deployment-lifecycle decision rule of Section 8: where the verification predicate is fixed and proofs are frequent, the specialised static circuit is preferable; where the predicate, signature, or security parameters evolve over the deployment’s lifetime, the zkVM’s fixed universal relation confines each change to a guest-program edit, provided the change leaves the precompiled primitive suite intact, since a hash-family or field-width change regenerates the Griffin AIR. Against a non-preprocessing baseline its advantage there is a smaller re-audit footprint, not a faster runtime.

9.3 Limitations

The results are bounded in ways we state explicitly. Prove-mode wall-clock is measured at $\lambda = 80$, the largest level at which it is observable within the memory budget and not a deployment security level; cycle attribution is reported at $\lambda = 128$. The static baseline is Loquat over $\mathbb{F}_{p_{127}}$ with zero-knowledge disabled, not same-scheme PLUM, so the per-proof comparison is indicative and the lifecycle crossover is bounded in form but not located in wall-clock. The prove-mode figures are single runs. The security guarantees are conditional: the constraint-level soundness of the Griffin AIR (premise (T2)), the existence of a PLUM signature-transcript simulator (premise (T4)), the quantum-random-oracle lifts, and the canonical prime instantiation are carried as open premises (Section 6). Finally, the RISC Zero prototype does not yet bind the receipt journal to the public statement, so the artifact is at present a functional benchmark rather than a statement-bound credential verifier.

9.4 Future Work

The most informative next measurement is a *same-scheme* per-proof comparison, either running Loquat verification inside the zkVM to match the static Loquat/Aurora point or building an $\mathbb{F}_p$ Aurora harness to run PLUM statically; this would locate the crossover that the decision rule currently bounds only in form. The open premises are the substantive cryptographic work that remains: premise (T2) by metamorphic and fault-injection testing of the AIR, or constraint-level verification of the chip against the reference permutation, by the method of Arguzz [45]; premise (T4) by constructing a PLUM signature-transcript simulator; and the post-quantum-anonymity goal by a post-quantum-sound zero-knowledge wrap (a STARK- or lattice-based recursive argument) in place of the pairing-based wraps the toolchains currently expose. The immediate engineering step is to commit the public statement to the receipt journal, after which the credential-generation prove anomaly should be re-run with segment-level diagnostics. Confirming the canonical prime with PLUM’s authors would discharge the prime-instantiation premise.

9.5 Closing

Future-proof post-quantum identity systems require both cryptographic soundness and operational evolvability, and this thesis measured the price of pursuing both on the same substrate. What it contributes is a mapped frontier. The precompile makes post-quantum signature verification finite to prove on a personal computer. The deployment-lifecycle separation between the static and zkVM regimes is real, though it surfaces as a re-audit footprint and stays out of the wall-clock numbers. The obstruction to deployable post-quantum anonymity has two prongs that survive together: the proof one can afford

is feasible but not anonymous, and the anonymous wrap is not post-quantum, and no amount of additional memory or favourable timing removes either. That obstruction, named precisely and surrounded by its cost picture, is what we hand to the design of the next such system.

Acknowledgement

References

- [1] Z. Z. Li, X. Zhang, H. Cui, J. Zhao, and X. Chen, “Bdec: Enhancing learning credibility via post-quantum digital credentials,” in *Provable and Practical Security: 18th International Conference, ProvSec 2024, Gold Coast, QLD, Australia, September 25–27, 2024, Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 45–64. [Online]. Available: https://doi.org/10.1007/978-981-96-0957-4_3
- [2] C. Jeudy, A. Roux-Langlois, and O. Sanders, “Lattice signature with efficient protocols, application to anonymous credentials,” in *Advances in Cryptology – CRYPTO 2023*, ser. Lecture Notes in Computer Science, vol. 14082. Springer, 2023, pp. 351–383, cryptology ePrint Archive, Paper 2022/509. [Online]. Available: <https://eprint.iacr.org/2022/509>
- [3] X. Zhang, R. Steinfeld, M. F. Esgin, J. K. Liu, D. Liu, and S. Ruj, “Loquat: A SNARK-friendly post-quantum signature based on the legendre PRF with applications in ring and aggregate signatures,” Cryptology ePrint Archive, Paper 2024/868, 2024. [Online]. Available: <https://eprint.iacr.org/2024/868>
- [4] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, “Aurora: Transparent succinct arguments for R1CS,” Cryptology ePrint Archive, Paper 2018/828, 2018. [Online]. Available: <https://eprint.iacr.org/2018/828>
- [5] X. Zhang, Q. Fu, R. Steinfeld, J. K. Liu, T. H. Yuen, and M. H. Au, “Plum: SNARK-friendly post-quantum signature based on power residue PRFs,” in *Provable and Practical Security: 19th International Conference, ProvSec 2025, Yokohama, Japan, October 10–12, 2025, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 111–129. [Online]. Available: https://doi.org/10.1007/978-981-95-2961-2_6
- [6] T. Dokchitser, and A. Bulkin, “Zero knowledge virtual machine step by step,” Cryptology ePrint Archive, Paper 2023/1032, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1032>
- [7] J. Bruestle, P. Gafni, and the RISC Zero Team, “RISC Zero zkVM: Scalable, Transparent Arguments of RISC-V Integrity,” RISC Zero, Draft, Aug. 2023, draft, August 11. [Online]. Available: <https://dev.risczero.com/proof-system-in-detail.pdf>
- [8] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari, “A survey on the applications of zero-knowledge proofs,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.00243>
- [9] Succinct Labs, “SP1 zkVM documentation,” <https://docs.succinct.xyz/docs/sp1/introduction>, 2024, accessed 2026-05-29.
- [10] L. Grassi, Y. Hao, C. Rechberger, M. Schofnegger, R. Walch, and Q. Wang, “Horst meets fluid-SPN: Griffin for zero-knowledge applications,” in *Advances in Cryptology – CRYPTO 2023, Part III*, ser. Lecture Notes in Computer Science, vol. 14083. Springer, 2023, pp. 573–606, full version: Cryptology ePrint Archive, Paper 2022/403, <https://eprint.iacr.org/2022/403>.

-
- [11] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 204, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.204.pdf>
- [12] —, “Stateless Hash-Based Digital Signature Standard,” U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 205, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.205.pdf>
- [13] —, “FIPS 206: FN-DSA, a FALCON-based digital signature standard (forthcoming),” U.S. Department of Commerce, NIST, Federal Information Processing Standards Publication (in preparation), 2025, forthcoming. As of the writing, no public draft had been released: the draft was submitted for internal/Department of Commerce clearance on 28 August 2025 and an Initial Public Draft (IPD) had not yet been published. Status presented at the Sixth NIST PQC Standardization Conference, 25 September 2025. [Online]. Available: <https://csrc.nist.gov/pubs/fips/206>
- [14] G.-V. Policharla, B. Westerbaan, A. Faz-Hernández, and C. A. Wood, “Post-quantum privacy pass via post-quantum anonymous credentials,” *Cryptology ePrint Archive*, Paper 2023/414, 2023, presented at Real World Crypto 2023 (non-archival). [Online]. Available: <https://eprint.iacr.org/2023/414>
- [15] M. Chase, D. Derler, S. Goldfeder, C. Orlandi, S. Ramacher, C. Rechberger, D. Slamanig, and G. Zaverucha, “Post-quantum zero-knowledge and signatures from symmetric-key primitives,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1825–1842, *cryptology ePrint Archive*, Report 2017/279. [Online]. Available: <https://eprint.iacr.org/2017/279>
- [16] C. Baum, C. D. de Saint Guilhem, D. Kales, E. Orsini, P. Scholl, and G. Zaverucha, “Banquet: Short and fast signatures from aes,” in *Public-Key Cryptography – PKC 2021*, ser. Lecture Notes in Computer Science, vol. 12710. Springer, 2021, pp. 266–297, *cryptology ePrint Archive*, Report 2021/068. [Online]. Available: <https://eprint.iacr.org/2021/068>
- [17] C. D. de Saint Guilhem, E. Orsini, and T. Tanguy, “Limbo: Efficient zero-knowledge mpcith-based arguments,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 3022–3036. [Online]. Available: <https://doi.org/10.1145/3460120.3484595>
- [18] T. Feneuil, and M. Rivain, “CAPSS: A framework for SNARK-friendly post-quantum signatures,” *Cryptology ePrint Archive*, Paper 2025/061, 2025. [Online]. Available: <https://eprint.iacr.org/2025/061>
- [19] D. Chaum, “Security without identification: Transaction systems to make big brother obsolete,” *Communications of the ACM*, vol. 28, no. 10, pp. 1030–1044, 1985. [Online]. Available: <https://doi.org/10.1145/4372.4373>
- [20] J. Camenisch, and A. Lysyanskaya, “Signature schemes and anonymous credentials from bilinear maps,” in *Advances in Cryptology – CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 56–72.

-
- [21] D. Pointcheval, and O. Sanders, “Short randomizable signatures,” in *Topics in Cryptology – CT-RSA 2016*, ser. Lecture Notes in Computer Science, vol. 9610. Springer, 2016, pp. 111–126, cryptology ePrint Archive, Paper 2015/525. [Online]. Available: <https://eprint.iacr.org/2015/525>
- [22] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2019. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/coconut-threshold-issuance-selective-disclosure-credentials-with-applications-to-distributed-ledgers/>
- [23] D. Boneh, X. Boyen, and H. Shacham, “Short group signatures,” in *Advances in Cryptology – CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 41–55, cryptology ePrint Archive, Paper 2004/174. [Online]. Available: <https://eprint.iacr.org/2004/174>
- [24] World Wide Web Consortium, “Verifiable credentials data model v2.0,” W3C, W3C Recommendation, May 2025, w3C Recommendation, 15 May 2025. [Online]. Available: <https://www.w3.org/TR/vc-data-model-2.0/>
- [25] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *Theory of Cryptography – TCC 2016-B, Part II*, ser. Lecture Notes in Computer Science, vol. 9986. Springer, 2016, pp. 31–60, cryptology ePrint Archive, Paper 2016/116. [Online]. Available: <https://eprint.iacr.org/2016/116>
- [26] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed–Solomon interactive oracle proofs of proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, 2018.
- [27] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, “STIR: Reed–solomon proximity testing with fewer queries,” Cryptology ePrint Archive, Paper 2024/390, 2024. [Online]. Available: <https://eprint.iacr.org/2024/390>
- [28] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, “Proximity gaps for reed–solomon codes,” Cryptology ePrint Archive, Paper 2020/654, 2020. [Online]. Available: <https://eprint.iacr.org/2020/654>
- [29] A. Chiesa, D. Ojha, and N. Spooner, “Fractal: Post-quantum and transparent recursive proofs from holography,” Cryptology ePrint Archive, Paper 2019/1076, 2019. [Online]. Available: <https://eprint.iacr.org/2019/1076>
- [30] A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. P. Ward, “Marlin: Preprocessing zkSNARKs with universal and updatable SRS,” in *Advances in Cryptology – EUROCRYPT 2020*, ser. Lecture Notes in Computer Science, vol. 12105. Springer, 2020, pp. 738–768, cryptology ePrint Archive, Paper 2019/1047. [Online]. Available: <https://eprint.iacr.org/2019/1047>
- [31] A. Arun, S. Setty, and J. Thaler, “Jolt: SNARKs for virtual machines via lookups,” Cryptology ePrint Archive, Paper 2023/1217, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1217>

-
- [32] N. Kaskov, M. Komarov, and M. Aksenov, “zkLLVM Circuit Compiler: A compiler from high-level programming languages into provable-computations protocol input representations,” =nil; Foundation, Technical Report, Apr. 2024, accessed April 6, 2024. [Online]. Available: https://cms.nil.foundation/uploads/zkllvm_a27f4af786.pdf
- [33] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A circuit description language for building zero-knowledge applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2023.
- [34] RISC Zero, “R0VM 2.0: A GPU-accelerated, formally-verified zkVM,” RISC Zero technical announcement, Apr. 2025, reports Ethereum block proving reduced from ≈ 35 min to ≈ 44 s; accessed 2026-05-31. [Online]. Available: <https://risczero.com/blog>
- [35] J. Bootle, V. Lyubashevsky, N. K. Nguyen, and A. Sorniotti, “A framework for practical anonymous credentials from lattices,” in *Advances in Cryptology – CRYPTO 2023*, ser. LNCS, vol. 14082. Springer, 2023, pp. 384–417, iACR ePrint 2023/560.
- [36] S. Argo, T. Güneysu, C. Jeudy, G. Land, A. Roux-Langlois, and O. Sanders, “Practical post-quantum signatures for privacy,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS ’24)*. ACM, 2024, pp. 1523–1537, full version: IACR ePrint 2024/131.
- [37] M. Rosenberg, J. White, C. Garman, and I. Miers, “zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure,” in *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2023, iACR ePrint 2022/878.
- [38] StarkWare, “S2morrow: STARK-based signature aggregation for Falcon and SPHINCS+,” Non-peer-reviewed engineering artifact, <https://github.com/starkware-bitcoin/s2morrow>, 2025, falcon-512 and SPHINCS+ verified in the Cairo zkVM.
- [39] Kota, “A zero-knowledge verifier for Dilithium signatures in the SP1 zkVM,” Non-peer-reviewed engineering artifact, <https://github.com/kota1026>, 2025, mL-DSA verified in SP1, Groth16-wrapped.
- [40] Y. Yang, Y. Cheng, H. Tang, G. Yang, B. Zhang, and K. Ren, “SoK: Understanding zkVM: From research to practice,” Cryptology ePrint Archive, Paper 2026/525; to appear, ACM AsiaCCS 2026, 2026. [Online]. Available: <https://eprint.iacr.org/2026/525>
- [41] L. Grassi, D. Khovratovich, and M. Schofnegger, “Poseidon2: A faster version of the poseidon hash function,” in *Progress in Cryptology – AFRICACRYPT 2023*, ser. Lecture Notes in Computer Science, vol. 14064. Springer Nature Switzerland, 2023, pp. 177–203, cryptology ePrint Archive, Paper 2023/323. [Online]. Available: <https://eprint.iacr.org/2023/323>
- [42] X. Zhang, Z. Li, R. Steinfeld, R. K. Zhao, J. K. Liu, and T. H. Yuen, “Pegasus and PegaRing: Efficient (ring) signatures from sigma-protocols for power residue PRFs with (Q)ROM security,” Cryptology ePrint Archive, Paper 2025/1841, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1841>
- [43] U. Haböck, “Multivariate lookups based on logarithmic derivatives,” Cryptology ePrint Archive, Paper 2022/1530, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1530>

-
- [44] A. Fiat, and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology – CRYPTO ’86*, ser. Lecture Notes in Computer Science, vol. 263. Springer, 1987, pp. 186–194.
 - [45] C. Hochrainer, V. Wüstholtz, and M. Christakis, “Arguzz: Testing zkVMs for soundness and completeness bugs,” arXiv preprint arXiv:2509.10819, 2025. [Online]. Available: <https://arxiv.org/abs/2509.10819>
 - [46] Succinct Labs, “Security model — SP1 documentation,” SP1 Documentation, 2025, accessed 2026-06-01. [Online]. Available: <https://docs.succinct.xyz/docs/sp1/security/security-model>
 - [47] RISC Zero, “RISC Zero zkVM: Notes on zero-knowledge,” Security advisory GHSA-5xgj-pmjj-gw49, 2024. [Online]. Available: <https://github.com/risc0/risc0/security/advisories/GHSA-5xgj-pmjj-gw49>
 - [48] L2BEAT, “ZK catalog: RISC Zero,” ZK Catalog; zero-knowledge property assessment citing Habock and Al Kindi, 2025. [Online]. Available: <https://l2beat.com/zk-catalog/risc0>
 - [49] Succinct Labs, “Proof types — SP1 documentation,” SP1 Documentation, 2025, accessed 2026-06-01. [Online]. Available: <https://docs.succinct.xyz/docs/sp1/generating-proofs/proof-types>
 - [50] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, ser. Lecture Notes in Computer Science, vol. 9666. Springer, 2016, pp. 305–326, cryptology ePrint Archive, Paper 2016/260. [Online]. Available: <https://eprint.iacr.org/2016/260>
 - [51] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge,” Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: <https://eprint.iacr.org/2019/953>
 - [52] StarkWare Team, “ethSTARK documentation,” Cryptology ePrint Archive, Paper 2021/582, 2021. [Online]. Available: <https://eprint.iacr.org/2021/582>
 - [53] J. Avigad, L. Goldberg, D. Levit, Y. Seginer, and A. Titelman, “A verified algebraic representation of Cairo program execution,” in *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2022, <https://arxiv.org/abs/2109.14534>.
 - [54] Kardas, Kiraz, Savonin, Wang, and Dziadziuk, “Performance improvements of ZK-prover for rWasm: A sound and efficient AIR for 32-bit division and remainder,” Cryptology ePrint Archive, Paper 2025/2220, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2220>

A Formal Development of the Security Preservation

The body (Section 6) states the security-preservation theorem (Theorem 6.3) and sketches its proof. This appendix gives the formal development for audit: the building-block properties the reduction treats as black boxes, each carrying an error term; the relation-preservation lemma; the step-by-step reductions; a corollary for the case in which the open premises are closed; and a remark relating the result to BDEC. Throughout, $\mathcal{A}$ denotes a classical probabilistic polynomial-time adversary and all probabilities are in the random-oracle model. The advantages $\text{Adv}_{\text{zkVM-BDEC}}^\bullet$ are those of Definition 6.1.

A.1 Building-block properties

Each property is stated as the existence of an algorithm together with an error term. The premises of Theorem 6.3 assert the relevant errors negligible; we mark which properties are assumed and which carry an open premise.

Definition A.1 (Knowledge soundness). The zkVM argument Π is *knowledge-sound with knowledge error* ϵ_{KS} if there is a probabilistic polynomial-time extractor $\mathcal{E}$ with oracle access to the prover such that, for every prover P^* that makes $\Pi.\text{Verify}(x, \cdot)$ accept with probability ε , the extractor $\mathcal{E}^{P^*}(x)$ outputs a witness w with $R(x; w) = 1$ with probability at least $\varepsilon - \epsilon_{\text{KS}}$. This is the standard knowledge-soundness-with-error formulation; ϵ_{KS} is the additive extraction-failure term that enters the unforgeability bound (1), and the concrete soundness error of the STARK and BCS arguments we rely on is treated in §B.5. (Assumed; classical random-oracle model.)

Definition A.2 (Arithmetisation soundness error).

$$\epsilon_{\text{AIR}} = \Pr[\text{the Griffin AIR accepts a trace whose committed transition differs from } \pi]$$

, the probability taken over the prover-chosen trace and the argument verifier’s randomness, where π is the reference permutation of Definition B.1. Premise (T2) asserts ϵ_{AIR} negligible; the supporting argument is Appendix B.

Definition A.3 (Signature-transcript simulatability). PLUM *admits a signature-transcript simulator with error* ϵ_{sZK} if there is a probabilistic polynomial-time algorithm $\mathcal{S}_\sigma$ such that, for every message m and public key, the output of $\mathcal{S}_\sigma$ run without the secret key is ϵ_{sZK} -indistinguishable from a credential or pseudonym transcript produced with the secret key. Premise (T4) asserts ϵ_{sZK} negligible. This premise is *open*: PLUM’s analysis establishes EU-CMA security, which does not by itself furnish such a simulator.

Definition A.4 (Zero knowledge). The zkVM argument Π is *zero-knowledge with error* ϵ_{ZK} if there is a probabilistic polynomial-time simulator $\mathcal{S}_\Pi$ such that, for every (x, w) with $R(x; w) = 1$, the simulated proof $\mathcal{S}_\Pi(x)$ is ϵ_{ZK} -indistinguishable from an honest proof $\Pi.\text{Prove}(x, w)$. (Assumed for the analysis. In the measured deployment this premise is not met: the produced receipt mode (the default `prove` path) has no established ZK simulator, and the ZK-wrapped mode (`.compressed().groth16()`) is not produced within the resource and memory bound; see Section 6.3.)

Definition A.5 (Existential unforgeability). PLUM is *EU-CMA secure with error* ϵ_{EUF} if every probabilistic polynomial-time forger, given the public key and a signing oracle, produces a valid signature on a message it never queried with probability at most ϵ_{EUF} . (Established in PLUM’s analysis, [5, Theorem 1], in the classical random-oracle model.)

A.2 Relation preservation

Proof of Lemma 6.2. We account for the operations a guest evaluation invokes. The credential-generation relation $R_{\text{cre}}^{\text{Sig}}$, two PLUM verifications under pk_U , decomposes into large-field multiplications, returned identically by the modular-multiplication precompile, and Griffin permutations inside PLUM’s hashing and inside every Merkle-path and low-degree-test recomputation, returned identically by the Griffin precompile under the lookup binding, together with native control flow, whose faithful execution follows from the *soundness* of the zkVM argument for the RISC-V execution relation (an accepting proof certifies a valid execution); the stronger *knowledge-soundness* (Definition A.1) that extracts a witness is invoked separately, by the unforgeability reduction. For $R_{\text{cre}}^{\text{Sig}}$ this enumeration is complete at the predicate level, conditional on the open statement-binding refinement of Section 5 and on premise (T2). The presentation relation $R_{\text{show}}^{\text{Sig}}$ adds only $k + 2$ further signature-verification predicates built from the same multiplications and Griffin permutations; its ledger-membership, non-revocation, and disclosure checks are performed by the relying party outside the arithmetised relation. Composing the per-operation equalities along the finite, input-fixed execution yields equality of the accept-or-reject decision at the program level. This program-level equality lifts to credential-verifier acceptance only when the zkVM journal commits the public statement x ; the specification assumes this binding, whereas the current RISC Zero prototype commits the verification outcome but not x_{cre} itself (Section 5), so the code-level reading is conditional on that journal fix. The Griffin step’s constraint-level soundness rests on Appendix B, which is argued rather than machine-verified, so the equality holds modulo premise (T2). $\square$

A.3 Proof of the preservation theorem

We re-instantiate BDEC’s three reductions [1] with PLUM as the signature scheme Sig and the zkVM as the zkSNARK Π ; by Lemma 6.2 the two provers accept the same relation.

Proof of Theorem 6.3, unforgeability. Let $\mathcal{A}$ win the zkVM-BDEC unforgeability experiment. We build a PLUM EU-CMA forger $\mathcal{B}$. $\mathcal{B}$ runs $\mathcal{A}$, answering its issuance and showing queries with its own signing oracle and the zkVM prover, so $\mathcal{A}$ ’s view matches the real experiment. When $\mathcal{A}$ outputs an accepting (x^*, π^*) for a statement outside the issued set, $\mathcal{B}$ invokes the extractor $\mathcal{E}$ of Definition A.1 to obtain $w^* = (\text{pk}_U^*, \text{psk}^*)$, which fails with probability at most ϵ_{KS} . Conditioned on extraction, w^* satisfies $R_{\text{cre}}^{\text{Sig}}$ except when the committed Griffin transition differs from the reference permutation, an event of probability at most ϵ_{AIR} by Definition A.2; otherwise the extracted psk^* (the pseudonym secret, structurally a PLUM signature under pk_U , Section 3) is valid on a message $\mathcal{A}$ never had signed, and by premise (T3) the precompile substitution leaves PLUM’s EU-CMA reduction unchanged, so psk^* is a forgery against the unmodified scheme, which $\mathcal{B}$ outputs. Writing ϵ_{EUF} for PLUM’s EU-CMA advantage bound,

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{unf}}(\mathcal{A}) \leq \epsilon_{\text{EUF}} + \epsilon_{\text{KS}} + \epsilon_{\text{AIR}},$$

which is (1). $\square$

Proof of Theorem 6.3, anonymity. Let $\mathcal{A}$ be the anonymity distinguisher, with advantage $|\Pr[b' = b] - \frac{1}{2}|$. We proceed through a sequence of games on the challenge showing for the credential cred_b . Game G_0 is the real experiment. In game G_1 the pseudonym key and credential are produced by $\mathcal{S}_\sigma$ rather than with the secret key, so by Definition A.3 $|\Pr[G_0=1] - \Pr[G_1=1]| \leq \epsilon_{\text{sZK}}$. The role of

$\mathcal{S}_\sigma$ is to let the reduction produce the challenge credential c_b and pseudonym signature psk_b without the challenge user's long-term secret key sk_U , matching BDEC's anonymity simulator ([1, Appendix A.2]); the hop is load-bearing whenever PLUM's transcripts cannot be produced without sk_U . In game G_2 the proof is produced by $\mathcal{S}_\Pi$ rather than from the witness, so by Definition A.4 $|\Pr[G_1=1] - \Pr[G_2=1]| \leq \epsilon_{\text{ZK}}$. In G_2 we account for every b -dependent component of the challenge statement $x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V})$. The disclosed attribute set $A^\downarrow$ is equalised across the two credentials by the well-formedness condition $\text{Leak}(\text{stmt}_0) = \text{Leak}(\text{stmt}_1)$. The verifier-facing pseudonym $\text{ppk}_{U,V}$ is sampled freshly at showing time, $\text{ppk}_{U,V} \leftarrow \{0,1\}^\lambda$, independently of which credential is shown, so its distribution is uniform regardless of b . The teaching-authority pseudonyms $(\text{ppk}_{U,TA}^{(j)})_j$ are issuance-time, credential-bound values that the well-formedness condition equalises across the two challenge credentials, so Leak includes the multiset $\{\text{ppk}_{U,TA}^{(j)}\}$ (Section 3). The secret keys, the long-term key pk_U , and the credential $c_{U,V}$ are witness-only: they were replaced by $\mathcal{S}_\sigma$ in G_1 and removed from the proof by $\mathcal{S}_\Pi$ in G_2 . Every b -dependent component being equalised, freshly sampled, or simulated, the challenge is distributed identically under $b = 0$ and $b = 1$, so $\Pr[b' = b] = \frac{1}{2}$. Summing the two hops,

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{anon}}(\mathcal{A}) \leq \epsilon_{\text{ZK}} + \epsilon_{\text{sZK}},$$

which is (2). No witness is extracted, so ϵ_{KS} does not enter; the challenge showing is honestly generated, so ϵ_{AIR} does not enter. $\square$

Proof of Theorem 6.3, unlinkability. BDEC's unlinkability experiment ([1, Appendix A.3]) forwards a single challenge credential $c^{(b)}$ issued under one of two pseudonym keys $\text{ppk}^{(b)}$, with the verifier-facing pseudonym freshly sampled as in the anonymity step. BDEC's A.3 simulator simulates the pseudonym signature and the credential with $\mathcal{S}_\sigma$ and the proof with $\mathcal{S}_\Pi$. Applying the same two hops to the single transcript, $\mathcal{S}_\sigma$ for $(\text{psk}^{(b)}, c^{(b)})$ (cost ϵ_{sZK}) and $\mathcal{S}_\Pi$ for the proof (cost ϵ_{ZK}), makes it independent of b , so

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{unlink}}(\mathcal{A}) \leq \epsilon_{\text{ZK}} + \epsilon_{\text{sZK}}.$$

The single-transcript game incurs each simulation once; there is no factor of two. One difference of presentation: BDEC's A.3 carries $c^{(b)}$ as a public statement component, so its $\mathcal{S}_\sigma$ hop is forced; in this thesis's relation $R_{\text{show}}^{\text{Sig}}$ the credential $c_{U,V}$ is a witness component (Section 3), so the $\mathcal{S}_\Pi$ hop already hides it and $\mathcal{S}_\sigma$ is needed only to produce $c^{(b)}$ without sk_U inside the reduction. Either way ϵ_{sZK} enters once. $\square$

Corollary A.6 (Preservation under closed premises). *Suppose premises (T2) and (T4) hold, so that ϵ_{AIR} and ϵ_{sZK} are negligible; the zkVM is knowledge-sound and zero-knowledge with negligible $\epsilon_{\text{KS}}, \epsilon_{\text{ZK}}$; and PLUM is EU-CMA secure. Then zkVM-BDEC is unforgeable, anonymous, and unlinkable. Against quantum adversaries the same conclusion holds once the quantum-random-oracle lifts of PLUM's EU-CMA reduction and of the nested Fiat–Shamir compilation are established.*

Proof. Each bound of Theorem 6.3 is a sum of the named error terms; under the stated hypotheses every term is negligible, so each advantage is negligible. The quantum clause adds the two open lifts as further hypotheses. $\square$

Remark A.7 (Relation to BDEC). Theorem 6.3 re-instantiates BDEC's Theorems 1–3, proved for a generic (signature, zkSNARK) pair, with the concrete pair (PLUM, zkVM). The contribution is not a

new reduction but the explicit accounting of the substrate-specific errors ϵ_{AIR} (the precompile’s arithmetisation soundness) and $\epsilon_{\text{KS}}, \epsilon_{\text{ZK}}$ (the zkVM’s argument), together with the identification of ϵ_{sZK} as the open premise (T4) that BDEC’s generic anonymity argument silently requires of the signature. We further note that the relation $R_{\text{show}}^{\text{Sig}}$ places the credential $c_{U,V}$ in the witness rather than, as in BDEC’s literal layout, the public statement; the result is therefore a re-derivation of BDEC’s Theorems 1–3 for this concrete pair, not a verbatim re-instantiation.

B Evidence for Premise (T2): Specification-Level Soundness of the Griffin- $\mathbb{F}_p^{192}$ AIR

This appendix develops the soundness argument for the Griffin- $\mathbb{F}_p^{192}$ permutation AIR of Section 5, on which the conditional precompile-soundness theorem (Theorem 5.1) and the relation-preservation lemma of Section 6 rest. We follow the standard practice for arithmetisation correctness. As in ethSTARK [52] (Definitions 3 and 4), the verified Cairo representation [53], and per-gadget AIR-soundness arguments such as [54], the formal object is a constraint-soundness statement, a universally quantified implication over satisfying traces, together with a separate completeness statement, while the knowledge-soundness of the surrounding argument is treated apart in Section B.5. The malicious prover is captured by the quantifier “for every trace that satisfies the constraints”, so the statement uses no challenger, oracle, or advantage function. The one component that is supported empirically rather than proved, namely that the implemented chip realises the constraint set analysed here, appears as the explicit premise of the soundness theorem and is discussed in Section B.6.

B.1 Setup

Definition B.1 (Reference permutation). Let $\pi: \mathbb{F}_p^4 \rightarrow \mathbb{F}_p^4$ denote one application of the Griffin permutation at PLUM’s parameters, a degree-3 S-box over a width-4 state with 14 rounds and the SHAKE256-derived round constants and base quadratic coefficients (α, β) , scaled per lane by Griffin’s index law, fixed in Section 3, as computed by the reference implementation.

Definition B.2 (Algebraic satisfiability). The Griffin AIR is a finite constraint set $\mathcal{C} = \{(Q_i, H_i)\}_i$ over a trace w comprising 14 per-round rows and one controller row, where each Q_i is a constraint polynomial and H_i its enforcement domain. A trace w *satisfies* $\mathcal{C}$ on input $s \in \mathbb{F}_p^4$ with claimed output $s' \in \mathbb{F}_p^4$ if its first-row input lanes decode to s , its last-row output lanes decode to s' , and every constraint polynomial Q_i vanishes on its enforcement domain H_i under w .

B.2 Soundness

Theorem B.3 (Specification-level AIR soundness). *Suppose the implemented chip, comprising the per-round chip together with its sibling controller chip, enforces the constraint set $\mathcal{C}$ analysed below. Then for every input $s \in \mathbb{F}_p^4$ and every trace w satisfying $\mathcal{C}$ with claimed output s' , we have $s' = \pi(s)$. Equivalently, $\mathcal{C}$ admits no satisfying trace whose committed output differs from the reference permutation.*

Proof. By Lemmas B.4 and B.5. Lemma B.4 fixes, for each round, the non-linear, linear, and round-constant layers of w to equal the reference round's outputs on its inputs, and Lemma B.5 forces the 14 rows to compose as 14 sequential rounds on a single state, bound on the controller row to the syscall's memory input and output. Composing the per-round equalities of Lemma B.4 along the schedule fixed by Lemma B.5 yields $s' = \pi(s)$. The composition is unconditional given the two lemmas, and the status of the chip-constraint equality (premise (T2)) is discussed in Section B.6. $\square$

B.3 Per-family lemmas

The ten constraint families group into the round function (B-1 through B-5) and the schedule, binding, and canonicity apparatus (B-6 through B-10). Each clause states the reference relation a family enforces and, in contrapositive, the constraint a deviating trace must violate.

Lemma B.4 (Round-function soundness; families B-1 through B-5). *Fix a round with input lanes $s = (s_0, s_1, s_2, s_3)$ and let s' be the trace's claimed round output. Under constraint families B-1 through B-5, every trace satisfying $\mathcal{C}$ has $s' = \rho(s)$, where ρ denotes one reference Griffin round.*

Proof. For each family we exhibit the constraint polynomial it enforces on the round domain and observe that any deviating value leaves that polynomial non-vanishing, hence unsatisfying; equality of the surviving assignment with $\rho(s)$ then follows by composing the layers. *Forward S-box* (lane 1, family B-1). The chip enforces $Q_{sq} = sq - s_1^2$ and $Q_{cube} = cube - sq \cdot s_1$, both vanishing on the round domain, so the lane-1 nonlinear output is $cube = s_1^3 \bmod p$; any output $\neq s_1^3$ leaves Q_{cube} non-zero. *Inverse S-box* (lane 0, family B-2). The chip enforces $Q_{inv} = s_0 - (s_0^{nl})^3$, witnessing the cube root in the backward direction; since $\gcd(3, p-1) = 1$ (Remark 3.5, verified for the implemented modulus in Section 5), the cube map is a bijection on $\mathbb{F}_p$, so the witnessed s_0^{nl} is the unique cube root and equals the forward inverse S-box on s_0 . *Quadratic layer* (lanes 2, 3, family B-3). For $i \in \{2, 3\}$ the chip enforces $Q_i = s_i^{nl} - s_i (L_i^2 + \alpha_{i-2} L_i + \beta_{i-2})$, where $L_i = (i-1) s_0^{nl} + s_1^{nl} + s_{i-1}^{nl}$ reads the two updated S-box lanes (s_0^{nl} the inverse-S-box lane, s_1^{nl} the forward-S-box lane) and the preceding updated quadratic lane s_{i-1}^{nl} ; for $i = 2$ the third summand is absent, so $L_2 = s_0^{nl} + s_1^{nl}$. The quadratic coefficients follow Griffin's index law: a single base pair (α, β) is read from the SHAKE256 seed, and $\alpha_{i-2} = (i-1)\alpha$, $\beta_{i-2} = (i-1)^2\beta$ (so $\alpha_0 = \alpha$, $\beta_0 = \beta$ for $i = 2$ and $\alpha_1 = 2\alpha$, $\beta_1 = 4\beta$ for $i = 3$). The base (α, β) are the first two field elements read from SHAKE256 seeded by the domain string `PlumGriffin(p, 4, 2, 128)`, the same seed that produces the 52 round constants; a wrong α, β , a wrong index scaling, or an off-by-one in L_i leaves Q_i non-zero. *MDS layer* (family B-4). Per output lane j the chip enforces $Q_j^{mds} = s'_j - \langle \text{circ}(2, 1, 1, 1)_j, s^{nl} \rangle$, where $\text{circ}(2, 1, 1, 1)_j$ is the symmetric circulant row `first[(j+col) mod 4]` matching `build_matrix`, not the standard $(\text{col} - j) \bmod 4$ form (the two differ on rows 1, 3); any other matrix row leaves some Q_j^{mds} non-zero. *Round constants* (family B-5). Per lane ℓ the chip enforces $Q_\ell^{rc} = s'_\ell - (\text{MDS output})_\ell - \text{RC}[4r + \ell]$ under one-hot round selection r , with RC preprocessing-bound (family B-10); a substituted or omitted constant leaves Q_ℓ^{rc} non-zero. The preprocessing table holds $\text{STATE_WIDTH} \times (\text{rounds} - 1) = 52$ constants, applied to the first 13 rounds; the final round applies no round-constant layer, so its constraint is $Q_\ell^{rc} = s'_\ell - (\text{MDS output})_\ell$.

Composing B-1 through B-5 along the round, the unique satisfying assignment of s' is the reference round $\rho(s)$. $\square$

Lemma B.5 (Composition, binding, canonicity; families B-6 through B-10). *Under constraint families B-6 through B-10, every trace satisfying $\mathcal{C}$ realises its 14 rows as the 14 sequential reference rounds on a single state bound to the syscall's memory input and output, with canonical output lanes.*

Proof. *Round count* (family B-6). The schedule is fixed to 14 rounds by the trace-generation row count and the controller’s lookup anchors at directions 0 and NB_ROUNDS, rather than by a dedicated algebraic constraint; under the lookup binding (premise (T1)) a trace with a different round count produces an unbalanced lookup and has no satisfying assignment. *Memory binding* (family B-7). The controller chip constrains the input and output state lanes to the values at the syscall’s memory addresses and clock, a memory-consistency lookup constraint resolved under premise (T1), so a trace reading or writing a different address or clock produces an unbalanced lookup. *Output canonicity* (family B-8). Each output lane s'_j carries the range constraint $0 \leq s'_j < p$ enforced by a byte-wise less-than comparison (FieldLtCols); a non-canonical lane violates it. *Row scheduling* (family B-9). The real/padding and first/last-round selector flags enforce one ordered run with cross-row state threading $\text{state_out}_r = \text{state_in}_{r+1}$, realised by the per-row lookup interactions (a RECEIVE at direction r and a SEND at direction $r+1$) under the lookup binding (premise (T1)); a ghost or mis-ordered row creates an unbalanced lookup. *Parameter immutability* (family B-10). The modulus, round count, MDS entries, round constants, and quadratic coefficients are preprocessing-bound rather than prover-supplied, so a per-row choice of any of them has no satisfying assignment.

By B-9, and given the lookup binding (T1), the per-round equalities of Lemma B.4 chain into 14 sequential rounds on one state; with the round count fixed by the lookup schedule (B-6, premise (T1)) and with B-8 and B-10, this is the reference permutation, and with B-7 it is bound to the syscall’s memory input and output. $\square$

B.4 Completeness

Lemma B.6 (Completeness). *For every input $s \in \mathbb{F}_p^4$, the honest Griffin execution on s yields a trace satisfying $\mathcal{C}$ with output $\pi(s)$.*

Proof. Following the practice of [53], where soundness is the load-bearing direction and completeness is discharged operationally, we support the claim by the cross-codebase equivalence tests: the trace generator populated from the reference computation produces a satisfying assignment on every tested input, namely the 256-vector randomised suite, the hand-picked and zero/high-entropy vectors, and the single-permutation smoke proof that completes execute, prove, and verify (Section 5). $\square$

B.5 The argument layer

Theorem B.3 concerns the *arithmetisation*. The surrounding claim, that an accepting receipt implies a satisfying trace and that a witness is extractable, is the *knowledge-soundness* of the zkVM’s STARK, a separate object we do not re-establish. We take it in the IOP and extractor sense of ethSTARK [52] (Definition 4): for every prover producing an accepting proof with non-negligible probability there exists an extractor outputting a satisfying trace, up to a negligible knowledge error. This is the knowledge-soundness hypothesis of the preservation theorem of Section 6, itself obtained through the Fiat–Shamir and BCS compilation of an interactive oracle proof. The arithmetisation soundness of this appendix and the argument knowledge-soundness are independent objects. End-to-end soundness needs both, and we keep them separate.

B.6 The open step

The compositional argument of Theorem B.3 is unconditional given Lemmas B.4 and B.5 and the premise that the implemented chip enforces exactly the analysed constraint set $\mathcal{C}$. That premise is the one step we do not machine-verify. The evidence for it is the cross-codebase agreement of the chip’s native compute with the reference permutation on 256 random vectors, with per-vector false-pass at most $1/p \approx 2^{-199}$ and aggregate at most 2^{-191} across the suite, together with the hand-picked and zero/high-entropy vectors. This per-vector bound assumes a wrong executor produces outputs uniform over $\mathbb{F}_p$ and independent of the reference; a deterministic structural bug agreeing on every sampled input would not be caught, so 2^{-191} bounds only random disagreement on the sampled set and is *not* a bound on the open constraint-level premise. The suite pins the *executor* against the reference, but the *constraint system* checked against that executor is what the vector suite does not reach. A separate fault-injection suite reaches the constraint system in the *rejection* direction for eight of the ten families (B-1 through B-5, B-8, and the per-row B-9 selectors): a single-cell corruption of a satisfying trace is rejected by the per-row constraints in each case, including the MDS lanes 1 and 3 that distinguish the implemented symmetric circulant from the standard one. This evidence is itself bounded, and we state the bounds. It is evaluated through `debug_constraints`, which checks the per-row algebraic constraints but not the cross-table lookups, so the lookup-borne families (round count B-6, memory binding B-7, and the cross-row threading of B-9) lie outside its reach and would require a full-prover harness; parameter immutability (B-10) is enforced by construction, with no prover-chosen column to reject. The suite thus shows that specific local deviations are caught, not that no wrong witness satisfies $\mathcal{C}$, which remains the open premise. Closing it fully requires deriving the AIR witness directly from the reference code and checking, ideally by machine, that $\mathcal{C}$ admits only that witness. We therefore report Theorem B.3 as argued at the specification level and supported empirically at the implementation level, with the chip–constraint equality carried as the open premise (T2) of both this theorem and the conditional precompile-soundness theorem Theorem 5.1.